

סיכום מושגים וטענות למבחן באלגוריתמים 1

8 בפברואר 2026

גיא יער-און

יתכנו שנפלו טעויות, ואם כן אשמח לדעת: בהצלחה לכולנו!

1 FFT

פולינום יסומן כך: $P(x) = \sum_{i=0}^{n-1} a_i x^i$. פולינום זה הוא מדרגה $n-1$ והוא מדרגה חסומה k לכל $k \geq n$.

פעולות על פולינומים:

1. חיבור/חיבור פולינומים: $C(x) = A(x) \pm B(x) = \sum_{i=0}^{n-1} (a_i \pm b_i) x^i$
2. כפל פולינומים: $C(x) = A(x) \times B(x) = \sum_{i=0}^{2n-2} c_i x^i$ כאשר $c_i = \sum_{j=0}^i a_j b_{i-j}$. לפעולה זו קוראים קונבולוציה.
3. חישוב ערך: יהי פולינום $A(x) = \sum_{i=0}^{n-1} a_i x^i$, בהינתן ערך x_0 , נרצה לחשב את $A(x_0)$.

ייצוגים של פולינומים:

- א. ייצוג ע"י מקדמים. יהי מערך $A = (a_0, \dots, a_{n-1})$ כך ש- $A[i]$ הוא המקדם ה- i של הפולינום. חיבור וחיסור של פולינומים בשיטה זו בעלות $O(n)$ באופן טריוואלי, כפל באמצעות קונבולוציה בזמן $O(n^2)$ בצורה נאיבית. חישוב ערך בצורה נאיבית בזמן $O(n)$.
- ב. ייצוג ע"י נקודות. מייצגים את הפולינום A מדרגה חסומה $n-1$ ע"י n נקודות $(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})$ כך שערך x שונה לכל נקודה.

$$\begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} x_0^0 & x_0^1 & x_0^2 & \dots & x_0^{n-1} \\ x_1^0 & x_1^1 & x_1^2 & \dots & x_1^{n-1} \\ x_2^0 & x_2^1 & x_2^2 & \dots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{n-1}^0 & x_{n-1}^1 & x_{n-1}^2 & \dots & x_{n-1}^{n-1} \end{pmatrix} \times \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix}$$

יצוג זה מניב את מטריצת ונדרמונדה, שהפיכה, ומניב את מערכת המשוואות $\vec{Y} = V \times \vec{a}$. כיוון שמטריצת ונדרמונדה V הפיכה יתקיים $\vec{a} = V^{-1} \times \vec{b}$.

מחברים בשיטה זו בדרך נאיבית, כלומר בהינתן ייצוג ע"י נקודות מגדירים את החיבור כך $C(x_i) = A(x_i) + B(x_i) \forall 0 \leq i \leq n-1$ בזמן $O(n)$. אם הפולינומים לא מיוצגים ע"י אותם נקודות - עלינו לבצע אינטרפולציה בכל מקרה, לחבר ע"י מקדמים, ולחזור לייצוג ע"י נקודות בזמן $O(n^2)$. אם הפולינום מיוצג ע"י אותם נקודות (הערה: הם צריכים להיות מיוצגים ע"י $2n-2$ נקודות על מנת שהכפל יוגדר טוב) אזי פולינום הכפל יוגדר כך $C(x_i) = A(x_i) \times B(x_i) \forall 0 \leq i \leq 2n-1$ ועלותו לחישוב $O(n)$. כאשר לחישוב ערך - בכל מקרה עלינו לבצע אינטרפולציה ולעבור לייצוג ע"י מקדמים, ושם לחשב ערך. (פרט למקרה שבו הערך שאנחנו רוצים לחשב הוא אחד הערכים שיש אצלנו, ואז זמן

הריצה לחיפוש בהינתן שהנקודות מסודרות בסדר עולה הוא כמובן $(O(\log n))$.

מעברים בין ייצוגים:

א. מעבר ממקדמים לנקודות: ע"י n פעולות של חישוב ערך. נבחר $x_0 \neq x_1 \neq \dots \neq x_{n-1}$ ונחשב $A(x_0), \dots, A(x_{n-1})$. כמה יעלה מעבר זה? כל חישוב עולה $O(n)$ ולכן סה"כ n חישובים יעלו לנו $O(n^2)$.

ב. מעבר מנקודות למקדמים: לפעולה זו יש שם - אינטרפולציה. משתמשים בנוסחת לגראנז':

$$A(x) = \sum_{i=0}^{n-1} y_i \cdot \frac{\prod_{j \neq i} (x - x_j)}{\prod_{j \neq i} (x_i - x_j)}$$
 נשים לב כי חישוב זה יעלה $O(n^2)$.
מבוא ל-FFT:

הגדרה: נגדיר את הפולינומים הבאים.

$$A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + \dots + a_{\frac{n}{2}-2}x^{\frac{n}{2}-2}$$

$$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + \dots + a_{\frac{n}{2}-1}x^{\frac{n}{2}-1}$$

משפט 1. מתקיים $A_{\text{even}}(x^2) - xA_{\text{odd}}(x^2) = A(-x)$ וכן $A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2) = A(x)$

הגדרה 2. (תכונת הנגדיות החלשה): יהי k חזקה של 2. לסדרת ערכי האינס: $(x_0, x_1, \dots, x_{k-1})$ יש את תכונת הנגדיות החלשה אם אחד מהתנאים הבאים מתקיים:

$$k = 1$$

$$\forall_{0 \leq j \leq \frac{k}{2}-1} : x_{\frac{k}{2}+j} = -x_j$$

הגדרה 3. (תכונת הנגדיות החזקה): יהי k חזקה של 2. לסדרת ערכי האינס: $(x_0, x_1, \dots, x_{k-1})$ יש את תכונת הנגדיות החזקה אם אחד מהתנאים הבאים מתקיים:

$$k = 1$$

ב. לסדרה יש את תכונת הנגדיות החלשה, וגם לסדרה $(x_0^2, x_1^2, \dots, x_{\frac{k}{2}-1}^2)$ יש את תכונת הנגדיות החזקה (באופן רקורסיבי).

הגדרה 4. (שורשי היחידה): מס' ω יקרא שורש יחידה מרוכב מסדר n אם $\omega^n = 1$. המס' ω_n יוגדר להיות: $\omega_n = e^{\frac{2\pi i}{n}}$. שורשי היחידה מסדר n הינם: $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$.

משפט 5. לכל $0 \leq k \leq n-1$ מתקיים כי $(\omega_n)^k$ הוא שורש יחידה מסדר n .

משפט 6. יהי $n > 1$ חזקה של 2. אזי לכל $0 \leq k \leq \frac{n}{2}-1$ מתקיים $\omega_n^{\frac{n}{2}+k} = -\omega_n^k$.

משפט 7. יהי $n > 1$ חזקה של 2. אזי הריבועים של $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$ הם בדיוק $\frac{n}{2}$ שורשי היחידה מסדר $\frac{n}{2}$. (כלומר, נעלה אותם בריבוע, נקבל בדיוק את שורשי היחידה מסדר $\frac{n}{2}$, וכל אחד מהם יופיע פעמיים - כלומר יהיו כפילויות).

משפט 8. יהי $n \geq 1$ חזקה של 2. סדרת n שורשי היחידה מסדר n : $\omega_n^0, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-1}$ מקיימים את תכונת הנגדיות החזקה.

האלגוריתם FFT

קלט: $A = (a_0, a_1, \dots, a_{n-1})$ (מערך המקדמים של הפולינום A) מדרגה חסומה n
פלט: ערך הפולינום ב n שורשי היחידה מסדר n .

$$:FFT(A = (a_0, \dots, a_{n-1}))$$

א. אם $n = 1$, החזר a_0 .

ב. כעת נגדיר את $A_{even} = (a_0, a_2, \dots, a_{n-2})$ מדרגה חסומה $\frac{n}{2}$

ג. כעת נגדיר את $A_{odd} = (a_1, a_3, \dots, a_{n-1})$ מדרגה חסומה $\frac{n}{2}$

ד. כעת נתחיל את הרקורסיה: $P_{even} = FFT(A_{even})$, כאשר P_{even} יפעיל את FFT על A_{even} - כלומר מחשבים את A_{even} ב n שורשי היחידה מסדר $\frac{n}{2}$.
כלומר, מה שחוזר מהרקורסיה הינו $P_{even} = [A_{even}(\omega_{\frac{n}{2}}^0), A_{even}(\omega_{\frac{n}{2}}^1), \dots, A_{even}(\omega_{\frac{n}{2}}^{\frac{n}{2}-1})]$

ה. בדומה - $P_{odd} = FFT(A_{odd})$

ו. החל מ $j = 0$ עד $j = \frac{n}{2} - 1$ בצע: (כעת אנחנו רוצים לחשב את הפלט שלנו - מחזירים לבסוף וקטור $\vec{y}$ עם חישוב הערכים בהתאם לפי הנוסחאות שראינו)

$$1. y_j = P_{even}[j] + w_n^j \times P_{odd}[j]$$

(נשים לב כי $y_j = A(\omega_n^j)$ ולפי נוסחה שראינו $A(x) = A_{even}(x^2) + xA_{odd}(x^2)$, כמו כן ניתן להמירה בהתאם $A(\omega_n^j) = A_{even}(\omega_n^{j^2}) + \omega_n^j A_{odd}(\omega_n^{j^2})$, וכפי שראינו מתקיים $(\omega_n^j)^2 = \omega_{\frac{n}{2}}^j$ לפי טענה לעיל, ולכן $A_{even}(\omega_{\frac{n}{2}}^j) + \omega_{\frac{n}{2}}^j A_{odd}(\omega_{\frac{n}{2}}^j) = A(\omega_n^j)$ כלומר אם נסתכל על מערכי הקלט ששמרנו זה ממש שקול ל $(P_{even}[j] + w_n^j \times P_{odd}[j])$.

$$2. y_{\frac{n}{2}+j} = P_{even}[j] - w_n^j \times P_{odd}[j]$$

ז. כשהרקורסיה נגמרת - החזר את $(y_0, \dots, y_{n-1})$

סיבוכיות זמן הריצה: הנוסחה - $T(n) = 2T(\frac{n}{2}) + O(n) = O(n \log n)$ ולכן סה"כ זמן חישוב האלגוריתם שמקבל פולינום בשיטת המקדמים, וממיר אותם ל n נקודות ספציפיות (שורשי היחידה מסדר n) לפי שיטת הנקודות הוא $O(n \log n)$.

מעבר חזרה משיטת הנקודות לשיטת המקדמים

נשים לב כי אנחנו יודעים את ערכי x הנקודות שלנו, הם n שורשי היחידה מסדר n .
 נחזור למטריצת ונדרמונדה. נציב $x_0 = (\omega_n^0)^0 = 1, x_1 = (\omega_n^1)^1, \dots, x_{n-1} = (\omega_n^{n-1})^{n-1}$

$$\begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_n^1 & (\omega_n^1)^2 & \dots & (\omega_n^1)^{n-1} \\ 1 & (\omega_n^2)^1 & (\omega_n^2)^2 & \dots & (\omega_n^2)^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega_n^{n-1})^1 & (\omega_n^{n-1})^2 & \dots & (\omega_n^{n-1})^{n-1} \end{pmatrix} \times \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix}$$

משפט 9. כפל של מטריצת ונדרמונדה שמוגדרת ע"י n מספרים שזכרנו את תכונת הנגזרות החזקה, בוקטור $\vec{a}$ עולה $O(n \log n)$ (בדיוק אותו תהליך שעשה האלגוריתם)

משפט 10. תהליך כפל פולינומים:

- בידיו פולינומים $A(x), B(x)$ מיוצגים ע"י מקדמים.
- באמצעות $FFT(A, B)$ מקבלים את A, B מיוצגים ע"י n שורשי היחידה מסדר n .
- מכפילים את הפולינומים A, B בזמן $O(n)$ באמצעות שיטת הנקודות, כיוון שהם כעת מיוצגים ע"י אותם ערכים.
- מבצעים FFT^{-1} לחזרה משיטת הנקודות לשיטת המקדמים.
- ה. בידיו מקדמי הפולינום $C = AB$.

משפט 11. כפל פולינומים $A(x) = \sum_{i=0}^{n-1} a_i x^i, B(x) = \sum_{i=0}^{n-1} b_i x^i$ כאשר $m < n$ עלותו $O(n \log m)$.

הסבר: מחלקים את הפולינום A ל $\frac{n}{m}$ פולינומים מזרקה חסומה m , ומבצעים על כל אחד מהם FFT עם פולינום B בעלות $O(m \log m)$ וסה"כ זמן הריצה יהיה $O(n \log m) = O(\frac{n}{m} \times m \log m)$.

הגדרה 12. בעיית 3SUM:

קלט: מערך $A = [a_0, \dots, a_{n-1}]$ באשר $\forall 0 \leq i \leq n-1 : a_i \in \mathbb{R}$
 פלט: האם קיימים a_i, a_j, a_k שונים כך ש $a_k = a_i + a_j$

משפט 13. אם המספרים $a_i \in A$ חסומים בטווח כלשהו (בה"כ $[1, 10n^{1.5}]$) אזי זמן הריצה של $3SUM$ עלותו $O(n^{1.5} \log n)$

הוכחה: רעיון האלגוריתם יהיה לקודד את המערך A לפולינום מזרקה חסומה $10n^{1.5}$ כך שהמקדם a_i של x^i יהיה 1 אם $i \in A$ ו 0 אחרת. בשלב השני, האלגוריתם יבצע כפל של הפולינום בעצמו $A(x)^2$, ויחפש מיקום $1 \leq i \leq n$ בפולינום המכפלה $A^2(x) = c(x)$ בו $c_i \geq 2$ וגם המקדם a_i במערך A שווה 1. הנכונות נובעת מחוק החזקות $x^{i+j} = x^i x^j$ שכן הפולינום מבצע מכפלה ולמעשה מחשב סכומים של איברי המערך. כפל בריבוע למעשה יוצר סכומים של שני איברים מהמערך. כלומר, המקדם של x^k בפולינום A^2 הוא מס' הדרכים לכתוב $a + b = k$ כאשר $a, b \in A$. מתמטית, אם המקדם $c_i \geq 2$ אזי בהכרח מדובר במכפלה של שני מספרים שונים זה מזה. לבסוף כל שנותר לאלגוריתם לבצע הוא לעבור במערך הפקודי ולחפש מס' i בו האם הוא שווה לאחד מהמקדמים c_i .

הגדרה 14. מרחק האמינג: יהיו A, B שתי מחרוזות באורך זהה. אזי מרחק האמינג שלהם הוא $HD(A, B) = |\{i | A[i] \neq B[i]\}|$

הגדרה 15. בעיית חישוב מרחק האמינג בין טקסט לתבנית:

קלט: מחרוזת $T[1, \dots, n]$ באורך n "טקסט". ומחרוזת $P[1, \dots, m]$ באורך $m \leq n$ "תבנית".
 פלט: לכל היסט $0 \leq i \leq n - m$ את מרחק ההאמינג $HD(P, T[i+1, \dots, i+m])$

משפט 16. אלגוריתם לחישוב מרחק האמינג של טקסט מעל Σ עלותו $O(|\Sigma| \times (n \log m))$
 הוכחה: למעשה נראה את האלגוריתם. יהי Σ א"ב. נגזיר את הפולינומים הבאים:

$$T_\sigma(i) = \begin{cases} 1 & T[i] = \sigma \\ 0 & o.w \end{cases}, P_\sigma(i) = \begin{cases} 1 & P[i] = \sigma \\ 0 & o.w \end{cases}$$

לספירת מס' ההתאמות כשלב מקדים. עלות זמן הריצה של החלק הנ"ל עלותו $O(n + m)$ פר σ . ואז:

- צור מערך חדש M בגודל $n - m + 1$
- לכל $\sigma \in \Sigma$ ספור לכל היסט אפשרי של התבנית בטקסט את מס' ההתאמות של σ באמצעות FFT כך: $T_\sigma \times P_\sigma^R$
- חבר את התוצאות מההיסטים השונים למערך קלט אחד, בשם M , והגדר $M[i] = m - M[i]$ (שהרי ספרנו התאמות, ומרחק האמינג מוגדר ע"י אי התאמות).
- חזר את M .

נבחין כי באופן ברור כל אות קלט עלותה $O(n \log m) = O(n + m + n \log m)$ ולכן זמן הריצה הכולל תלוי ב Σ ועלותו $O(|\Sigma| \times n \log m)$.

MST 2

הנחה מקדימה ל-MST: G קשיר, אחרת לא קיים לו MST. ניתן לבדוק קשירות בזמן לינארי.

הגדרה 17. (MST). יהי $G = (V, E)$ מולטי גרף קשיר ממושקל עם פונקציית משקל על הקשתות $w : E \rightarrow \mathbb{R}$. יהי $T = (V, E_T)$ עץ פורש של G . אזי, נאמר שהמשקל של T הוא $w(T) = \sum_{e \in E_T} w(e)$. עץ פורש מינימום של G הוא עץ פורש שמשקלו הוא הקטן ביותר. כלומר, לכל עץ פורש T' מתקיים $w(T') \geq w(T)$.

הגדרה 18. הגדרת הבעיה:

קלט: גרף לא מכוון קשיר $G = (V, E)$ ופונקציית משקל $w : E \rightarrow \mathbb{R}$
 פלט: עץ פורש מינימום של G (ביחס לפונקציית משקל w).

הגדרה 19. (חתך) יהי $G = (V, E)$ גרף ויהי $S \subset V$ כך ש $S \neq \emptyset$. החלוקה $(S, V/S)$ נקראת חתך של G והקשתות $\{e = (u, v) | u \in S, v \in V/S\}$ נקראות קשתות שחוצות את החתך / קשתות בחתך.

הגדרה 20. (קשת קלה ביותר בחתך) יהי $G = (V, E)$ גרף לא מכוון עם פונקציית משקל $w : E \rightarrow \mathbb{R}$. יהי $(S, V/S)$ חתך של G . קשת $(u, v) \in E$ נקראת קשת קלה ביותר בחתך $(S, V/S)$ אם לכל קשת e' שחוצה את החתך מתקיים $w(e) \leq w(e')$. (נשים לב, יתכן שיש כמה קשתות כאלו באותו משקל שהן הקלות ביותר).

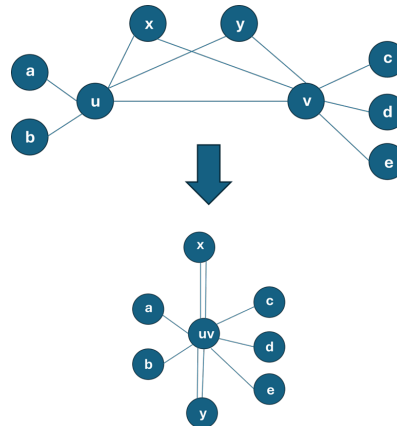
משפט 21. (למת הבחירה החמדנית): יהי $G = (V, E)$ מולטי גרף קשיר עם פונקציית משקל $w : E \rightarrow \mathbb{R}$. לכל $S \subset V$, $S \neq \emptyset$, לכל קשת e קלה בחתך $(S, V/S)$ קיים עץ"פ שמכיל את e .

הגדרה 22. כיווץ קשתות. יהי מולטי גרף $G = (V, E)$ לא מכוון, פעולת כיווץ על קשת $e = (u, v)$ מייצרת גרף חדש $G/e = (V/e, E/e)$ כך ש: $V/e = V \cup \{uv\} \setminus \{u, v\}$. נגדיר פונקציה: $F : V \rightarrow V/e$ כך ש:

$$F(x) := \begin{cases} x & x \neq v, u \\ uv & x = u \vee x = v \end{cases}$$

הפונקציה F ממפה את הקודקודים המקוריים, לקודקוד החדש שמייצג אותם. וקבוצת הקשתות תהיה: $E/e := \{(F(x), F(y)) | (x, y) \in E\}$

דוגמה 23. תהליך כיווץ על הקשת (u, v) :



הערה 24. נשים לב כי $(F(u), F(v)) = (uv, uv)$ וזו לולאה עצמאית. עם זאת - בגרף לא מכוון אסור לולאות עצמאיות. לכן, בתהליך הכיווץ נעלמת קשת אחת - אין לולאה עצמאית, הקשת שהייתה בין u לט איננה עוד.

משפט 25. אלגוריתם גנרי של MST :

$MST(G=(V,E),w)$:

1. $E_T = \emptyset$
2. while $|V| > 1$:
3. let e be the minimum weight edge of **some** cut in G
4. Add e to E_T
5. Contract e
6. Return E_T

הוכחה 26. (תכונות תת המבנה האופטימלי) יהי $G = (V, E)$ מולטי גרף קשיר, עם פונקציית משקל $w: E \rightarrow \mathbb{R}$. יהי $(S, V/S)$ חתך ב- G . ותהי $e = (u, v)$ קשת קלה בחתך $(S, V/S)$. יהי $T' = (V/e, E_{T'})$ עץ ע"פ עבור G/e . יהי $E_T = E_{T'} \cup \{e\}$, אזי, $T = (V, E_T)$ הוא עץ עבור G .

משפט 27. האלגוריתם של פריס:

הסבר. האלגוריתם של פריס מתמודד עם הבעיה - עם איזה חתך נתחיל בכל שלב. ברעיון זה בוחרים חתך בו בצד אחד קודקוד אחד ובצד השני שאר הקודקודים. כעת, בוחרים את הקשת הקלה ביותר בחתך, מכווצים אותה, ומתקדמים הלאה: עד שמקבלים קודקוד יחיד.

$Prim(G = (V, E), w)$:

$E_T = \emptyset$ 1.

2. עבור כל $u \in V$ בצע:

א. $u.key = \infty$ (המפתח יגיד את משקל הקשת הקלה ביותר בחתך שנוגעת ב- u ועוברת דרך החתך לצד השני)

ב. $u.\pi = null$ (אם u יגיד לנו מי הקודקוד בצד השני של הקשת, שהמשקל שלה הוא $u.key$)

3. בחר קודקוד שרירותי $r \in V$ ואתחל $r.key = 0$

4. $Q.init(V)$ - אתחל את תור הקדימויות.

כל עוד $|Q| \geq 1$ בצע:

א. $u = Q.extract.Min()$ (שקול לבחירת קשת קלה ביותר - מוציאים אותו מהתור)

ב. אם $u.\pi \neq null$ אזי $Add(u, u.\pi) \rightarrow E_T$

ג. עבור כל $v \in ADJ[u]$:

אם $v \in Q$ (אם עדיין בתוך התור) וגם $w(u, v) < v.key$ אזי -

1. $v.key = w(u, v)$

2. $v.\pi = u$

6. החזר E_T .

זמן הריצה של פריס: $O(|V| \log |V| + |E|)$.

הגדרה 28. האלגוריתם של קרוסקל. קרוסקל בוחר את הקשת הקלה בכל הגרף, ומוסיף אותה ל- MST . כיצד הוא דואג שלא יהיה מעגלים ולולאות עצמאיות? באמצעות $Union - find$:

$MST-KRUSKAL(G=(V,E),w)$:

1. $E_T = \emptyset$
2. for each $u \in V$:
 - a. make_set(u)
3. for every edge $e = (u, v) \in E$ in increasing order of weights
4. if find_set(u) $\neq$ find_set(v)

- a. Add (u,v) to E_T
- b. union (u,v)
5. return E_T

הסבר. האלגוריתם למעשה יוצר קבוצה לכל קודקוד, ועובר על הקשתות בסדר עולה (הוא ממיין אותן) - מכאן מגיע המינימום. כל קשת שמחברת בין שתי קבוצות שונות הוא מאחד לקבוצה אחת ומוסיף אותה לעץ - חייב אחרת העץ לא יהיה קשיר. לבסוף הוא מאחד את הקבוצות ומתקדם. לבסוף הוא מחזיר את העץ הפורש.

זמן הריצה: $O(|E| \times \alpha \times |V| + \text{Sort}(|E|))$.

משפט 29. תכונת המעגלים הכבדים של MST . יהי $G = (V, E)$ גרף לא מכוון עם פונקציית משקל על הקשתות $w : E \rightarrow \mathbb{R}$. יהי C מעגל ב- G כך ש- $e \in C$ היא קשת כבדה במעגל. אזי, קיים MST שלא מכיל את e .

הגדרה 30. אלגוריתם מחיקה כפולה:

$reserve - delete \text{ Algo}(G = (V, E), w)$

- א. מיין את E בסדר יורד, יהי הסדר לאחר המיין: $e_1, \dots, e_{|E|}$
- ב. לכל $i = 1$ עד $|E|$
1. מחק את e_i
2. אם הגרף ללא e_i אינו קשיר, החזר את e_i לגרף.
- ג. החזר את קבוצת הקשתות שלא נמחקו במהלך ריצת האלגוריתם.

משפט 31. לכל עץ $T = (V, E_T)$ של G קיים סדר של E שנסמנו π_T שהוא מיון של הקשתות ע"פ משקלן בסדר עולה, כך שההרצה של האלגוריתם של קרוסקל על G בהתאם ל- π_T תחזיר את T .
הערה חשובה. כאשר הוכחנו את הטענה (משפט 31) הסתכלנו על עץ כלשהו T של G והראינו עבורו סדר מיון כך שאלגוריתם קרוסקל מוציא את T כפלט. טעות נפוצה בשאלות מסוג זה היא נסיון להסתכל על עץ שהוא פלט של אלגוריתם קרוסקל (או כל אלגוריתם שפותר את הבעיה לצורך העניין) ונסיון לטעון טענות לגביו. נשים לב כי עץ של גרף הוא אובייקט מתמטי של הגרף, ויכולים להיות כמה עפמ"ס שונים. אלגוריתמים כמו פריס קרוסקל וכיוצא בזה תהליכים חישוביים שמוציאים אובייקט מתמטי שכזה (ביעילות), אך הוא ספציפי מבין כמה אפשריים, ולכן כאשר מתבקשים להוכיח טענה על אובייקט מתמטי שרירותי אסור לנו להניח שהוא פלט של אלגוריתם כזה או אחר (זהו מקרה פרטי של דגש הדורש מכס לשים לב כי ישנן שאלות בהן הטענה היא טענת לכל ולא טענת קיים, וכן להיפך).

משפט 32. מסקנה ממשפט 31: לפי הטענה אם ידוע כי G מכיל קשתות ממשקלים שונים, בהכרח קיים סדר מיון יחיד ולכן קיים MST יחיד!

3 $shorts \text{ path} - SSSP$

הגדרה 33. עבור $u, v \in V$ נסמן את העלות המינימלית של מסלול מ- u ל- v ב- $\delta(u, v)$. יהיה $P = (u, v_1, \dots, v_{k-2}, v)$ המסלול הקצר ביותר בין u ל- v (אם קיים). אזי,

1. אם G ממושקל: $\delta(u, v) = \sum_{v \in P} w(v)$
2. אם G אינו ממושקל: $\delta(u, v) = \text{length of } p$
3. אם לא קיים מסלול בין u ל- v נגדיר: $\delta(u, v) = \infty$

מסלול מ- u ל- v שעלותו $\delta(u, v)$ יקרא מסלול קצר ביותר. יתכן כמובן שישנם כמה מקב"ם.

הגדרה 34. בעיות $SSSP$:

1. **זוג קודקודים (Single Source) -** קלט: $G = (V, E)$ וזוג קודקודים $v, u \in V$. פלט: לחשב את $\delta(u, v)$ ואולי אף למצוא מסלול מ- u ל- v שהוא קצר ביותר.
2. **מקור יחיד (Singular Source Shortes Paths (SSSP)) -**

קלט: $G = (V, E)$ וקודקוד $s \in V$ שיקרא קודקוד מקור.
הפלט: לחשב עבור כל $v \in V$ את $\delta(s, v)$ ואולי גם למצוא מסלול קצר ביותר מ- s לכל $v \in V$.
3. **כל הזוגות (All Pairs Shortest Paths (APSP))** -
קלט: $G = (V, E)$
פלט: לכל $u, v \in V$ להחזיר את $\delta(u, v)$ ואולי אף למצוא את המסלול הקצר ביותר לכל $u, v \in V$.

הערה 35. כיצד נראה פתרון בכל אחת מהגרסאות?
זוג יחיד: מסלול $P = (u, v_0, \dots, v_{k-1}, v)$, שיעלה $O(|V|)$ זכרון.
מקור יחיד: עץ מסלולים קצרים ביותר (מבוסס על משפט 36). יתכנו כמה עצי מסלולים קצרים ביותר. עלות הזכרון תהיה $O(|V|)$. קודקוד שהינו ראש העץ יהיה בלולאה עצמית עם עצמו. קודקודים שאין אליהם מסלול יצביעו אל $null$.
כל הזוגות (APSP): במצב זה נרצה להחזיר מטריצה A בגודל $|V| \times |V|$, כשנרצה להחזיר את $\delta(v, u)$ אנחנו נייצג זאת במטריצה ע"י $A_{ij} = \delta(v_i, v_j)$ סה"כ יעלה $O(|V|^2)$ זכרון. אם נהיה מעוניינים במסלולים - נרצה $|V|$ עצי מסלולים קצרים ביותר, וסה"כ $O(|V|^2)$ מקום.

משפט 36. תת מסלול של מסלול קצר ביותר הוא גם מסלול קצר ביותר. כלומר, יהיה פסלול $P = (v, w_1, \dots, w_r, x, \dots, y, z_1, \dots, z_t, u)$ קצר ביותר. אזי, הפסלול בין x ל- y שמוכל במסלול P הוא גם הקצר ביותר.

הגדרה 37. אלגוריתם BFS מבצע סריקה לרוחב של העץ לפי רמות באמצעות תור קדימויות. האלגוריתם סורק את הצמתים בסדר שנקבע לפי מרחקם מהצומת ההתחלתי. אלגוריתם זה פותר $SSSP$ במקרה הלא ממושקל. **זמן ריצת האלגוריתם** $O(|V| + |E|)$ והוא **אופטימלי לבעיה**. האלגוריתם מאחסן את הפרטים הבאים לכל קודקוד:

1. $d[u]$ - אומדן לגבי $\delta(s, u)$. בסיום הריצה יתקיים $d[u] = \delta(s, u)$
 2. $\pi[u]$ - כלי עזר לחישוב המסלולים הקצרים ביותר מ- s . בסיום הריצה הוא מצביע לאבא של u בעץ המסלולים הקצרים.
 3. $Color[u]$ - מעין מזהה:
 - א. $w(hite) =$ האלגוריתם עוד לא ביקר ב- u .
 - ב. $g(ray) =$ האלגוריתם ביקר ב- u ולא טיפל בו.
 - ג. $b(lack) =$ האלגוריתם סיים לטפל ב- u .
- הערה 38. פסאודו קוד של $BFS(G = (V, E), s)$:


```

BFS( $G = (V, E), s$ )
1  for each  $u \in V$ 
2       $d[u] \leftarrow \infty$ 
3       $\pi[u] \leftarrow NULL$ 
4       $color[u] \leftarrow w$ 
5   $d[s] \leftarrow 0$ 
6   $color[s] \leftarrow g$ 
7   $Q.Enqueue(s)$ 
8  while  $Q \neq \emptyset$ 
9       $u \leftarrow Q.Dequeue()$ 
10     for each  $v \in ADJ[u]$ 
11         if  $color[v] = w$ 
12              $color[v] \leftarrow g$ 
13              $d[v] \leftarrow d[u] + 1$ 
14              $\pi[v] \leftarrow u$ 
15              $Q.Enqueue(v)$ 
16      $color[u] \leftarrow b$ 

```

משפט 39. למת אי שוויון המשולש. יהי $G = (V, E)$ לא ממושקל, ויהי $s \in V$, ותהי $(u, v) \in E$, אזי $\delta(s, v) \leq \delta(s, u) + 1$

משפט 40. לאחר הרצת $BFS(G, s)$ לכל $v \in V$ מתקיים $d[v] \geq \delta(s, v)$. (ההוכחה באינדוקציה על מס' פעולות enqueue).

משפט 41. בכל זמן באלגוריתם אם $Q = (v_1, \dots, v_r)$ מתקיימות שתי תכונות:

- א. $d[v_1] \leq d[v_2] \leq \dots \leq d[v_r]$
- ב. $d[v_r] \leq 1 + d[v_1]$ (כלומר לא יתכן שבתור בו זמנית ישנם יותר משתי שכבות במקביל).

משפט 42. אם $u \in V$ יוצא מ- Q לפני $v \in V$ תוך כדי ריצת $BFS(G, s)$ אזי $d[u] \leq d[v]$. (כלומר, ערכי ה- d של הקודקודים יכולים רק לעלות לאורך ההרצה).

הגדרה 43. סריקת DFS על הגרף היא סריקה לעומק של הגרף. הסריקה עוברת על כל הקודקודים של הגרף באופן הבא: כל עוד יש קודקוד שלא ביקרנו בו - נבקר בו. כאשר מבקרים בקודקוד, בודקים אם יש מישהו משכניו שטרם ביקרו בו - ואם כן מבקרים בו בקריאה רקורסיבית. האלגוריתם DFS סורק את כל הקודקודים. **בדומה ל- BFS , האלגוריתם משייך לכל קודקוד צבע שמסמל את מצב הקודקוד:**

- b שחור, ביקרנו וסיימנו לטפל בקודקוד.
 - w לבן, טרם ביקרנו בקודקוד
 - g אפור, ביקרנו אך טרם סיימנו לטפל בקודקוד.
- בנוסף לכל קודקוד $u \in V$ שומר האלגוריתם שלושה ערכים:
- א. $d(u)$ - זמן הגעה (צביעה באפור)
 - ב. $f(u)$ - זמן עזיבה (צביעה בשחור)

ג. $\pi(u)$ קודקוד קודם. השדה π מגדיר לכל קודקוד קודקוד קודם, אשר אם נסתכל עליו כ"אבא" של הקודקוד הראשון נקבל אוסף של עצים, המכונה גם יער העומק. נשים לב שיער העומק תלוי בריצה ספציפית של האלגוריתם ה-DFS ולגרף נתון יכולים להיות מס' יערי עומק שונים.

הערה 44. פסאודו של DFS:

$(u\text{DFS-Visit})$	$\text{DFS}(G)$
1. $color[u] \leftarrow g$	1. $u \in V$ for
2. $d[u] \leftarrow t \leftarrow t + 1$	(א) $color[u] \leftarrow w, \pi[u] \leftarrow \text{NULL}$
3. $v \in adj(u)$ for	2. $t \leftarrow 0$
(א) $color[v] = w$ if	3. $u \in V$ for
(ב) $(v)\text{DFS-Visit}$ then	(א) $color[u] = w$ if
i. $\pi[v] \leftarrow u$	(ב) $\text{DFS-Visit}(u)$ then
4. $color[u] \leftarrow b$	
5. $f[u] \leftarrow t \leftarrow t + 1$	

הסבר על האלגוריתם: האלגוריתם שומר כמשתנה גלובלי את t שמאותחל בתחילה לאפס וגדל בהתאם לאלגוריתם. t מציין את זמן הכניסה הנוכחי. מוחזר לנו מהאלגוריתם מערך π שהוא יער העומק. במערך מציין את שורש העץ. יער העומק מכיל עצים שונים.

סיבוכיות זמן ריצה: $O(|V| + |E|)$

הערה חשובה: לא מתואר כיצד אנחנו רצים על האלגוריתם, ולכן כל סדר של הקודקודים (כל עוד אנחנו יודעים אותם מראש) הינו חוקי. יתכנו $|V|!$ פרמוטציות אפשריות לריצת DFS.

משפט 45. משפט הסוגריים: בכל DFS על גרף $G = (V, E)$ מכוון או שלא מכוון, עבור כל זוג קודקודים $v, u \in V$ מתקיים אחד מהשלושה הבאים (עבור האינטרוול של זמן ההתחלה עד זמן הסיום):

1. $[d[u], f[u]] \cap [d[v], f[v]] = \emptyset$
2. $[d[u], f[u]] \subseteq [d[v], f[v]]$
3. $[d[u], f[u]] \supseteq [d[v], f[v]]$ (וכן קודקוד v הוא צאצא של u ביער העומק אמ"פ מקרה זה מתקיים)

משפט 46. משפט המסלול הלבן: $v \in V$ הוא צאצא של $u \in V$ ביער העומק אמ"פ בזמן $d[u] - 1$ קיים ב- G מסלול מ- u ל- v שכל קודקודיו צבועים לבן.

הגדרה 47. סיווג קשתות: ניתן לסווג את הקשתות בגרף בהתאם לריצת ה-DFS כך שכל קשת מסווגת לפי אחד מהסוגים הבאים, כך שאין קשת שנמצאת בשתי קבוצות:

1. קשת עץ: קשת מהצורה $(\pi(u), u)$ עבור $u \in V$ כלשהו. יש בדיוק $k - |v|$ קשתות כאלו כאשר k הוא מס' העצים.
2. קשת אחורה: קשת מ- v לאב קדמון של v שנקרא לו u . צריך להתקיים $d[u] < d[v]$
3. קשת קדימה: קשת מ- v לצאצא לא ישיר של v , נקרא לו u . צריך להתקיים $d[v] < d[u]$
4. קשת חוצה: קשת מ- u לקודקוד שאינו צאצא ואינו אב קדמון של u . זה כל שאר הקודקודים, ה-DFS של צמתי הקשת זרים.

הגדרה 48. גרף DAG או גמ"ל הוא גרף מכוון חסר מעגלים.

משפט 49. גרף G הוא DAG $\iff$ בהרצת DFS אין קשתות אחוריות.

הגדרה 50. מיון טופולוגי של קודקודי הגרף $G = (V, E)$ שהוא גמ"ל הוא סידור $(v_1, \dots, v_n)$ של הקודקודים ב- V כך שלכל קשת $(v_i, v_j) \in E$ מתקיים $i < j$.
Topological – Sort(G)

1. הרץ DFS (המחשב את $f[u]$ לכל $u \in V$)
 2. החזר את קודקודי V בסדר יורד של $f[u]$
- זמן ריצה: ישירות מ- DFS , זמן של $O(|E| + |V|)$

משפט 51. לגרף קיים מיון טופולוגי $\iff$ הגרף הוא DAG .

הגדרה 52. רכיב קשיר היטב\חזק בגרף $G = (V, E)$ מכוון הוא קבוצה מקסימומית $C \subseteq V$ כך שלכל $u, v \in C$ מתקיים $u \rightsquigarrow v$ וגם $v \rightsquigarrow u$.

הגדרה 53. גרף $G = (V, E)$ יקרא קשיר היטב אם ורק אם G מכיל בדיוק רכיב קשיר היטב אחד.

הגדרה 54. יהי $G = (V, E)$ גרף מכוון. נגדיר את גרף הרכיבים הקשירים היטב כגרף $G^{SCC} = (V^{SCC}, E^{SCC})$ כאשר V^{SCC} היא קבוצת הרכיבים הקשירים היטב של G . בנוסף עבור שני רכיבים קשירים היטב שונים $C_1, C_2 \in V^{SCC}$ מתקיים $(C_1, C_2) \in E^{SCC}$ אם ורק אם קיימים $v_1 \in C_1$ ו- $v_2 \in C_2$ כך ש- $(v_1, v_2) \in E$.

משפט 55. יהי $G = (V, E)$ גרף מכוון. אזי, G^{SCC} יהיו גרף מכוון ללא מעגלים.

משפט 56. יהי $G = (V, E)$ ויהי $C \subseteq V$ רכיב קשיר חזק ב- G . לאחר ריצת DFS על G כל קודקודי C נמצאים באותו עץ ביער העומק. הערה: נשים לב שכל רכיב היטב יהיה ביחד בעץ ה- DFS , אם כן יתכן שכמה רכיבים קשירים היטב יהיו יחד באותו עץ ביער העומק, ועל כך נצטרך להתגבר באלגוריתם G^{SCC} .

הגדרה 57. יהי $G = (V, E)$ גרף מכוון. הגרף המשוחלף בגרף G ונסמנו $G^T = (V, E^T)$ הוא הגרף המתקבל מהיפוך כל קשת בגרף G . כלומר, $(u, v) \in E \iff (v, u) \in E^T$. באופן שקול, הגרף הוא הגרף שמטריצת השכנויות שלו היא שיחלוף של מטריצת השכנויות של הגרף המקורי.

הערה 58. פסאודו למציאת רכיבי G^{SCC} :

$Strongly\ Connected\ Component(G)$:

א. הרץ $DFS(G)$ (המחשב תוך כדי ריצתו את $f[u]$ לכל $u \in V$)

ב. חשב את G^T

ג. הרץ $DFS(G^T)$ כאשר בלולאה הראשית בפונקציה DFS עבור על הקודקודים ע"פ סדר יורד של $f[u]$.

ד. דווח על כל עץ ביער העומק כרכיב קשיר היטב.

זמן ריצה: $O(|E| + |V|)$.

הגדרה 59. בהינתן גרף $G = (V, E)$ דו"צ קשיר, נרצה למצוא חלוקה חוקית שלו ל- R, L :
האלגוריתם:

א. הרץ BFS מקודקוד s שרירותי.

ב. כל מי שבמרחק זוגי יהיה ב- L וכל מי שבמרחק אי זוגי יהיה ב- R .

זמן הריצה: $O(|V| + |E|)$.

הגדרה 60. מציאת עץ מקב"ם ב- DAG :

קלט: גרף $G = (V, E)$ גמ"ל עם פונקציית משקלים $w : E \rightarrow \mathbb{R}$ וקודקוד מטרה s .

פלט: $SSSP$ (מערך מרחקים)

האלגוריתם יפעל בתכנות דינמי:

$$f(u) = \begin{cases} 0 & u = s \\ \min_{\{v|(v,u) \in E\}} \{f(v) + w(v, u)\} & o.w \end{cases}$$

נכונות הנוסחה: יהי $P = (s, v_1, \dots, v_k, u)$ מסלול קצר ביותר מ- s ל- u . נשים לב כי הקודקוד v_k שהוא הקודם ל- u מקיים $(u_k, u) \in E$ ולכן נשקל כאחת האפשרויות בביטוי שהפונקציה מנסה להביא למינימום. נשים לב כי העלות של המסלול P היא $w(P) = \sum_{e \in P} w(e) = w(P') + w(v_k, u)$ כאשר $P' = (s, \dots, v_k)$ הוא מק"ב בעצמו.

זמן הריצה: הרצת מיון טופולוגי ומעבר על הקודקודים לפי סדר המיון הטופולוגי. מה שיבטיח זמן לכל קודקוד של $O(deg(v))$. סה"כ $O(|V| + |E|)$.

הגדרה 61. סוגי מעגלים:

א. מעגל שלילי הוא מעגל שסכום הקשתות שלו הוא מס' שלילי. הליכה על מעגל שכזה תהיה תמיד טובה לנו ואם נלך על המעגל אנסוף פעמים נוכל לקבוע $\delta(u, v) = -\infty$ (אם יש מעגל שלילי במסלול בין u ל- v).

ב. מעגל אפס הוא מעגל שסכום המשקלים שלו הוא אפס ואין לנו סיבה לעבור בו.

ג. מעגל חיובי הוא מעגל שסכום המשקלים שלו חיובי, ושוב אין לנו סיבה לעבור בו.

מסקנה 62. אם אין בגרף מעגלים שליליים, ניתן להניח כי מסלול קצר ביותר בין זוג קודקודים הוא מסלול פשוט.

הגדרה 63. (אלגוריתם מבוסס הקלות) הוא אלגוריתם שמתחזק לכל $u \in V$ ערך $d[u]$ שהוא חסם עליון על $\delta(s, u)$. כלומר, $d[u] \geq \delta(s, u)$. האלג' על פני ריצתו יוריד ערכי d עד ש(בתקווה) כל ערכי d מקיימים: $\forall u \in V : d[u] = \delta(s, u)$. כמו כן, נשתמש במשתנה $\pi[u]$ להגדיר את עץ המסלולים הקצרים ביותר.

כך יראה האתחול בכל אלגוריתם מבוסס הקלות:

```
INITIALIZE-SINGLE-SOURCE( $G = (V, E), s$ )
1  for each vertex  $v \in V$ 
2       $d[v] \leftarrow \infty$ 
3       $\pi[v] \leftarrow NULL$ 
4   $d[s] \leftarrow 0$ 
```

כך תראה פעולת ההקלה בכל אלגוריתם מבוסס הקלות:

```
RELAX( $(u, v), w$ )
1  if  $d[v] > d[u] + w(u, v)$ 
2       $d[v] \leftarrow d[u] + w(u, v)$ 
3       $\pi[v] \leftarrow u$ 
```

משפט 64. למת אי שוויון המשולש (במקרה הממושקל): עבור $G = (V, E)$ עם פונקציית משקל $w : E \rightarrow \mathbb{R}$ וכן $s \in V$ ו- $(u, v) \in E$ מתקיים:

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

משפט 65. (למת חוסר המסלול): לאחר ביצוע $ISS(G, s)$ (אתחול) באלגוריתם מבוסס הקלות, אם אין מסלול מ- s ל- $v \in V$ אזי תמיד מתקיים $\delta(s, v) = d[v] = \infty$.

משפט 66. (למת הקלת מסלול): נניח כי המסלול $P = (v_0, \dots, v_k)$ הוא מסלול קצר ביותר עם $s = v_0$. נניח שבזמן ריצת אלגוריתם מבוסס הקלות לאחר ביצוע $ISS(G, s)$, סדרת ההקלות שהאלגוריתם מבצע מכילה את סדרת הקשתות של p כתת סדרה לפי סדרן ב P . (כלומר, לאחר ביצוע פעולות רילקס על כל (v_i, v_{i+1}) לכל $0 \leq i \leq k-1$ לפי הסדר אך לא בהכרח ברצף) אזי, בסיום ריצת האלגוריתם מתקיים $d[v_k] = \delta(s, v_k)$. (כלומר, הבעיה נפתרה עבור קודקוד v_k).

מסקנה 67. נשים לב כי אם פתרנו את הבעיה עבור v_k , אזי פתרנו את הבעיה לכל $0 \leq i < k$, כלומר $\forall 0 \leq i < k : d[v_i] = \delta(s, v_i)$. כלומר, אם נריץ פעם אחת נפתור את הבעיה עבור v_1 . אם נריץ פעמיים, נפתור את הבעיה עבור v_2 . וכן אם נריץ עבור $|V| - 1$ נקבל את הפתרון להבעיה.

משפט 68. (למת החסם העליון של $d[v]$): באלגוריתם מבוסס הקלות, לאחר ביצוע האתחול $ISS(G, s)$ לכל קודקוד $v \in V$ תמיד מתקיים $d[v] \geq \delta(s, v)$. בנוסף, מהרגע שבו האלגוריתם מציב $d[v] = \delta(s, v)$, הערך של $d[v]$ לא ישתנה יותר עד לסיום ריצת האלגוריתם.

משפט 69. (תכונת ההתכנסות): נניח שהמסלול $s \rightsquigarrow u \rightarrow v$ הוא מסלול קצר ביותר מ s ל v . אזי, לאחר ביצוע $ISS(G, s)$ באלגוריתם מבוסס הקלות, מתקיים שאם $d[u] = \delta(s, u)$ מתישהו לפני ביצוע הקלה על הקשת (u, v) , אזי לאחר ביצוע הקלה על הקשת מתקיים $d[v] = \delta(s, v)$. (כלומר, אם המסלול מ $s \rightsquigarrow u$ כבר ידוע לי, לאחר ביצוע הקלה אחת אנחנו נדע את $s \rightsquigarrow v$ כי הרי שה"כ מוסיפים הליכה על קשת אחת).

הגדרה 70. האלגוריתם של בלמן פורד: האלגוריתם של בלמן פורד הוא אלגוריתם מבוסס הקלות. לפי למות קודמות, אם נדע את המסלול הקצר ביותר מ s אל v סיימנו. נשים לב אי אם נבצע הקלה לכל קשתות הגרף, בהכרח ביצענו הקלה על הקשת הראשונה במק"ב אך לא ידוע לנו באיזה שלב של ההכלות ביצענו עליה הקלה. אנחנו יודעים כי $d[v_1] = \delta(s, v_1)$. לאחר k איטרציות לכל קודקוד u שיש אליו מק"ב עם לכל היותר k קשתות, נדע את $\delta(s, u)$. וכיוון שמק"ב יכול להיות באורך לכל היותר $|V| - 1$ אם אין מעגלים שליליים, אנו זקוקים למס' זה של איטרציות. זמן הריצה $O(|V||E|)$. להלן פסאודו:

```

BELLMAN-FORD( $G = (V, E), w, s$ )
1  INITIALIZE-SINGLE-SOURCE( $G = (V, E), s$ )
2  for  $i = 1$  to  $|V| - 1$ 
3      for  $(u, v) \in E$ 
4          RELAX( $(u, v), w$ )

```

מסקנה 71. נשים לב: ב DAG ישנו מיון טופולוגי, ולכן ניתן להחליט שנעבור בסדר מסוים על הגרף. וכך, לא נצטרך בכל שלב באלגוריתם בשורה 3 לעבור על כל הקשתות, נוכל לעבור בכל קודקוד רק על הקודקודים שיוצאים ימינה. נשים לב כי לקודקוד הראשון יש אפשרויות ללכת רק ימינה וכן הלאה על ההבאים. נקבל כי סיבוכיות זמן הריצה היא $O(|E| + |V|)$.

מסקנה 72. כיוון שאמרנו שמסלול ארוך ביותר יהיה בגודל $|V| - 1$ כי הוא פשוט (לא נרצה לעבור על מעגלים). ניתן לפצוא דרך לזהות מעגל שלילי - ניתן להריץ בלמן פורד ולהוסיף איטרציה מס' $|V|$. נעבור בה על הקשתות. אם עדיין אפשר לעשות הקלה לאחת הקשתות ישר נחזיר נחזיר נחזיר מעגל שלילי. מדוע? כי אם אפשר לשפר, מתכונה זו בדיוק, זה מסלול בגודל $|V|$, מעגל שלילי.

הגדרה 73. האלגוריתם של דייקסטרה:

קלט: גרף $G = (V, E)$ ממושקל עם פונקציית משקל $w : E \rightarrow \mathbb{R}^+$ (משקלים חיוביים בלבד), וקודקוד מקור $s \in V$
פלט: $\forall u \in V : \delta(s, u)$ וכן להחזיר עץ מסלולים קצר ביותר מ- s .

הסבר רעיוני על האלגוריתם של דייקסטרה: באלגוריתם של פריס, למציאת MST , בכל איטרציה האלגוריתם בוחר את הקשת בעלת המשקל הנמוך ביותר שחוצה את החתך ומוסיף אותה לעץ. האלגוריתם של דייקסטרה פועל באופן דומה. בכל איטרציה, האלגוריתם של דייקסטרה בוחר את הקשת הטובה ביותר להוסיף לעץ המסלולים. השוני בין האלגוריתמים - מה זה אומר "טובה ביותר".

הגדרה 74. (מסלול מיוחד) תהי S קבוצת הקודקודים שנמצאת כעת בעץ המסלולים הקצרים ביותר. מסלול $P = (v_0, \dots, v_{k-2}, v_{k-1})$ יקרא מסלול מיוחד אם $v_i \in S$ $\forall 0 \leq i \leq k-2$ וכן $v_{k-1} \notin S$ (כלומר, כל הקודקודים על המסלול נמצאים ב- S פרט לקודקוד האחרון). יתכנו כמה מסלולים מיוחדים.

הערה 75. האלגוריתם של דייקסטרה בוחר בכל שלב קודקוד $u \notin S$ כך שהמסלול מ- s אליו הוא הקצר ביותר ביחס לכל המסלולים המיוחדים מ- s ל- u . האלגוריתם מוסיף את הקשת האחרונה על המסלול המיוחד (זאת שנראית הטובה ביותר כרגע) אל העץ ומוסיף את u ל- S .

הגדרה 76. האלגוריתם של דייקסטרה: אלגוריתם מבוסס הקלות, זמן הריצה $O(|V|\log|V| + |E|)$ פסאודו:

 DIJKSTRA($G = (V, E), w, s$)

```

1  INITIALIZE-SINGLE-SOURCE( $G = (V, E), s$ )
2   $S \leftarrow \emptyset$ 
3   $Q.Init(V)$ 
4  while  $Q \neq \emptyset$ 
5       $u \leftarrow Q.extract\_min()$ 
6      Add  $u$  to  $S$ 
7      for each  $v \in ADJ[u]$ 
8          RELAX( $(u, v), w$ )

```

האלגוריתם פותח באתחול $ISS(G, s)$, ומשתמש בתור קדימויות כך שמפתחות התור הם ערכי d . כל עוד התור לא ריק מוציאים את האיבר עם המפתח המינימלי (שקול למרחק הכי קטן) ומכניסים אותו אל S . בשלב הבא - מבצעים הקלה לכל שכניו שלא נמצאים ב- S (מדובר בנקודה קריטית). הסיבה שזה חוקי (וכך גם הוכיח דייקסטרה) הינה שהמשקלים אי שליליים ולכן לא תהיה דרך עוד לשפר את המסלול ה"ל". כל קודקוד שנמצא ב- S כבר קבע את $d[u] = \delta(s, u)$.

משפט 77. (הוכחת נכונות של דייקסטרה) בכל פעם שהאלגוריתם של דייקסטרה פגיע לשורה 4 בפסודו קוד, אז לכל $u \in S$ מתקיים $d[u] = \delta(s, u)$.

shorts path – APSP 4

הגדרה 78. כפל מטריצות מהיר:

קלט: שתי מטריצות $A, B \in \mathbb{F}^{n \times n}$. נסמן: $A = \{a_{ij}\}, B = \{b_{ij}\}$
פלט: לחשב את $C = A \times B$, כלומר את $C = \{c_{ij}\}$

משפט 79. קיימים מס' אלגוריתמים לכפל מטריצות מהיר:

א. אלגוריתם נאיבי באמצעות הנוסחה $C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$ בעלות $O(n^3)$
 ב. האלגוריתם של שטרסן: מגדיר כפלים חדשים ומגיע לזמן ריצה $T(n) = 7T(\frac{n}{2}) + O(n^2) = O(n^{\log_2(7)})$
 ג. האלגוריתם של *Coppersmith–Winograd*: זמן ריצה של $O(n^{2.37287})$

הגדרה 80. נסמן ב- ω את האקספוננט של האלגוריתם הכי מהיר לפתרון בעיית כפל מטריצות. מתקיים:
 $2 \leq \omega \leq 2.37287$. ולכן, האלגוריתם המהיר ביותר לכפל מטריצות ריבועיות מגודל $n \times n$ בעלות $O(n^\omega)$.

האלגוריתם של פלוייד וורשל

האלגוריתם של פלוייד וורשל פותר *APSP* במקרה הממושקל ללא מעגלים שליליים. זמן הריצה: $O(|V|^3)$.

הגדרה 81. יהי $P = (v_1, \dots, v_\ell)$ מסלול. אזי, הקודקודים $v_2, \dots, v_{\ell-1}$ הם קודקודי ביניים של P .

משפט 82. יהי P מסלול קצר ביותר מ- i ל- j מבין המסלולים המשתמשים בקודקודים $\{1, \dots, k\}$. נראה כי כיוון G לא מכיל מעגל שלילי או יכולים להניח בה"כ כי P מסלול פשוט. ובהכרח מתקיים אחד מהשניים:

א. אם k אינו קודקוד ביניים ב- P אזי המסלול P משתמש רק בקודקודים $\{1, \dots, k-1\}$ ובהכרח אורכו קצר ביותר מבין כלל המסלולים המשתמשים ב- $\{1, \dots, k-1\}$ (ואפשר למצוא את אורכו ברוסיה).
 ב. אם k הינו קודקוד ביניים ב- P , אזי ניתן לחלק את P לשני מסלולים. מסלול p_1 מ- i אל k ומסלול p_2 מ- k אל j . כל אחד מהמסלולים p_1, p_2 לא מכיל את k כקודקוד ביניים (הוא הקצה) ולכן בהכרח משתמש רק בקודקודים מתוך $\{1, \dots, k-1\}$ כקודקודי ביניים. כיוון ששת מסלול של מסלול קצר ביותר הוא מסלול קצר ביותר, p_1 הוא מסלול קצר ביותר מבין המסלולים המשתמשים ב- $\{1, \dots, k-1\}$ מ- i אל k ומסלול קצר ביותר מבין המסלולים המשתמשים ב- $\{1, \dots, k-1\}$ מ- k אל j .

משפט 83. נגדיר $d_{ij}^{(k)}$ כאורך המסלול הקצר ביותר בין i ל- j שמשתמש בקודקודי ביניים מהקבוצה $\{1, \dots, k\}$. נוסחת הנסיגה שפלוייד וורשל משתמש בה באלגוריתם היא:

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & k = 0 \\ \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\} & k > 0 \end{cases}$$

הערה 84. פסאודו של פלוייד וורשל:

Ployd-Warshall(W):

1. $n \rightarrow \text{rows}[W]$
2. $D^{(0)} = W$
3. for $k = 1$ to n
 - a. for $i = 1$ to n
 - b. for $j = 1$ to n
- $d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$
4. return $D^{(n)}$

סגור טרנזיטיבי של גרף מכוון

הגדרה 85. עבור גרף מכוון $G = (V, E)$ הסגור הטרנזיטיבי $G^* = (V, E^*)$ הוא גרף על אותם קודקודים כך שאם G יש מסלול מקודקוד i לקודקוד j אזי ב- G^* ישנה קשת בניהם. כלומר:

$$E^* = \{(u, v) | u \rightsquigarrow_G v\}$$

משפט 86. קיים אלגוריתם לחישוב סגור טרנזיטיבי של גרף מכוון בזמן $O(|V|^\omega)$.
הוכחה: הרעיון יהיה להתבונן במטריצה $(A \vee I)^{|V|}$ - זו מטריצה שמייצגת את כל המסלולים מאורך עד $|V|$ בגרף. נבחין כי $A^{|V|}$ היא המטריצה של המסלולים מאורך בדיוק $|V|$. כיוון שהוספנו I על האלכסון, אנחנו מאפשרים לקודקוד להשאר במקום. חישוב המטריצה עלותה $O(|V|^\omega \log |V|)$ כמובן. נרצה להפטר מ- $\log(|V|)$ ולשם כך נניח את ההנחות הבאות שלגיטימיות להנחה בכל גרף:
 1. G הוא DAG (בזמן $O(|E| + |V|)$ ניתן לחשב את G^{SCC} והוא בהכרח DAG, לכן אם G לא DAG נוכל לפתור עבור G^{SCC} , שכן בתוך כל רכיב קשיר אנו יודעים כיצד נראה הסגור קלין (בהכל יש קשתות אחדות במטריצה כי יש מסלול).
 2. G ממויין טופולוגית (אם לא, ניתן למיין טופולוגית ב- $O(|E| + |V|)$).
 3. V הוא חזקה שלפיה של 2. (ברור כי ניתן להניח כי זאת שכן נוכל להוסיף קודקודי דמה - כמו אפסים)
 כעת, כיוון ש- G ממויין טופולוגית, מדובר במטריצה משולשית עליונה בהכרח שמייצגת את $A \vee I$.
 נסמן:

$$(A \vee I) = \begin{pmatrix} & X & Y \\ 0 & 0 & Z \\ 0 & 0 & \end{pmatrix}$$

ונטען כי

$$(A \vee V)^* = \begin{pmatrix} X^* & X^* \times Y \times Z^* \\ 0 & 0 & Z^* \\ 0 & 0 & \end{pmatrix}$$

כלומר, על מנת לחשב את $(A \vee I)^*$ כל שעלינו לעשות הוא לבצע הפרד ומשול, ולחשב את החלקים השונים של המטריצה בזמן ריצה $O(n^\omega) = O(n^\omega) + 2T(\frac{n}{2})$ כנדרש.

האלגוריתם של Jhonson

האלגוריתם של ג'ונסון מניח שיתכנו קשתות שליליות אך לא מעגל שלילי ומטפל ב-APSP. ג'ונסון רוצה להריץ דייקסטרה מכל קודקוד אך עליו לדאוג שהגרף לא יכיל קשתות שליליות. לשם כך ג'ונסון מגדיר פונקציית משקל חדשה $\hat{w}$ שתקיים:
 א. $\hat{w} : E \rightarrow \mathbb{R}^+$
 ב. מסלול p מקודקוד u ל- v יהיה קצר ביותר לפי $\hat{w} \iff$ מסלול p מקודקוד u ל- v יהיה קצר ביותר לפי w .

הגדרה 87. תהי $h : V \rightarrow \mathbb{R}$ פונקציית משקל לקודקודים. נגדיר $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$

משפט 88. יהי $G = (V, E)$ גרף מכוון עם פונקציית משקל $w : E \rightarrow \mathbb{R}$ ותהי $h : V \rightarrow \mathbb{R}$ ולכל $(u, v) \in E$ נגדיר $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$. יהי $P = (v_0, \dots, v_k)$ מסלול מ- v_0 ל- v_k . אזי P מסלול מינימלי ע"פ w אם ורק אם P מינימלי עבור $\hat{w}$. בנוסף, G מכיל מעגל שלילי ע"פ w אם ורק אם G מכיל מעגל שלילי ע"פ $\hat{w}$.

מסקנה 89. אם נחשב את מטריצת המרחקים לפי $\hat{w}$ נוכל במעבר אחד על המטריצה לחשב את מטריצת המרחקים לפי w שכן מדובר במעבר לינארי שעלותו $O(1)$ פר תא.

ג'ונסון הריץ את האלגוריתם של בלמן פורד, מגרף חדש שהתקבל ע"י הוספת קודקוד s וחיבורו במשקל אפס לכל הקשתות בגרף. פורמלית:
נגדיר $G' = (V \cup \{s\}, E')$ באשר $E' = \{E \cup \{(s, u) | u \in V\}\}$ וכן $w' : E \rightarrow \mathbb{R}$ המוגדרת כך:

$$w'(u, v) = \begin{cases} w(u, v) & (u, v) \in E \\ 0 & u = s \end{cases}$$

לאחר מכן, ג'ונסון הגדיר $h(v) = \delta_{G'}(s, v)$ לכל $v \in V$.

משפט 90. פונקציית המשקל $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ המוגדרת ע"י $h(v) = \delta_{G'}(s, v)$ לכל $v \in V$ היא אי שלילית. כלומר, $\hat{w}(u, v) \geq 0$.

הערה 91. פסאודו של ג'ונסון:

אלגוריתם 1 ג'ונסון ($G = (V, E)$)
(א) צור קודקוד חדש s והוסף קשתות במשקל 0 מ- s לכל קודקוד $v \in V$.
(ב) הרץ את האלגוריתם של בלמן-פורד מקודקוד s .
(ג) אם האלגוריתם של בלמן-פורד מצא מעגל שלילי -
i. החזר "מעגל שלילי".
(ד) אחרת
i. לכל $u \in V$
א. שמור $h(v) = \delta(s, v)$
ii. הגדר לכל $(u, v) \in E$
א. $\hat{w}(u, v) = w(u, v) + \delta(s, u) - \delta(s, v)$
iii. תהי $D = (d_{uv})$ מטריצה בגודל $n \times n$
iv. לכל $u \in V$
א. הרץ את האלגוריתם של דייקסטרה עם הפונקציה $\hat{w}$ מ- u כקודקוד מקור
ושמור את $\hat{\delta}(u, v)$
ב. עבור כל $v \in V$
ג. $\hat{\delta}(u, v) + h(v) - h(u) \rightarrow d_{uv}$
v. החזר את D

זמן הריצה: האלגוריתם מריץ בלמן פורד, ולאחר מכן $|V|$ פעמים מריץ דייקסטרה. סה"כ זמן הריצה $O(|V||E| + |V|^2 \log |V|)$

האלגוריתם של Seidel

סיידל יפתור את בעיית APSP בזמן $O(|V|^\omega \log(|V|))$ על גרף לא מכוון ולא ממושקל. נניח מראש כי G קשיר, כי אם G לא קשיר (ניתן לזהוי בזמן לינארי) ניתן לפתור את הבעיה על כל רכיב קשירות בנפרד.

הגדרה 92. כפל מטריצות בוליאני (BMM) מוגדר כך: $C_{ij} = \bigvee_{k=1}^n a_{ik} \wedge b_{kj}$

משפט 93. תהי A מטריצת השכנויות של הגרף. נגדיר $B = A^2 \vee A$. אזי, $B_{ij} = 1 \iff$ קיים G מסלול באורך 1 או 2 בין הקודקודים i ו- j .

משפט 94. נגדיר $\delta'(u, v)$ להיות אורך המסלול הקצר ביותר מ- u ל- v ב- G' (הגרף המיוצג ע"י B לעיל ממשפט 93). אזי,

$$\delta'(u, v) = \left\lceil \frac{\delta(u, v)}{2} \right\rceil$$

משפט 95. יהיו u, v שני קודקודים ויהי P המסלול הקצר ביותר בניהם. יהי w שכן של v , כך שקיים מסלול קצר ביותר בין u ל- w . אזי,

1. אם $\delta(u, v)$ זוגי, אזי לכל שכן w של v מתקיים $\delta'(u, w) \geq \delta'(u, v)$
2. אם $\delta(u, v)$ אי זוגי, אזי לכל שכן w של v מתקיים $\delta'(u, w) \leq \delta'(u, v)$

משפט 96. $\delta(u, v)$ זוגי $\iff \sum_{w \in \Gamma(v)} \delta'(u, w) \geq \deg(v) \times \delta'(u, v)$

הערה 97. (פסאודו של סיידל) האלגוריתם של סיידל מבוסס על המשפטים לעיל. האלגוריתם מקבל מטריצת שכנויות A ומחזיר מטריצה Δ כך ש- $\Delta_{ij} = (\delta(i, j))$. האלגוריתם מתחיל בתנאי עצירה: אם G הוא קליקה נרצה להחזיר את A , במקרה זה בקליקה לכל $u \neq v$ מתקיים $\delta(u, v) = 1$. במקרה זה מטריצת השכנויות מוגדרת להיות אחדות בכל המטריצה פרט על האלכסון. זו בדיוק מטריצת ה- $APSP$. אחרת, נרצה לקרוא לאלגוריתם עם B שהוגדרה לעיל. בסיום כל קריאה האלגוריתם מחשב את $M = \Delta' \times A$ שהרי היא מכפלה של מטריצת השכנויות המקורית A עם המטריצה שהתקבלה. נבחין כי במיקום u, v ישנה מכפלה של כל השורה u ב- Δ' עם העמודה v ב- A . למעשה, כיוון שהמכפלה הנ"ל אינה בוליאנית - אלא ממשית, המכפלה של השורה והעמודה יתנו בדיוק את הסכום $\sum_{w \in \Gamma(v)} \delta'(u, w)$. לכן אם $m_{u,v} = \sum_{w \in \Gamma(v)} \delta'(u, w) < \deg(v) \delta'(u, v)$ הרי שמדובר לפי הטענה לעיל במקרה האי זוגי ולכן $\delta(u, v)$ מוגדרת כפי שהוגדרה. אחרת: מקרה זוגי. להלן הפסאודו:

SEIDEL(A)

- 1 if A is all 1s except for the diagonal
- 2 return A
- 3 else $\Delta' \leftarrow \text{SEIDEL}(A^2 \vee A)$
- 4 $M \leftarrow \Delta' \cdot A$
- 5 for $u, v \in V$
- 6 if $m_{u,v} < \deg(v) \delta'(u, v)$
- 7 $\delta(u, v) = 2\delta'(u, v) - 1$
- 8 else $\delta(u, v) = 2\delta'(u, v)$
- 9 return Δ

משפט 98. זמן הריצה של סיידל הינו $O(|V|^\omega \log(|V|))$. הוכחה: ניתן לחשב את זמנות הקודקודים טריוויאלית בזמן $O(|V|^2)$. שאר השורות באלגוריתם עלותן $O(|V|^\omega)$. לפי טענה לעיל, אם אורך המסלול הקצר ביותר ב- G הוא ℓ אזי ב- G' אורך המסלול הקצר ביותר מ- u ל- v הוא $\lceil \frac{\ell}{2} \rceil$. מכאן, שלאחר $\log(|V|)$ קריאות רקורסיביות אורך המסלול הקצר ביותר בגרף הוא קליקה שכן: $n = \log(|V| - 1) = O(\log|V|) \iff |V| - 1 = 2^n \iff \frac{|V|-1}{2^n} = 1$ ולכן פסי האיטרציות $O(\log|V|)$ כפול הזמן פר איטרציה יניבו את זמן הריצה.

הבחנה. אם ידוע כי האורך הכי גדול של מסלול בגרף בין שני קודקודים הוא d , אזי סיבוכיות זמן הריצה של האלגוריתם הינה $O(|V|^\omega \log(d))$.

5 A^*

הגדרה 99. A^* הוא רעיון אלגוריתמי שמטרתו לענות על הבעיה הבאה: בהינתן גרף מכוון וממושקל $G = (V, E)$ עם פונקציית משקל $w : E \rightarrow \mathbb{R}^+$ ושני קודקודים $s, t \in V$, נרצה למצוא את המסלול הקצר ביותר בין s ל- t .

הערה 100. נבחין כי דייקסטרה יפתור את הבעיה בזמן $O(|V| \log |V| + |E|)$ אך אנחנו שואפים ליותר טוב מזה. הרי אנחנו רוצים לפתור single source.

הרעיון יהיה להגדיר פונקציית משקל חדשה: $w'(u, v) = w(u, v) + p(v) - p(u)$

משפט 101. לכל מסלול $P_{u \rightarrow v}$ (ובפרט לקצר ביותר) מתקיים $w'(P_{u \rightarrow v}) = w(P_{u \rightarrow v}) + p(v) - p(u)$

מסקנה 102. ניתן לחשב מק"ב באמצעות w' ובאמצעות תוספת לינארית לדעת את $w(P_{u \rightarrow v})$.

הגדרה 103. אם לאחר ההתמרה (הגדרת w') כל $w' \geq 0$ אזי p (פונקציית הפוטנציאל) הינה פיזיבילית. כלומר לכל $(u, v) \in E$ $w'(u, v) \geq 0 \iff p \iff$ פיזיבילית.

משפט 104. אם p פיזיבילית, וקיים $t \in V$ כך ש $p(t) \leq 0$, אזי $\forall v \in V : p(v) \leq \delta(v, t)$

הבחנה. נשים לב כי אם $p(v) = \delta(v, t)$ (פונקציית הפוטנציאל שלנו קירבה בדיוק!) לכל $v \in V$ אזי $p_{s \rightarrow t} = (s, v_1, \dots, v_k, t)$ ואם נסתכל על (v_i, v_{i+1}) קשת במסלול, אזי:

$$w'(v_i, v_{i+1}) = w(v_i, v_{i+1}) + \delta(v_{i+1}, t) - \delta(v_i, t) = \delta(v_i, t) - \delta(v_i, t) = 0$$

משום ש $w(v_i, v_{i+1}) + \delta(v_{i+1}, t) = \delta(v_i, t)$. ולכן - כל קשת במק"ב $s \rightarrow t$ תהיה במשקל 0. כלומר: דייקסטרה ירוץ ישירות על המסלול שלנו st . ראשית הוא יוציא את s עם $d[s] = 0$ ואז יבצע סדרת הקלות קודם כל על המק"ב st : בדיוק כפי שרצינו. מסקנה: נרצה למצוא פונקציית פוטנציאל שתקרב אותנו כמה שיותר אל $\delta(v, t)$ - שהרי איננו יודעים את $\delta(v, t)$.

משפט 105. אם p פיזיבילי ו $p(t) > 0$ אזי ניתן להפוך ל' p' פיזיבילי עם $p'(t) \leq 0$.

דוגמה 106. נתבונן בדוגמה הבאה. נרצה לבדוק את המרחק הקצר ביותר בין זוג צמתים במישור. נתבונן במפה דו מימדית $\{(x_i, y_i)\}_{i=1}^n$ נסמן את המרחק האוקלידי בין הצמתים: $\|g(v_1) - g(v_2)\| = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. נגדיר את פונקציית הפוטנציאל $p(v) = \|g(v) - g(t)\|$. נבחין כי $p(t) = \|g(t) - g(t)\| = 0$ ולכן $p(t) \geq 0$. נוכיח כי p פיזיבילית:

$$w'(u, v) = w(u, v) + p(v) - p(u) = w(u, v) + \|g(v) - g(t)\| - \|g(u) - g(t)\|$$

נבחין כי תמיד מתקיים $w(u, v) \geq \|g(u) - g(v)\|$ (הרי לא יתכן כי הכביש יהיה רחוק יותר מהמרחק האווירי) ומאבחנו זו נקבל:

$$w'(u, v) \geq \|g(u) - g(v)\| + \|g(v) - g(t)\| - \|g(u) - g(t)\| \geq_* 0$$

באשר המעבר האחרון נבע מאי שוויון המשולש על מרחקים אוקלידים.

משפט 107. אם לכל $u \in V$ $p(u) = \delta(u, t)$ אזי סריקת A^* תבקר רק בקודקודים על גבי מסלול קצר ביותר מ- s ל- t .

נשים לב - לא נובע מטענה זו שזמן הריצה יהיה O של אורך המסלול, שכן יתכן מצב אחד בדיוק בעייתי: אם כל הקשתות בגרף הם אפסים, במקרה הזה דייקסטרה לא בהכרח יתעדף את הקודקודים על המסלול שלנו. לכן במקרה הגרוע ביותר נגיע שוב לזמן הריצה של דייקסטרה. אם כן - הטענה כן אומרת שלא נבקר בקודקוד שלא במק"ב.

הגדרה 108. תהי פונקציית פוטנציאל $p : V \rightarrow \mathbb{R}$, p תקרא קבילה אם $\forall u \in V : p(u) \leq \delta(u, t)$

משפט 109. אם לכל $u \in V$ $p(u) \leq \delta(u, t)$ אזי סריקת A^* קבילה - כלומר A^* לא תבקר בקודקוד $u \in V$ כך ש- $\delta(s, u) + p(u) > \delta(s, t)$

משפט 110. אם לכל $u \in V$ $p(u) \leq 2\delta(u, t)$ אזי סריקת A^* לא תבקר בקודקוד $u \in V$ כך ש- $\delta(s, u) + p(u) > 2\delta(s, t)$

6 Network Flow

הגדרה 111. (רשת זרימה) רשת זרימה היא גרף מכוון $G = (V, E)$ עם קודקוד מקור $s \in V$ וקודקוד יעד $t \in V$. פונקציית קיבולת $C : V \times V \rightarrow \mathbb{R}^+ \cup \{0\}$ באשר $C(u, v) > 0 \iff (u, v) \in E$

הגדרה 112. (זרימה) זרימה ברשת זרימה $G = (V, E)$ עם פונקציית קיבולת C , היא פונקציה $f : V \times V \rightarrow \mathbb{R}$ שמקיימת את התנאים הבאים:

1. **אילווצי קיבולת:** $\forall u, v \in V : f(u, v) \leq C(u, v)$
2. **סימטריה:** $\forall u, v \in V : f(u, v) = -f(v, u)$
3. **שימור זרימה:** סכום הזרימה שיוצא מ- s ערכו אפס: $\sum_{u \in V} f(s, u) = 0$

הגדרה 113. הערך של זרימה f הוא $|f| = \sum_{v \in V} f(s, v)$

משפט 114. הפונקציה $f(u, v) = 0$ לכל $u, v \in V$ היא פונקציית זרימה.

הגדרה 115. הגדרת הבעיה:

קלט: רשת זרימה $G = (V, E)$ עם פונקציית קיבולת $C : V \times V \rightarrow \mathbb{R}^+ \cup \{0\}$
פלט: זרימה f ב- G בעלת ערך גדול ביותר מבין כל הזרימות האפשריות. *Max flow*.

הגדרה 116. יהיו $X, Y \subseteq V$

- א. נגדיר את הזרימה בין שתי קבוצות הקודקודים כך: $f(X, Y) = \sum_{x \in X} \sum_{y \in Y} f(x, y)$
- ב. נגדיר את הקיבולת בין שתי קבוצות קודקודים כך: $C(X, Y) = \sum_{x \in X} \sum_{y \in Y} C(x, y)$

משפט 117. תהי זרימה ברשת הזרימה $G = (V, E)$, c, s, t אזי:

1. $f(\{s\}, V) = |f|$
2. $\forall X \subseteq V : f(X, X) = 0$
3. $\forall X, Y \subseteq V : f(X, Y) = -f(Y, X)$
4. $\forall X, Y, Z \subseteq V \wedge X \cap Y = \emptyset$ מתקיים:
 - a. $f(X \cup Y, Z) = f(X, Z) + f(Y, Z)$
 - b. $f(Z, X \cup Y) = f(Z, X) + f(Z, Y)$

משפט 118. תהי זרימה ברשת זרימה $G = (V, E)$, אזי, $|f| = f(V, t)$

מסקנה 119. מתקיים $f(V, t) = f(s, V)$ (סך הזרם שיוצא מ- s שווה לסך הזרם שנכנס אל t).

הגדרה 120. חתך (s, t) הוא חתך $(S, T) = (S, V \setminus S)$ כאשר $s \in S$ וכן $t \in T$. נשים לב כי G הינו מכוון ולכן קשת שחוצה את החתך היא קשת שעוברת מ S אל T (הקשתות בכיוון השני לא נקראות כאלו ולא מעניינות אותנו). כמו כן בהכרח $\emptyset \neq S \subset V$ (כלומר מתקיים $T \cap S = \emptyset$ וכן $T \cup S = V$).

משפט 121. תהי $G = (V, E)$ רשת זרימה עם פונקציית קיבולת C . ותהי f זרימה ב G . יהי (S, T) חתך (s, t) של G . אז, $|f| = f(S, T)$.

שיטת פורד פרקלסון

פורד פרקלסון פותח בזרימה $f = 0$ ומחפש מסלולי שיפור. לפני שנמשיך נגדיר את המושג החשוב הבא:

הגדרה 122. קשת $(x, y) \in E$ תקרא רוויה תחת זרימה f אם $f(x, y) = C(x, y)$.

מסקנה. נשים לב כי אם קשת איננה רוויה, אזי ניתן להשתמש בקיבולת הנוותרת.

הגדרה 123. (הקיבולת השיורית) תהי $G = (V, E)$ רשת זרימה עם פונקציית קיבולת C . תהי f זרימה ב G . הקיבולת השיורית של C תחת f היא הפונקציה: $C_f(u, v) = C(u, v) - f(u, v)$.

הגדרה 124. (הרשת השיורית) הרשת השיורית של $G = (V, E)$ תחת f היא רשת זרימה $G_f = (V, E_f)$ שפונקציית הקיבולת שלה C_f . כמו כן, $E_f = \{(u, v) | C_f(u, v) > 0\}$.

נבחין כי הרשת השיורית מהווה עבורנו מסלול התחרטות - חזרה אחורה. כמו כן, נבחין כי הרשת השיורית מאפשרת קיום קשתות שלא היו קיימות ברשת $G = (V, E)$ המקורית. ישנה אפשרות כעת בזכות האבחנה הזו להתחרט: כלומר, לבצע בחירה שנראית טוב כרגע, ולתקן בהמשך במידת הצורך. מתי נדע לעצור? אם לא נוכל להתחרט יותר - כשכל הקשתות נכנסות אל s ולא יוצאות מ s : או שאין מסלול משפר בין s ל t .

הגדרה 125. בהינתן מסלול P ברשת השיורית G_f נגדיר (את הקיבולת השיורית המינימלית) כך: $C_f(P) = \min\{c_f(u, v) | (u, v) \in P\}$.

הגדרה 126. שיטת פורד פרקלסון. מצורף לעיל פסאודו קוד של השיטה:

FORD-FULKERSON($G = (V, E), s, t, c$)

- 1 initialize $f(u, v) = 0$ for all $u, v \in V$
- 2 $G_f \leftarrow G, c_f \leftarrow c$
- 3 **while** there exists a path P from s to t in G_f
- 4 $c_f(P) \leftarrow \min_{(u,v) \in P} \{c_f(u, v)\}$
- 5 **for** each edge $(u, v) \in P$
- 6 $f(u, v) \leftarrow f(u, v) + c_f(P)$
- 7 $c_f(u, v) \leftarrow c_f(u, v) - c_f(P)$
- 8 $f(v, u) \leftarrow f(v, u) - c_f(P)$
- 9 $c_f(v, u) \leftarrow c_f(v, u) + c_f(P)$
- 10 update E_f
- 11 **Return** f

מה פורד פרקלסון עושה? הוא מקבל פונקציית קיבולת, ומאתחל את פונקציית הזרימה ל-0 שהיא פונקציית זרימה חוקית ובונה את הרשת השזורית (בתחילה, רשת הזרימה עצמה). כל עוד קיים מסלול כלשהו (זה חשוב הכלשהו, בהמשך נראה שאם נחפש מסלול ספציפי מצבנו יהיה טוב הרבה יותר בזמן הריצה) בין s ל- t , אזי אנחנו נוסיף ונחסר בהתאמה מכל הקשתות בגרף את הקיבולת השזורית המינימלית - צוואר הבקבוק, ונעדכן את הרשת השזורית E_f . כשנסיים, כלומר, כשלא יהיה מסלול משפר: בידינו הזרימה המקסימלית.

הגדרה 127. מסלול P שכזה בשיטת פורד פרקלסון נקרא מסלול שיפור. ומינוח: אנו משתמשים ב- P על מנת לשפר את f .

הגדרה 128. (בעיית חתך מינימום)

קלט: גרף $G = (V, E)$ מכוון. ושני קודקודים $s, t \in V$ וכן פונקציית משקל $w : E \rightarrow \mathbb{R}^+$
 פלט: חתך (s, t) ששכום קשתות שעוברות מ- S ל- T הוא כמה שיותר קטן.

משפט 129. תהי $G = (V, E)$ רשת זרימה עם פונקציית קיבולת C ותהי f רשת זרימה ב- G . ויהי (S, T) חתך (s, t) ב- G . אזי, $|f| \leq C(S, T)$

משפט 130. **משפט max-flow-min-cut:** תהי $G = (V, E)$ רשת זרימה עם פונקציית קיבולת C ותהי f זרימה ב- G . אזי, כל התנאים הבאים שקולים:

- א. f זרימת מקסימום.
- ב. ב- G_f אין מסלול שיפור
- ג. קיים חתך (s, t) שנשמנו (S, T) ב- G כך ש- $|f| = C(S, T)$.

משפט 131. (מסקנה ממשפט 130) זרימת מקסימום = חתך (S, T) מינימום (!)

הערה 132. השיטה של פורד פרקלסון עלולה שלא להסתיים לעולם (ראו ערך דוגמה עם מס' הזהב). אך אם כל הקיבולות מס' שלמים / רציונליים, אזי האלגוריתם כן יסתיים. ולכן בכל איטרציה הזרימה

תשתפר בלפחות אחד. אם הזרימה המקסימלית היא $|f^*|$ אזי שכל איטרציה עלותה $O(|E|)$ למציאת מסלול שיפור. ולכן זמן ריצת האלגוריתם $O(|f^*| \times |E|)$.

משפט 133. תהי רשת זרימה $G = (V, E)$ עם פונקציית קיבולת C וזרימה f , אזי פתקיים f' זרימה ב G_f $f + f' \iff G_f$ זרימה ב G .
בפרט: f' זרימת מקסימום ב G_f $f + f' \iff G_f$ זרימת מקסימום ב G .

הגדרה 134. אלגוריתם למציאת חתך מינימום בזמן של אלגוריתם לזרימת מקסימום:

$find - min - cut(G = (V, E), c, s, t)$
א. הרץ אלגוריתם למציאת זרימת מקסימום ברשת הזרימה ותהי f זרימת המקסימום המתקבלת.
ב. חשב את הרשת השיוויונית G_f
ג. חשב את הקבוצה $S_f = \{u \in V | \exists s \rightsquigarrow u\}$ כלומר כל הקודקודים שקיים מסלול מ s אליהם ברשת השיוויונית.
ד. החזר את החתך $(S_f, V \setminus S_f)$

הגדרה 135. אלגוריתם לבדיקה האם חתך מינימום הוא ייחודי:

א. הרץ אלגוריתם למציאת זרימת מקסימום ברשת הזרימה ותהי f זרימת המקסימום המתקבלת.
ב. חשב את הקבוצה $S_f = \{u \in V | \exists s \rightsquigarrow u\}$
ג. חשב את הקבוצה $T_f = \{u \in V | \exists u \rightsquigarrow t\}$
ד. החזר שקיים חתך יחיד אמ"מ $S_f \cup T_f = V$
משפט 136. קיים חתך מינימום יחיד אם"מ $S_f \cup T_f = V$.

האלגוריתם של אדמונס קארפ

האלגוריתם של אדמונס קארפ הוא **צורת מימוש** לשיטה של פורד-פרקלסון. בכל שלב אנחנו נמצא מסלול שיפור ברשת השיוויונית בעל מספר מינימלי של קשתות, מציאת המסלול מתבצעת על ידי הרצת BFS מ s עד שמגיעים ל t . ברור כי כל שיפור מסלול עלותו $O(|E|) = O(|V| + |E|)$ זמן (כי מניחים קשירות), ולאחר שנוכיח כי האלגוריתם מוצא את זרימת המקסימום בתוך לכל היותר $O(|E| \times |V|)$ איטרציות נקבל כי סיבוכיות זמן הריצה של האלגוריתם הינה $O(|E|^2 \times |V|)$.

אלגוריתם 1 אדמונס-קארפ $(G = (V, E), s, t, c)$

1. לכל קשת $(u, v) \in E$

$$0 \rightarrow f[u, v] \rightarrow f[v, u] \quad (\text{א})$$

2. כל עוד קיים מסלול ברשת השיוויונית G_f מ- s ל- t .

(א) הרץ BFS מ- s עד מציאת t . ויהי p המסלול שנמצא בעץ המסלולים הקצרים ביותר מ- s ל- t .

$$\min \{c_f(u, v) \mid (u, v) \in p\} \rightarrow c_f(p) \quad (\text{ב})$$

(ג) לכל קשת $(u, v) \in p$

$$f[u, v] - c_f(p) \rightarrow f[u, v] \quad \text{i.}$$

$$-f[u, v] \rightarrow f[v, u] \quad \text{ii.}$$

האלגוריתם משתמש ברעיון של פורד ומממש אותו שונה.

הגדרה 137. נסמן $\delta_f(u, v)$ כאורך המסלול הקצר ביותר בין u ל v ב G_f .

משפט 138. תהי זרימה הפתקבלת מזרימה f ע"י שיפור על גבי מסלול באורך הקצר ביותר מ s ל t ב G_f . אזי לכל $u \in V$ פתקיים $\delta_f(s, u) \leq \delta_{f'}(s, u)$
(כלומר, במסלול השיפור קודקודים רק מתרחקים מקודקוד המקור - מדוע זה נכון אינטואיטיבית? כיוון שעל המסלול הקצר ביותר הזרפת ובגרף השיוויוני קשתות אלו לא מופיעות עוד - או שמופיעות במשקל נמוך יותר, שרפת לעצמך את המסלול הקצר ביותר ולכן בהכרח אתה תתרחק).

משפט 139. תהי f' זרימה המתקבלת מזרימה f על ידי שיפור על גבי מסלול באורך קצר ביותר מ- s ל- G_f . אזי, לכל $u \in V$ מתקיים $\delta_f(u, t) \leq \delta_{f'}(u, t)$

משפט 140. תהי f' זרימה המתקבלת מזרימה f ע"י שיפור על גבי מסלול באורך קצר ביותר מ- s ל- G_f . אם $\delta_f(s, t) = \delta_{f'}(s, t)$ אזי כל מסלול קצר ביותר מ- s ל- $G_{f'}$ הוא גם מסלול קצר ביותר מ- s ל- G_f . (כלומר, אם יש שוויון שכזה אזי לאחר השיפור לא נוצרו מסלולים קצרים ביותר חדשים)

משפט 141. תהי G רשת זרימה ו- f זרימה כלשהי. נתבונן באלגוריתם אשר משפר על גבי מסלולים מאורך קצר ביותר מ- s ל- G רץ כל עוד אורכם הוא $\delta_f(s, t)$. אזי, מרגע שקשת (u, v) נהיית רוויה ע"י האלגוריתם, קשת זאת לא תהיה בשימוש על ידי אף מסלול שיפור אחר במהלך הפאזה הזו. (אינטואיטיבית: בשביל שתהיה משומשת שוב בפאזה הזו, משהו היה צריך להפעיל זרימה בכיוון ההפוך - זה לא הגיוני שיקרה בפאזה הזו כי פעולה כזו מאריכה את אורך הפסלול - ובפרט לא תהיה בפאזה הזו).

הגדרה 142. תהי ריצת האלגוריתם אדמונס קארפ ונתבונן בכל מסלולי השיפור שאורכם ℓ . נקרא לאיטרציות ששיפרו אותם: הפאזה ה- ℓ של האלגוריתם. כלומר, הפאזה ה- ℓ היא סדרת האיטרציות בהן אורך המסלול הקצר ביותר הוא ℓ קשתות. יתכן כי פאזה תהיה ריקה.

מסקנה 143. כל מסלול שיפור בפאזה ה- ℓ ניתן לשייך לקשת אחת לפחות אותה הוא הפך לרוויה. לפי הלמה, כל קשת תשווין למסלול אחד בפאזה ה- ℓ לכל היותר ולכן בפאזה ה- ℓ יכולות להיות לכל היותר $|E|$ איטרציות. יתכנו לכל היותר $|V|$ פאזות. לכן מס' האיטרציות של האלגוריתם לכל היותר $O(|V||E|)$. ומכאן שזמן ריצתו $O(|E|^2|V|)$ (בכל איטרציה $O(|E|)$ של ריצת BFS).

האלגוריתם של דיניץ'

דיניץ' גם הוא משתמש ב- $O(|V|)$ פאזות. עלות כל פאזה תהיה $O(|V||E|)$ ולכן זמן הריצה של דיניץ' יהיה $O(|V|^2|E|)$.

הגדרה 144. יהי $G = (V, E)$ גרף מכוון עם קודקוד מקור s וקודקוד יעד t , נסמן ב- ℓ את $\delta(s, t)$ (מס' הקשתות במסלול הקצר ביותר) ונגדיר באמצעותו את גרף השכבות, בו יש $\ell + 1$ שכבות, ובשכבה ה- i יהיו כל הקודקודים $u \in V$ כך ש- $\delta_f(s, u) = i$ וגם u נמצא על מסלול קצר ביותר מ- s אל t . (כלומר, הוא תת מסלול של מסלול קצר ביותר). סה"כ, נקבל את השכבות $0, 1, \dots, \ell$, נשים לב כי בהכרח בשכבה 0 ישנו את s ורק את s . כמו כן, בהכרח בשכבה ℓ ישנו את קודקוד t ורק אותו שכן $\ell = \delta(s, t)$ לפי הגדרה. הקודקודים שנכנסים אל ℓ הם קודקודים מהשכבה $\ell - 1$ ונמצאים על המסלול הקצר ביותר בדרך אל t באורך $\ell - 1$ ואם נוסיף את הקשת שאיתה הם נכנסים אל ℓ אזי נקבל מסלול קצר ביותר באורך $\ell = \delta(s, t)$ בדיוק. הקשתות בין השכבות הן קשתות שנמצאות על איזשהו מסלול קצר ביותר מ- s אל t . נשים לב כי לא כל הקודקודים נמצאים בגרף השכבות - רק קודקודים שנמצאים על קשתות שנמצאות על אחד מן המסלולים הקצרים ביותר מ- s אל t . באופן דומה, לא כל הקשתות נמצאות בגרף השכבות אלא רק הקשתות שבאחד המסלולים הקצרים ביותר. הרעיון באלגוריתם של דיניץ', יהיה בתחילת כל פאזה לבנות גרף שכבות L מ- G_f . לפי למה קודמת, בתחילת כל פאזה, ב- G_f נמצאים כל המסלולים הקצרים ביותר שהאלגוריתם ימצא תוך כדי הפאזה. לכן, לפי ההגדרה של L : גרף השכבות L מכיל את כל המסלולים הקצרים ביותר מ- s אל t ב- G_f . בתחילה אנו בונים את גרף השכבות, לוקחים את G_f ובונינו את גרף השכבות. כמה עולה לבנות את גרף השכבות? זמן הבניה של גרף השכבות הוא $O(|E|) = O(|V| + |E|)$ - כי יש קשירות. כיצד בונים את גרף השכבות? מריצים BFS מ- s , ומריצים BFS מ- t על G^T . כעת, לפי טענה שקשת (u, v) נמצאת במסלול קצר ביותר אם $\delta(s, u) + 1 + \delta(v, t) = \delta(s, t)$, נוכל לבדוק לכל $(u, v) \in E$ האם מתקיים השוויון ואם כן היא על מסלול קצר ביותר מ- s אל t ונוסיף אותה לגרף השכבות. סה"כ בניית גרף השכבות תעלה $O(|E| + |V|)$, אך $|V| = O(|E|)$ לכן סה"כ $O(|E|)$.

כיצד מוצאים מסלול קצר ביותר בעזרת גרף השכבות?

מתחילים מ- s שבשכבה 0, ואנו יודעים כי כל קשת מ- s תוביל אותנו לקודקוד שנמצא בשכבה הראשונה. בדומה, בשכבה 1 לא משנה איזה קשת נבחר נעבור לקודקוד בשכבה השנייה. באופן כללי, אם אנו בשכבה ה- i , וישנו קודקוד v_i , אנו יודעים כי גם אם ישנם הרבה קשתות מהשכבה ה- i לשכבה

ה-1, לא משנה איזה קשת נבחר היא תמיד נמצאת על מסלול קצר ביותר כלשהו מ- s אל t . לפי הבניה של L , כל קשת $e \in L$ נמצאת על מסלול קצר ביותר מ- s אל t . לכן, בחירה של קשת שרירותית שיוצאת מקודקוד u בשכבה ה- i בהכרח תוביל לקודקוד שנמצא בשכבה ה- $i+1$.

לכן, האלגוריתם למציאת מסלול קצר ביותר כלשהו מ- s ל- t מאוד פשוט:

- נאתחל את $s \rightarrow u$. כל עוד $u \neq t$ בחר קשת שרירותית שיוצאת מ- u , (u, v) .
- נוסיף את (u, v) למסלול.
- ועדכן $u = v$ וחזור לשלב א'.
- לבסוף, החזר את המסלול.

כמה זמן לוקח למצוא מסלול קצר ביותר מ- s ל- t שכזה? נראה כי אנחנו בוחרים כל אחת מהקשתות, לכן הזמן שאני משקיע בכל שכבה הינה $O(1)$ זמן. אם כן, זמן הריצה הוא מס' השכבות, אם $\ell + 1$ הוא מס' השכבות זמן הריצה יהיה $O(\ell)$.

אז מה קורה בתחילת האלגוריתם? בשלב הראשון של הפאזה, בנינו את גרף השכבות L שיעלה $O(|E|)$ זמן.

כעת, נחפש מסלול קצר ביותר מ- s אל t ב- L . אנו יודעים כי מסלול קצר ביותר מ- s אל t ב- L הוא מסלול קצר ביותר גם ב- G_f . זה יעלה $O(\ell) = O(|V|)$ כי אורך המסלול הוא לכל היותר $|V| - 1$. הרעיון יהיה, להמשיך לחפש מסלולים קצרים ביותר ב- L . אך - ישנה בעיה. הבעיה היא שלאחר שמצאנו את המסלול הראשון, חלק מהקשתות נהיות רוויות וצריכות להמחק מגרף השכבות L . יתכן גם שצריכים למחוק קודקודים מסוימים מהגרף. למה זה חשוב לנו? אמרנו שאנחנו יכולים לבחור קשת שרירותית בעת שמצאנו מסלול קצר ביותר - למה אמרנו שניתן לבחור שרירותית? כי לא משנה איזה קשת נבחר, תמיד בצד השני יהיה קודקוד שמגיע אליו ומשם ממשיכים, אך אם מחקנו קשת באיטרציה הקודמת יתכן (מאוד) שהגישא של לקחת שרירותית לא תעזור לנו ואנחנו נתקע - כי הקשת השרירותית אליה הלכנו, היא קשת שממנה אין להתקדם (היה באיטרציה הקודמת דרך להתקדם, אך מחקנו את הקשת). לכן, עלינו לפתח מנגנון שיבטיח שגם לאחר מציאת מסלול, גרף השכבות יהיה גרף מעודכן שעודנו גרף שכבות (ואז כן נוכל לקחת קשת שרירותית).

עדכון גרף השכבות: נסתכל על קשת $u \rightarrow v$ שצריכה להמחק מהגרף. מה יכול לקרות?

מבחינת u , יתכן שישנה קשת נוספת שיוצאת מ- u , למשל (u, x) - אז המחיקה של (u, v) לא משפיעה על u כי באיטרציה הבאה ישנן דרכים אחרים להתקדם למשל דרך x . אך, מה אם הקשת היחידה שיוצאת מ- u היא (u, v) ? אם נמחק אותה כעת - זה אומר שאין u איך להתקדם אל t בהמשך כי אולי יש הרבה קשתות שנכנסות אליו אך אין קשתות שיוצאות ממנו. הקושי בזה הוא שבשלב לפני u , אם נבחר בקשת אל u שרירותית אנחנו נתקע כי u אין איך להתקדם. במצב כזה, אם $deg_{out}(u) = 0$, נרצה למחוק את u מ- L ולמחוק את כל הקשתות (w, u) שנכנסות אל u . ומה, אם היה קודקוד x שיש לו קשת (x, u) , וכעת מחקנו את u , והדרך היחידה לצאת מ- x הייתה דרך הקשת (x, u) , כעת מחקנו את (x, u) כי היא נכנסת אל u ומחקנו את u - אז כעת אנחנו צריכים למחוק גם את x והקשתות שיוצאות ממנו: ומכאן שזה יכול להיות תהליך ארוך מאוד, כל המחיקה הזו ברקורסיה שנפטרת מכל המבויים הסתומים. נגדיר זאת פורמלי:

הגדרה 145. (מבוי סתום) מבוי סתום הוא אחד מהמצבים הבאים:

- קודקוד $v \neq t$ כך $deg_{out}(v) = 0$
- קודקוד $v \neq s$ כך $deg_{in}(v) = 0$

עדכון של גרף השכבות L בעקבות מחיקה של קשת: כל עוד קיים מבוי סתום כלשהו, מחק אותו ואת כל קשתותיו. על פניו, זה נשמע יקר מאוד. אך נבחין כי כל קודקוד נהיה מבוי סתום פעם אחת לכל היותר בכל הפאזה. כיוון, שברגע שנמחק אותו הוא בהכרח לא יחזור להיות מבוי סתום וכל קשת הרי נמחקת פעם אחת. לכן ידנס סה"כ $O(|V|)$ קודקודים ו- $O(|E|)$ קשתות שנמחקו. ולכן ההחלטה כיצד למחוק הרי שהיא לוקאלית - אין צורך בסריקה נוספת של הגרף בעת שמצאנו מבוי סתום. לכן: עלות העדכונים של L וביצוע המחיקות פר פאזה אחת עלותו $O(|E|)$.

הגדרה 146. פסאודו קוד של דיניץ.

```

DINIC( $G = (V, E), s, t, c$ )
1  initialize  $f(u, v) = 0$  for all  $u, v \in V$ 
2  while there exists a path from  $s$  to  $t$  in  $G_f$ 
3      build layer graph  $L$ 
4      while there exists a path  $P$  from  $s$  to  $t$  in  $L$ 
5          augment  $f$  on  $P$ 
6          update  $L$  by continuously removing dead-ends

```

משפט 147. זמן הריצה של דיניץ הוא $O(|V|^2|E|)$.
הוכחה: בדומה לשיטה של פורד פרקלסון, הזרימה מאותחלת בהתחלה להיות $f = 0$. בתחילה מאתחלים בעלות $O(|V|^2)$. ההבדל בין פורד לדיניץ הוא בפציאת מסלול השיפור. בהמשך: נכנסים ללולאת *while* כל עוד יש מסלול s ל- t ב- G_f . בכל פאזה (זה *while*) של האלגוריתם - נבחין שישנו $O(|V|)$ כאלו:

1. בויים גרף שכבות - בעלות $O(|E|)$ כפי שכבר הבנו.
2. מתחילים באיטרציה: כל עוד ישנו מסלול בגרף השכבות (עלות לבדיקה $O(|V|)$), אם מצאנו, משפרים (עלות לשיפור $O(\ell)$) על המסלול עם צוואר הבקבוק.
3. לאחר ששיפרנו - מעדכנים את L : מורידים מבוים סתומים. זה עולה $O(|E|)$ **פר פאזה**.
נסמן ב- $k \leq O(|E|)$ את מס' האיטרציות בכל פאזה. מכאן שכל איטרציה עלותה $O(\ell)$ למציאת מסלול ושיפורו ועוד $O(|E|)$ לעדכון מבוים סתומים ולכן עלות פאזה היא $O(|E| + k \times O(\ell))$. כמה פאות ישנן? כפי שכבר אמרנו: $O(|V|)$. מכאן שזמן הריצה:

$$|V| \times (|E| + k \times \ell) \leq |V| \times (|E| + |E||V|) = O(|V|^2|E|)$$

האלגוריתם של Hopcroft – Karp

הגדרה 148. הגדרת הבעיה:

קלט: רשת זרימה עם פונקציית קיבול על הצמתים. נגדיר פונקציה $b : V \setminus \{s, t\} \rightarrow \mathbb{R}^+$, ונגדיר לכל צומת $b(u)$ את הקיבול על הצומת.
פלט: זרימה מקסימלית.

מה שנעשה יהיה להגדיר גרף חדש:

$$G' = (V', E')$$

$$V' = \{S_{out}, S_{in}\} \cup \{u_{in}, u_{out} | u \in V\}$$

באשר, כל צומת תפוצל ל-2 צמתים: $u_{in} \cup u_{out}$, באשר הקיבול על הקשת $u_{in} \rightarrow u_{out}$ יוגדר להיות $b(u)$. כל מה שנכנס אל u כלומר בגרף החדש אל u_{in} , יוכל לעבור דרך הקשת $u_{in} \rightarrow u_{out}$ עם אילוף הקיבול המתאים. כמו כן, נגדיר:

$$C'(u_{in}, u_{out}) = b(u), C'(u_{out}, v_{in}) = c(u, v)$$

משפט 149. בגרף החדש מתקיים:

$$1. |V'| = 2|V| - 2$$

$$2. |E'| = |E| + |V| - 2$$

משפט 150. זרימה מקסימלית בגרף G' היא זרימה מקסימלית בגרף G עם הקיבולות על הצמתים.

הערה 151. מעתה אנחנו נניח כי הקיבולים שלמים $\forall u \in V \setminus \{s, t\} b(u) = 1$, ונראה כי נוכל למצוא אלגוריתם טוב יותר.

משפט 152. אם אורך הפס"ב G s אל t הוא x , אזי חסם על הזרימה הגדולה ביותר הוא לכל היותר $\frac{|V|-2}{x-1}$.

נשים לב כי דיניץ' מקרה פרטי של פורד פרקלסון ולכן החסם של $O(|f^*||E|)$ תקף. אם כן, $|f^*| \leq |V|$ כיוון שברשת זו הזרימה יכולה להיות לכל היותר $|V|$, ומכאן לפי דיניץ' הממש את אדמונס קארפ זמן הריצה הינו $O(|E||V|)$. אם כן - נרצה לשפר. מה קורה בפאזה של דיניץ'? בעת שמצאנו מסלול משפר, נמחקה קשת אחת. כאן, נמחקות יותר קשתות. כל מסלול שנבחר יהיה מהצורה:

$$s \rightarrow \dots \rightarrow \dots \rightarrow \dots \rightarrow t$$

כל קשת בקיבול 1 בין (u_{in}, u_{out}) תמחק לאחר הזרמה באיטרציה. ולכן - כל המסלול עצמו נמחק באיטרציה הזו (!). דרגה יוצאת של $u_{in} = 0$ ודרגה נכנסת של $u_{out} = 0$ ולכן הצמתים נמחקים וגם הקשתות שנוגעות בהם ומכאן שכל המסלול ימחק. כלומר: בדיניץ' רגיל, סך הזמן שלוקח בפאזה הינו $O(|V||E|)$ שכן בדיניץ' רגיל אנו מזרימים דרך מסלול, רק הקשת עם הקיבול הכי קטן נשרפת לגמרי ושאר הקשתות נשארות עם קיבול שיורי. הן יכולות להשתתף בהרבה מסלולים לאורך הפאזה. לכן פאזה אחת עולה שם $O(|E||V|)$. אם כן, כאן כל הקשתות במסלול נשרפות לגמרי - ולכן כל המסלול עצמו נמחק. המשמעות היא שכל קשת יכולה להשתתף במסלול אחד בדיוק בפאזה, ולכן כל קשת נבדקת פעם אחת בדיוק בפאזה - מה שעולה לפאזה $O(|E|)$. אם כן, מס' הפאזות הינו $O(|V|)$, ולכן סה"כ זמן הריצה יהיה $O(|V||E|)$. כלומר: בגלל הגדרת הגרף המסלולים זרים - מחיקת קשת משפיעה על מסלול כולו: קשת תהיה פעם אחת בדיוק בפאזה ואז לא תהיה שוב בניגוד לדיניץ' הרגיל. לכן דיניץ' ירוץ כאן בזמן $O(|V||E|)$.

הגדרה 153. (האלגוריתם). להלן פסאודו:

$Hopcroft - Karp(G = (V, E), b, c, s, t)$:

א. נגדיר את הגרף החדש G' כפי שתואר לעיל.

ב. נריץ את דיניץ' על $\sqrt{|V|}$ פאזות ראשונות

ג. נריץ את פורד פרקלסון על שאר הפאזות.

משפט 154. זמן הריצה של הופקרופט קארפ הוא $O(\sqrt{|V|} \times |E|)$.
הוכחה: הרעיון הוא להריץ את דיניץ על P פאזות. מכאן שזמן הריצה יהיה $O(P|E|)$. לפי למה קודמת, לאחר P פאזות אורך המק"ב s tl הוא לפחות P , כיוון שבכל פאזה המסלול הקצר ביותר גדל ולכן אורך המק"ב לפחות P . מכאן, אם אורך המק"ב הכי גדול הוא P אזי שנותר להזרים עוד $O(\frac{|V|}{P})$ יחידות זרימה. ולכן לאחר P פאזות נריץ את פורד פרקלסון, הרי מתקיים $|f^*| \leq \frac{|V|}{P}$ ולכן פורד פרקלסון עלותו $O(\frac{|V||E|}{P})$. כעת נגדיר את זמן הריצה:

$$T(P) = O(FK) + O(Dinic) = P \times |E| + \frac{|V| \times |E|}{P}$$

ואם נגזור נקבל:

$$T'(P) = |E| - \frac{|V| \times |E|}{P^2} = 0 \implies P^2 = |V| \implies P = \sqrt{|V|}$$

זיווג מקסימום בגרף דו צדדי

הגדרה 155. יהי $G = (V, E)$ גרף לא מכוון. תת קבוצה $M \subseteq E$ היא זיווג (או התאמה) של G אם $\deg_M(v) \leq 1$ לכל $v \in V$ כאשר $\deg_M(v)$ מסמל את הדרגה של v בגרף המושרה ע"י M : $G' = (V, M)$. קודקוד v שדרגתו $\deg_M(v) = 1$ נקרא קודקוד מזווג, ואילו קודקוד v שדרגתו $\deg_M(v) = 0$ נקרא קודקוד לא מזווג.

הגדרה 156. נאמר כי M זיווג מקסימלי - אם לא ניתן להוסיף לו קשתות. כלומר, לא קיים $M' \subset M$.

הגדרה 157. נאמר כי M זיווג מקסימום - אם לכל זיווג M' מתקיים $|M'| \leq |M|$.
אלגוריתם פשוט למציאת זיווג מקסימלי: מתחילים עם הזיווג הריק, עוברים על כל קשת $(v, u) \in E$. אם שני הצמתים פנויים מוסיפים את הקשת, אחרת לא מוסיפים. אלגוריתם חמדני ופשוט מאוד שירץ בזמן לינארי.

נרצה לתאר רדוקציה מבעיית זיווג מקסימום לבעיית זרימת מקסימום בגרף.

יהי $G = (L, R, E)$ גרף לא מכוון דו צדדי. נבנה רשת זרימה $G' = (V', E')$ באופן הבא:

$$V' = V \cup \{s, t\}$$

$$E' = \{(s, u) | u \in L\} \cup \{(u, v) | (u, v) \in E, u \in L, v \in R\} \cup \{(v, t) | v \in R\}$$

כמו כן, נגדיר את הקיבולת $\forall (u, v) \in E'$ להיות $C(u, v) = 1$ ולכל $(u, v) \notin E'$ כמובן $C(u, v) = 0$.

משפט 158. אם M הוא זיווג ב G אזי קיימת ב G' זרימה f בעלת ערכים שלמים שערכה $|f| = |M|$.
 ומעצ שני, אם f היא זרימה בערכים שלמים ב G' אזי קיים ב G זיווג M כך ש $|M| = |f|$.

הגדרה 159. אם לכל $(v, u) \in E$ מתקיים $c(u, v) \in \mathbb{N}$ אזי קיימת זרימת מקסימום בה לכל $(u, v) \in V \times V$ מתקיים $f(u, v) \in \mathbb{Z}$.

7 BMM, משולשים וקל כבד

הגדרה 160. אלגוריתם קומבינטורי אינו מוגדר היטב. אך, אלגוריתם שאינו קומבינטורי הוא אלגוריתם שמשמש בפעולות חיסור: פעולות חיסור מגיעה לבטל משהו שביצענו יותר מדי. ולכן אלגוריתם קומבינטוריים שלא משתמשים באלו נוטים להיות מהירים ביחס למקדמי הקבועים והם גם לרוב אלגוריתמים פשוטים.

השערה 161. (השערת BMM) לכל קבוע $\varepsilon > 0$ לא קיים אלגוריתם קומבינטורי שזמן הריצה שלו הוא $O(n^{3-\varepsilon})$ לפתרון BMM.

הגדרה 162. הסימון $\tilde{O}()$ משמעותו שאנחנו מעוניינים באקספוננט בלבד. למשל: $O(\frac{n^3}{\log^4 n}) = \tilde{O}(n^3)$

הגדרה 163. יהי $G = (V, E)$ גרף. מסלול (v_0, v_1, v_2, v_0) יקרא משולש. כלומר, משולש הוא מעגל בגודל 3.

הגדרה 164. הגדרת הבעיה: יהי $G = (V, E)$ גרף לא מכוון.

פלט: ישנם מס' אפשרויות.

1. האם ב G יש משולש? (כעת נתמקד בזה)
 2. אם ב G יש משולש, מצא אחד שכזה. (בהמשך נסביר כיצד לפתור זאת בהינתן שפתרנו את 1)
 3. דווח על כל המשולשים שיש ב G . (נראה אלגוריתם קל כבד שעושה זאת)
- אנחנו נתמקד בבעיה 1: האם ב G יש משולש?

משפט 165. תהי M מטריצת שכנויות של גרף $G = (V, E)$. $\exists_{1 \leq i \leq n} M^3[i][i] = 1$ (באלכסון) $\Leftrightarrow$ ב G יש משולש.

משפט 166. אם קיים אלגוריתם קומבינטורי שפותר זיהוי משולשים בזמן $\tilde{O}(|V|^{3-\varepsilon})$ אזי קיים אלגוריתם קומבינטורי שפותר את BMM בזמן $\tilde{O}(n^{3-\frac{\varepsilon}{3}})$.

מסקנה: על סמך השערת BMM הקומבינטורית, לא קיים אלגוריתם קומבינטורי שפותר זיהוי משולשים בזמן $\tilde{O}(|V|^{3-\varepsilon})$.

משפט 167. תהי $T(X) = \Omega(X)$ פונקציה עולה. אם קיים אלגוריתם TD (זיהוי משולשים) שרץ בזמן $O(T(|V|))$ אזי קיים אלגוריתם שמצא את המשולש בזמן $O(T(|V|))$.

הוכחה: הרעיון יהיה הרעיון הבא: נקח את הגרף ונחלק אותו ל 4 חלקים בגודל $\frac{n}{4}$: V_1, V_2, V_3, V_4 . נבחין כי אם ישנו משולש בגרף - הוא לא יכול להיות ב 4 החלקים והוא נמצא כל פעם ב 3 מתוכם. לכן, אנחנו בכל שלב נוריד חלק אחד מן V_i ונבדוק האם קיים משולש באחד מ 4 החלקים: אם נמצא אחד כזה - באחת מן 4 האיטרציות נמצא משולש, נפטר בכל שלב מרבע מהגרף: ולכן נריץ שוב ברקורסיה. **להלן האלגוריתם:**

Algo:

לכל V_i : הרץ TD על $V \setminus V_i$. אם יש משולש - הפשך ברקורסיה והחזר את התשובה.
זמן הריצה: $\hat{T}(|V|) = 4T(\frac{3}{4}|V|) + \hat{T}(\frac{3}{4}|V|) = \hat{T}(\frac{3}{4}|V|) + O(T|V|)$ ולפי משפט האב נקבל כי $\hat{T}(|V|) = O(T(|V|))$.

נבחין כעת, כי בסוף נגיע ל 4 חלקים בגודל 1 (תנאי העצירה) ונמצא את המשולש הספציפי. סה"כ נשים לב שבאותו זמן של האם קיים משולש ניתן למצוא אותו גם, כדרש.

משפט 168. קיים אלגוריתם למציאת משולש בזמן $O(|E|^{\frac{2\omega}{\omega+1}})$
הוכחה: נזכר כי ראינו שניתן למצוא ב $O(|V|^\omega)$ בגרף כללי, האלגוריתם שנציג יהיה יעיל יותר עבור גרף דליל. נגדיר פרמטר τ . נאמר כי קודקוד u הוא קודקוד כבד אם $\deg(u) \geq \tau$. אחרת - נאמר כי u הוא קודקוד קל. נבחין כי מס' הקודקודים הכבדים הוא לכל היותר $\frac{2|E|}{\tau}$ ישירות מלמת לחיצת הידדים. נרצה לראות אלגוריתם שמאזן את שני הפקרים, קודקודים קלים וכבדים.
עבור u קודקוד קל (וסה"כ עבור הקלים): נעבור על כל זוגות השכנים $x, y \in N(u)$: לכל היותר $\deg^2(u)$ זוגות שכאלו. וכן, אם $(x, y) \in E$ נחזיר כן: ישנו משולש. (שכן, x שכן של u וגם y שכן של u וישנה קשת (x, y) וסה"כ משולש x, y, u). זמן הריצה של החלק הזה: $O(\sum_u \deg^2(u))$

עבור כל הכבדים יחד: יהי $\hat{G}$ הגרף שמושרה ע"י כל הקודקודים הכבדים. מתקיים $|\hat{V}| \leq \frac{2|E|}{\tau}$. נשתמש ב-FMM על $\hat{G}$ (נעלה בשלישית ונעזר בטענה שבאלכסון יש אחד אפי"מ משולש). נקבל זמן ריצה: $O((\frac{|E|}{\tau})^\omega)$. סה"כ, נריץ את שני האלגוריתמים יחד. נבחין כי ישנו שני סוגים של משולשים: משולש שכל הקודקודים בו כבדים. נגלה אותו בשלב של ה-FMM. אחרת, יש לפחות קודקוד אחד קל במשולש ונגלה אותו בשלב השני שנריץ עבור הקלים. סה"כ, זמן הריצה יהיה: $O(\sum_u \deg^2(u)) + O((\frac{|E|}{\tau})^\omega)$ וע"י חסימה יפה: $O(\sum_u \deg^2(u)) \leq O(\sum_u \tau \times \deg(u)) = \tau \sum_u \deg(u) = 2\tau|E|$ מכאן שזמן הריצה: $2\tau|E| + (\frac{|E|}{\tau})^\omega$ ואם נשווה את שני הביטויים נקבל $\tau = |E|^{\frac{\omega-1}{\omega+1}}$. לאחר קצת אלגברה נקבל שזמן ריצת האלגוריתם

$$(\frac{|E|}{\tau})^\omega = (\frac{|E|}{|E|^{\frac{\omega-1}{\omega+1}}})^\omega = (|E|^{1-\frac{\omega-1}{\omega+1}})^\omega = O(|E|^{\frac{2\omega}{\omega+1}})$$

דיווח כל המשולשים בגרף

הגדרה 169. הגדרת הבעיה:

קלט: גרף $G = (V, E)$ לא מכוון

פלט: כל שלשות הקודקודים (u, v, w) כך $(u, v), (v, w), (w, u) \in E$

פתרון ראשון:

$Algo1(G = (V, E))$:

א. לכל $u, v, w \in V$

ב. בדוק אם $(u, v) \in E \wedge (v, w) \in E \wedge (w, u) \in E$ אם כן זה המצב דווח על (u, v, w) כמשולש.

קל לראות שהאלגוריתם מבצע את הנדרש, וזמן הריצה שלו $O(\binom{|V|}{3}) = O(|V|^3)$.

פתרון שני:

$Algo2(G = (V, E))$:

א. עבור כל $w \in V$

ב. עבור כל $(u, v) \in E$:

1. אם $(v, w) \in E \wedge (u, w) \in E$ דווח על u, v, w

אכן האלגוריתם מבצע את הנדרש, וזמן הריצה כמובן $O(|V||E|)$.

פתרון שלישי:

ב מבחינה מתמטית, קודקודים w שמהווים משולש יחד עם u ו- v הם בדיוק הקודקודים שהם שכנים גם של u וגם של v . כלומר, $w \in \Gamma(v) \cap \Gamma(u)$. אם הגרף מיוצג ע"י רשימת שכנויות, וכל הרשימות ממוינות לפי סדר גלובלי של קודקודי הגרף, אזי ניתן לחשב חיתוך כזה במעבר אחד מסונכרן על שתי הרשימות שעלותו $O(\deg(v) + \deg(u))$. להלן אלגוריתם שמנצל תובנה זו-

$Algo3(G = (V, E))$:

א. עבור על כל הקשתות $(v, u) \in E$:

1. חשב את החיתוך $\Gamma(v) \cap \Gamma(u)$

2. לכל $w \in \Gamma(v) \cap \Gamma(u)$ דווח על המשולש (u, v, w)

הנכונות מיידית, וזמן הריצה עלותו $O(\sum_{(u,v) \in E} (\deg(u) + \deg(v)))$

הגדרה 170. יהי $G = (V, E)$ גרף לא מכוון. קודקוד $v \in V$ נקרא קל אם מתקיים $\deg(v) \leq \sqrt{|E|}$. אחרת, $\deg(v) > \sqrt{|E|}$ וקודקוד זה נקרא כבד.

משפט 171. בכל גרף ישנם לכל היותר $2\sqrt{|E|}$ קודקודים כבדים.

כעת, נחלק את המשולשים לשני סוגים:

א. משולש שמכיל קודקודים כבדים

ב. משולש שכל קודקודיהם קלים.

תחילה, נפעיל את האלגוריתם השני על הקודקודים הקלים בלבד. כלומר, נעבור על כל הזוגות של קודקוד כבד וקשת כלשהי ונבדוק אם הם יוצרים משולש. בשלב השני, נעבור על כל הקשתות (u, v) כך שגם u וגם v קלים, ונחתוך את רשימת השכנויות שלהם. (ושוב כמו באלגו 3 אנו נניח כי רשימת השכנויות נתונה כרשימת שכנויות ממוינת).

$Algo4(G = (V, E))$:

א. קבע לכל קודקוד $v \in V$ האם הוא קל או כבד.

ב. עבור על כל קודקוד כבד $w \in V$:

1. אם $(v, w) \in E$ וגם $(u, w) \in E$ דווח על המשולש u, v, w .

ג. עבור על כל הקשתות $(v, u) \in E$.

1. חשב את החיתוך $\Gamma(v) \cap \Gamma(u)$

2. לכל u, v, w המשולש $w \in \Gamma(v) \cap \Gamma(u)$ דווח על המשולש

זמן הריצה: השלב הראשון עלותו $O(|E|^{1.5})$ $O(2\sqrt{|E|} \times |E|) = O(|V|_{heavy} \times |E|)$, השלב השני עלותו

$$\sum_{(u,v) \in E} (deg(u) + deg(v)) \leq \sum_{(u,v)} 2\sqrt{E} \leq |E| \times 2\sqrt{|E|} = O(|E|^{1.5})$$

וסה"כ זמן הריצה הינו $O(|E|^{1.5})$.

מרחק האמינג עבור Σ כללי

נרצה לפתור את בעיית מרחק האמינג עבור א"ב כללי כלומר מצב בו $|\Sigma| = O(n + m) = O(n)$. באופן דומה, נחשב את מס' ההתאמות עבור על מיקום במקום מס' אי ההתאמות כיוון שניתן לעבור מההתאמות לאי ההתאמות ב $O(n)$ זמן.

באופן הנאיבי, עבור האלגוריתם שראינו קודם לכן אם נריצו עבור $|\Sigma| = O(n)$ נקבל זמן ריצה של $O(n^2 \log m)$.

רעיון נוסף: לשם הפשטות, נחשוב על מקרה בו בתבנית יש מופע אחד של a במיקום השלישי נניח. כעת, ניתן לספור את ההתאמות של a בין התבנית לטקסט באופן הבא: נבצע סריקה של הטקסט, ובכל פעם שנראה a במיקום i נוסף $+1$ למונה ההתאמות $M[i - 3 + 1]$ לפני האינדקס אותו אנו סורקים. אם יש שני תווי a במיקום x ו y , ניתן לבצע סריקה דומה כאשר כל פעם שרואים a מוסיפים $+1$ במיקומים $M[i - x + 1]$ ו $M[i - y + 1]$. נשים לב, שאם התו b נמצא בנוסף במיקום אחד במחרוזת, ניתן באותה סריקה של הטקסט למנות את מופעי a וגם מופעי b כיוון שכל פעם כשסורקים אות בטקסט צריך לבצע הוספות למונים רק לפי המופעים של האות הספציפית הזאת בתבנית. כלומר, $M[k]$ מציין כמה התאמות יש במיקום k כאשר מתחילים ב k .

באופן פורמלי, נסמן לכל $\sigma \in \Sigma$ את מס' המופעים של σ בתבנית כ $c(\sigma)$ ואת המופעים עצמם כאינדקסים $i_\sigma, i_{\sigma 2}, \dots, i_{\sigma c(\sigma)}$. האלגוריתם בשלב ראשון יבנה מבנה נתונים המכיל את כל האינדקסים הללו לכל האותיות ובשלב השני יבצע מעבר אחד על הטקסט ובכל מקום j כאשר האות המופיעה היא σ כלשהי יוסיף לכל המונים $1 + i_{\sigma c(\sigma)}, j - i_{\sigma c(\sigma)} + 1, \dots, j - i_{\sigma 2} + 1, j - i_{\sigma 1} + 1$. (וכך הוא יספור התאמות של כל σ עבור כל ההיסטים השונים).

אלגוריתם 6 מספר-התאמות-2 (T, P)

1. צור מערך I של מצביעים לרשימות מקושרות Σ .

2. עבור i מ-1 עד m בצע:

(א) הוסף את האינדקס i לרשימה המקושרת ב- $I[P[i]]$.

3. צור מערך M בגודל $n - m + 1$

4. עבור j מ-1 עד n בצע:

(א) עבור i ברשימה המקושרת $I[T[j]]$:

i. $M[j - i + 1] + 1 \rightarrow M[j - i + 1]$

5. החזר את M

אם מס' המופעים של כל אות בתבנית חסום במס' כלשהו c אזי זמן הריצה הכולל למניית מופעים של כל האותיות יהיה בסה"כ nc . במקרה שלא נתון שום חסם $c = O(m)$ וסה"כ ניתן לחסום את זמן הריצה של האלגוריתם ב- $O(nm)$. קיבלנו אלגוריתמים שרצים בזמנים $O(|\Sigma|n \log m)$ (הנאיבי) וכן $O(nm)$. נרצה לשלב אותם לאלגוריתם מהיר בהרבה - נחלק את האותיות בא"ב לשני סוגים לפי מס' המופעים של כל אות בתבנית. לשם כך נעזר בערך סף c אותו נקבע בהמשך.

הגדרה 172. אותיות שמופיעות יותר מ- c פעמים בתבנית יקראו כבדות. אותיות שמופיעות פחות מ- c פעמים בתבנית יקראו קלות.

משפט 173. מס' האותיות הכבדות בתבנית הוא לכל היותר $\frac{m}{c}$.

כעת, נספור תחילה את ההתאמות שנוצרות בין המופעים של האותיות הכבדות, בעזרת האלגו הראשון. סה"כ $O(\frac{nm}{c} \log m) = O(\frac{m}{c} n \log m) = O(|\Sigma|_{heavy} n \log m)$. אח"כ נספור את ההתאמות שנוצרים בין מופעים של אותיות קלות, בעזרת האלגוריתם השני: סה"כ $O(nc)$.

1. צור מערך חדש M בגודל $n - m + 1$ 2. צור מערך C בגודל Σ .3. צור מערך I של מצביעים לרשימות מקושרות בגודל Σ .4. עבור i מ-1 עד m בצע:(א) $C[P[i]] + 1 \rightarrow C[P[i]]$ (ב) הוסף את האינדקס i לרשימה המקושרת ב- $I[P[i]]$.5. לכל $\sigma \in \Sigma$ כך ש $C[\sigma] > c$ (א) ספור לכל היסט אפשרי של התבנית בטקסט את מספר ההתאמות של התו σ ע"י FFT של $T_\sigma \cdot P_\sigma^R$ והוסף את ההתאמות למערך M 6. עבור j מ-1 עד n בצע:(א) אם $C[T[j]] \leq c$ i. עבור i ברשימה המקושרת $I[T[j]]$:א'. $M[j - i + 1] + 1 \rightarrow M[j - i + 1]$ 7. החזר את M

זמן ריצת האלגוריתם יהיה $O(\frac{nm}{c} \log m + nc)$. אם נשווה נקבל $c = \sqrt{m \log m}$ ולכן זמן ריצת האלגוריתם $O(n\sqrt{m \log m})$.

אלגוריתמים רנדומיים

הגדרה 174. יהי אלגוריתם רנדומי A . אלגוריתם רנדומי הוא אלגוריתם שמשתמש במחרוזת אקראית r כקלט (פרט לקלט והפלט הרגילים). יש לכך פירושים שונים - בקורס אלגוריתמים 1: אנחנו מניחים כי כל מספר r הוא מספר אקראי שמתפלג באופן אחיד מטווח המספרים השלמים $[0, n]$ או $[1, n]$ באשר n הוא גודל הקלט של האלגוריתם A .

הגדרה 175. אלגוריתמי מונטה קרלו (*Monte - Carlo*) הם אלגוריתמים רנדומיים שזמן הריצה שלהם הוא דטרמיניסטי (ניתן לחסום אותו), אך הנכונות היא משתנה מקרי. כלומר, האלגוריתם עלול לטעות ולא להצליח. נרצה לנתח את הסיכוי לטעות: שיהיה כמה שיותר קטן. (לזכור - *correct* *Maybe*).

בהינתן גודל קלט n , נרצה כי הסיכוי לטעות $\frac{1}{n^\alpha} \geq$ עבור $\alpha \geq 1$ קבוע. אם אכן הסיכוי לטעות $\frac{1}{n^\alpha} \geq$ אזי אנחנו נאמר שהאלגוריתם צודק בסיכוי גבוה $1 - \frac{1}{n^\alpha}$.

הגדרה 176. אלגוריתמי לאס וגאס (*Las - Vegas*) הם אלגוריתמים רנדומיים שזמן הריצה שלהם הוא משתנה מקרי, והנכונות היא דטרמיניסטית. כלומר, האלגוריתם תמיד צודק אך זמן הריצה משתנה. באלגוריתמי לאס וגאס אנחנו נרצה לנתח את התכונות ההסתברויות של זמן הריצה. למשל: לחשב את תוחלת זמן הריצה, או חסם עליון לזמן הריצה בסיכוי גבוה.

וידוא כפל מטריצות

הגדרה 177. הגדרת הבעיה:

קלט: 3 מטריצות בינאריות מגודל $n \times n$. נסמן A, B, C .

פלט: לבדוק האם $C = A \times B$ כאשר הכפל הוא במודולו 2. (לא בינארי, אלא כפל מעל $\mathbb{Z}_2$).

להלן האלגוריתם:

VERIFY-BINARY-MM-BASIC(A, B, C)

```

1  Pick a random vector  $\vec{V} = (v_1, v_2, \dots, v_n)$  of bits
2   $\vec{V}_B \leftarrow B \cdot \vec{V}$ 
3   $\vec{V}_{AB} \leftarrow A \cdot \vec{V}_B$ 
4  if  $V_{AB} = C \cdot \vec{V}$ 
5      return "true"
6  else return "false"
```

האלגוריתם די פשוט וברור. ברור כי זמן הריצה הוא $O(n^2)$. באמצעות תכונות וקטורים, אם $AB = C$ אזי גם $ABv = Cv$ עבור וקטור $\vec{v}$ כלשהו. נבחין כי יתכן מצב שבו $AB \neq C$ אך $ABv = Cv$ למשל הוקטור $v = 0$ יקיים תמיד את השוויון. לכן, סיכוי לטעות עבורנו הוא מצב בו תחת ההשערה $D = C - AB$ המקיימת $D \neq 0$, אזי $Dv = 0$. נסמן $D = \{d_{ij}\}$, כיוון ש D שונה מאפס קיים לפחות $d_{ij} = 1$ אחד.

הגדרה 178. נאמר כי וקטור $\vec{v}$ הוא רע אם מתקיים $D \times \vec{v} = (C - AB) \times \vec{v} = 0$ (וקטור שכזה יגרום לאלגוריתם שלנו לטעות). אם $\vec{v}$ אינו רע, אזי נאמר כי $\vec{v}$ הוא טוב. כלומר: אם $D \times \vec{v} \neq 0$.

משפט 179. מספר הוקטורים הטובים הוא לפחות כמספר הוקטורים הרעים. (תחת הנחה ש $D \neq 0$)

מסקנה: אם בחרנו וקטור באקראי, והלפה אכן נכונה, אזי הסיכוי לבחור וקטור טוב הוא לפחות $\frac{1}{2}$. ולכן האלגוריתם שלנו יטעה בסיכוי לכל היותר $\frac{1}{2}$.

הוכחה: נתאר פונקציה חד חד ערכית שממפה וקטורים רעים לוקטורים טובים. מכאן שבהכרח יתקיים כי $|bad - vectors| \leq |good - vectors|$ לפי בדירה, ואז סייענו את ההוכחה.

יהי $\vec{v}$ וקטור רע. כלומר, $D \times \vec{v} = 0$. כלומר, $\forall 1 \leq \ell \leq n : \sum_{k=1}^n d_{\ell k} \times v_k = 0$. נזכר כי קיים $d_{ij} = 1$, כיוון שהנחנו $D \neq 0$ (אם אכן $D = 0$ אזי אין סיכוי לטעות). נגדיר וקטור

$$w_j = \begin{pmatrix} 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix} \quad \text{באשר הגדרת הוקטור הינה } 0 \text{ פרט למיקום } j \text{ (אותו } j \text{ פ } d_{ij} = 1 \text{) בו יהיה } 1. \text{ נזכיר.}$$

יצאנו מתוך נקודת הנחה ש $\vec{v}$ הינו רע (כלומר, הוא עבד עלינו). נראה כי הוקטור $\vec{v}' = \vec{v} + \vec{v}_j$ הוא וקטור טוב. כלומר, נוכיח כי $D \times \vec{v}' \neq 0$.

$$\vec{v}' = \begin{pmatrix} v'_1 \\ v'_2 \\ \vdots \\ v'_j \\ \vdots \\ v'_n \end{pmatrix} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_j \\ \vdots \\ v_n \end{pmatrix} + \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_j \\ \vdots \\ w_n \end{pmatrix} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_j + 1 \\ \vdots \\ v_n \end{pmatrix} \quad \text{נסמן}$$

לפי הגדרה כי -

$$(D \times \vec{v}')_i = \sum_{k=1}^n d_{ik} v'_k = \sum_{k=1}^n d_{ik} (v_k + w_k) = \sum_{k=1}^n d_{ik} v_k + \sum_{k=1}^n d_{ik} w_k$$

נשים לב כי $\sum_{k=1}^n d_{ik} v_k = 0$ כפי שהסברנו מעלה (כי הנחנו $D \times \vec{v}$ שווה לאפס, וטענו לגבי ℓ בפרט $\ell = i$ יתקיים). כלומר:

$$(D \times \vec{v}')_i = \sum_{k=1}^n d_{ik} w_k = d_{ij} w_j + 0 + 0 + \dots + 0 = 1 \times 1 = 1$$

באשר כל שאר הערכים הינם אפס פרט ל- $w_j = 1$, וכן $d_{ij} = 1$. סה"כ קיבלנו כי $(D \times \vec{v})_i = 1$ ולכן $D \times \vec{v}' \neq 0$.
סה"כ קיבלנו פונקציה:

$$f(\vec{v}) = \vec{v} + \vec{w}_j$$

נוכיח כי f חח"ע. יהיו V, W שני וקטורים כל $f(V) = f(W)$ אזי $V + \vec{w}_j = W + \vec{w}_j$ ולכן $V = W$ כנדרש.

הגדרה 180. אמפליפיקציה. אפליפיקציה היא שיטה נפוצה באלגוריתמים רנדומיים בה נריץ את האלגוריתם הבסיסי k פעמים, ועבור k מתאים (לרוב $\alpha \log(n)$ עבור α קבוע) נקבל שהסיכוי שהאלגוריתם יטעה לכל היותר $\frac{1}{n^\alpha}$.

משפט 181. קיים אלגוריתם שרץ בזמן $O(n^2 \log n)$ ובהסתברות גבוהה מוודא כפל מטריצות.
הוכחה: הרעיון הוא: נריץ את האלגוריתם הבסיסי k פעמים. נתבונן באלגוריתם הבא:

```

VERIFY-BINARY-MM( $A, B, C$ )
1   $k \leftarrow \alpha \log n$ 
2  for  $i = 1$  to  $k$ 
3      if VERIFY-BINARY-MM-BASIC( $A, B, C$ ) = "false"
4          return "false"
5  return "true"

```

אם באחת האיטרציות יוחזר לנו $false$ מהאלגוריתם הבסיסי, אז האלגוריתם יחזיר שקר. כלומר: בשביל שהאלגוריתם יטעה הוא צריך להחזיר "כן" לפרות שהמצב בפועל לא בפשך k ריצות. נסמן ב- A את המאורע לפיו בכל איטרציה אנחנו בוחרים וקטור רע. נבחין כי האיטרציות ב"ת זו בזו, ולכן אם נגדיר B כמאורע של הסיכוי לטעות באיטרציה אחת שהרי $Pr[B] \leq \frac{1}{2}$ כפי שראינו הרי ש- $Pr[A] = (Pr[B])^k \leq \frac{1}{2^k}$. נרצה כי זמן הריצה יהיה $\frac{1}{n^\alpha}$ עבור $\alpha \geq 2$ קבוע. זה יקרה כאשר $\frac{1}{2^k} = \frac{1}{n^\alpha}$ כלומר $n^\alpha = 2^k$ ומכאן $k = \log(n^\alpha) = \alpha \log(n)$. נפעצ זה אכן נקבל $Pr[A] \leq \frac{1}{2^k} = \frac{1}{2^{\alpha \log(n)}} = \frac{1}{n^\alpha}$.

מיון מהיר - Quick Sort

הגדרה 182. (אלגוריתם מיון מהיר). באלגוריתם בוחרים איבר אקראי a_r ויוצרים $partition$ סביב a_r . כל מה שמימינו יהיה גדול ממנו ומה שמשמאלו קטן ממנו. זה נפתר ברקורסיה. לאחר רקורסיה מקבלים מערך L מממויין, a_r בניהם וכן R מממויין: שרשור המערכים מניב מערך A מממויין. באלגוריתם הקלאסי r נבחר באופן אקראי אחד בטווח האינדקסים השלמים $[1, n]$. מדובר באלגוריתם לאס וגאס, שתמיד מצליח למיין. זמן הריצה של האלגוריתם נתון ע"י הנוסחה $T(n) = T(|L|) + T(|R|) + O(n)$, שחסומה ב- $O(n^2)$ במקרה הגרוע.

במקרה טוב עבורנו, a_r הוא החציון של A ולכן $|L| = |R| = \frac{n}{2}$ ונקבל $T(n) = O(n \log n)$.

הגדרה 183. (מיון מהיר פרנואידי). האלגוריתם דומה לאלגוריתם קוויק סורט המקורי. ומבצע בו שינוי פשוט מאוד - אם לאחר החלוקה מתקיים $|G|, |L| < \frac{n}{4}$ אזי האלגוריתם מבטל את בחירת $pivot$ הנוכחי ומחפש אחד חדש. זמן הריצה כאן כמובן משתנה מקרי. נראה כי נוסחת הנסיגה המתאימה לתיאור זמן הריצה הנ"ל כיוון שמדובר בעבודה על צד G, L ועל הזמן למציאת $pivot$.

$$\mathbb{E}[T(n)] = \mathbb{E}[T(|L|) + T(|G|) + T(pivot)] = \mathbb{E}[T(|L|)] + \mathbb{E}[T(|G|)] + \mathbb{E}[T(pivot)]$$

נבחין כי $|G| + |L| = n - 1$, וכן נבחין כי $T(pivot)$ הוא הזמן לחיפוש $pivot$ כפול $O(n)$, שהרי בכל חיפוש $Pivot$ אנחנו סורקים את המערך פעם אחת בשביל לגלות מי מעליו, מי מתחתיו - על מנת לדעת את גדלי הקבוצות. בחירת $pivot$ היא ניסוי שמתבצע שוב ושוב, מדובר במשתנה שמתפלג גאומטרי. נבחין כי $pivot$ רע הוא $pivot \notin [\frac{n}{4}, \frac{3n}{4}]$. הסיכוי כי $pivot$ יפול בתחום זה הוא $\frac{1}{2}$ ולכן מדובר במשתנה מקרי גאומטרי $pivot \sim Geo(\frac{1}{2})$ כלומר תוחלתו 2. ומכאן:

$$\mathbb{E}[T(n)] = \mathbb{E}[T(n - 1 - |G|)] + \mathbb{E}[T(|G|)] + 2 \times O(n)$$

כעת אם נגדיר פונקציה $T'(k) = \mathbb{E}[T(k)]$ ונסמן $|G| = x$. נקבל $T'(n) = T'(n - 1 - |G|) + T'(|G|) + 2 \times O(n)$.
 $\mathbb{E}[T(n)] \leq T'(\frac{3n}{4} - 1) + T'(\frac{n}{4}) + 2O(n) \leq T'(x) + T'(n - x) + 2 \times O(n)$
 $\mathbb{E}[T(n)] = O(n \log n)$ ולכן $2T'(\frac{3n}{4}) + O(n)$

משפט 184. תוחלת זמן הריצה של מיון מהיר רגיל היא $O(n \log n)$.
הוכחה: אלגוריתם קוויק סורט מבצע השוואות. נבחין כי מס' ההשוואות היא תוחלת זמן הריצה. אם כן, האלגוריתם לא משווה כל זוג איברים. מדוע? האלגוריתם מקבל כקלט את A ובוחר r כפיבט. אם בצד L ישנו a_i ובצד R ישנו a_j איברים אלו לא ישוו לעולם. מתי הוא כן משווה? הוא משווה זוג איברים אם ורק אם אחד מהאיברים נבחרו ל- $pivot$. נגדיר משתנה מקרי X_{ij} להיות משתנה אינדיקטור לכך שהאלגוריתם משווה בין זוג איברים i ו- j . כעת נבחין כי:

$$E[T(n)] = E[\sum_{i=1}^{n-1} \sum_{j=i+1}^n x_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$$

אם כן, לפי הגדרת משתנה אינדיקטור $\mathbb{E}[X_{ij}] = Pr[X_{ij} = 1]$. מהי ההסתברות הזו? כפי שאמרנו, איברים ישוו אם אחד מהם נבחר ל- $pivot$. מס' האיברים בטווח הוא $j - i + 1$, וכן כיוון שה- $pivot$ נבחר באקראי, ישנו סיכוי שווה לכל אחד מהאיברים להבחר. בפרט, הסיכוי הנ"ל יהיה $\mathbb{E}[X_{ij}] = Pr[X_{ij} = 1] = \frac{2}{j-i+1}$ כל שנותר מכאן הוא קצת אלגברה:

$$\begin{aligned} E[T(n)] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{j-i+1} = 2 \sum_{i=1}^{n-1} (\frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n-i+1}) \\ &\leq 2 \sum_{i=1}^{n-1} \sum_{\ell=1}^{n-j+1} \frac{1}{\ell} = 2 \sum_{i=1}^{n-1} \ln(n-i+1) \leq 2 \sum_{i=1}^{n-1} \log(n) = O(n \log n) \end{aligned}$$

מיון דלי (Bucket Sort)

נניח כי הקלט מיוצר בצורה אקראית. נבחר כי האלגוריתם הנ"ל דטרמיניסטי. נרצה לפתור את בעיית המיון כאשר כל מס' נבחר בהתפלגות אחידה מ- $[0, 1]$. האלגוריתם לוקח את טווח המספרים $[0, 1]$ ומחלק אותו ל- n דליים. כלומר $[0, \frac{1}{n}, \frac{2}{n}, \frac{3}{n}, \dots, \frac{n-1}{n}, 1]$. נתחיל "לזרוק" את האיברים אחד אחד. לכל דלי יכנסו מס' של איברים. נמיון את הדליים, ולבסוף נשרשרם.

מיון דלי ($A = (a_0, \dots, a_n)$)

- נכניס כל איבר לדלי המתאים (האיבר נבחר מראש אקראית!)
- נמיון כל דלי באמצעות מיון הכנסה
- נשרשר את הדליים לפי סדרם

משפט 185. תוחלת זמן הריצה של מיון דלי היא $O(n)$

הוכחה: נסמן $B_i = [\frac{i-1}{n}, \frac{i}{n}]$ הדלי ה- i . נסמן ב- n_i את מס' האיברים ב- B_i . נראה כי n_i הוא משתנה מקרי.

להכניס כל איבר לדלי הפתאים עולה $O(n)$ זמן. שרשרת הדליים גם כן יעלה $O(n)$ זמן. נראה כי השלב השני, מיון כל דלי באמצעות מיון הכנסה, יעלה לכל n_i $O(n_i^2)$ כי משתמשים במיון הכנסה. מכאן, זמן הריצה יהיה $O(n) + \sum_{i=1}^n n_i^2$. נסתכל על תוחלת זמן הריצה: $E[n + \sum_{i=1}^n n_i^2] = n + \sum_{i=1}^n E[n_i^2]$. נשים לב שהאלגוריתם לוקח n איברים וכל אחד מהם מצליח להכנס אל הבאקט B_i בסיכוי $\frac{1}{n}$. n_i סופר את מספר ההצלחות. כלומר: יש לנו ניסוי שחוזר n פעמים עם סיכוי הצלחה $p = \frac{1}{n}$. ספירת מס' ההצלחות היא התפלגות בינומית $n_i \sim \text{Bin}(n, \frac{1}{n})$. מכאן $E[n_i] = \frac{1}{n} \times n = 1$ וכן $Var[n_i] = np(1-p) = 1 - \frac{1}{n}$. נזכר כי $E[n_i^2] = Var[n_i] + E[n_i]^2 = 1 - \frac{1}{n} + 1^2 = 2 - \frac{1}{n}$. ומכאן נוכל לקבל כי $E[n_i^2] = 2 - \frac{1}{n}$. ולכן: $E[\sum_{i=1}^n n_i^2] = \sum_{i=1}^n E[n_i^2] = n + \sum_{i=1}^n (2 - \frac{1}{n}) = n + 2n - 1 = 3n - 1 = O(n)$

שבירת סימטריה

נתאר תרחיש שבו יש לנו זוג אנשים, אליס ובוב, שנמצאים בשיחת זום ורוצים לדבר זה עם זה. כל אחד מהם מעוניין לספר לשני על שמו - אך אם שניהם ידברו בו זמנית הם יעלו על הקול אחד של השני ולא ישמעו זה את זה. נבחינו: כל אלגוריתם דטרמיניסטי לא יפתור את הבעיה. מדוע? אלגוריתם דטרמיניסטי הוא קוד כתוב מראש, אם בוב ואליס ישתמשו שניהם בקוד הזה - הם יעשו אותו דבר בדיוק. לכן אנחנו חייבים להשתמש ברנדומיות. תאורטית, נוכל לתת להם מטבע להטיל, האפשרויות להטלת מטבע במקרה זה הם: $\{00, 01, 10, 11\}$. 0 יציין כי לא מדבר, 1 יציין כי מדבר. ברור שמצב טוב עבורנו הוא בו תוצאת ההטלות היא $\{01, 10\}$. מכאן הסיכוי להצלחה הוא $\frac{1}{2}$. מדובר במשתנה גאומטרי, ולכן תוחלת מס' הנסיונות עד להצלחה היא 2. בפרט, עבור $\alpha \log(n)$ סבבים, הסיכוי שלא להצליח יהיה $\frac{1}{n^\alpha}$ לכל היותר: קטן פולינומית. ניתן כמובן להרחיב את הבעיה, אם יש לנו שלושה אנשים כל אחד מהם ישדר בסיכוי שליש, הסיכוי שנצליח יהיה: $\frac{4}{9} = \frac{4}{3} \times (\frac{2}{3})^2 = \frac{4}{3} \times (\frac{2}{3})^2 \times (\frac{3}{3})^2$. ומה אם ממש נרחיב? כלומר, יהיו $x_1, \dots, x_n$ אליסים ובובים שלנו, מה הסיכוי להצלחה כעת?

$$Pr[Success] = \binom{n}{1} \times p \times (1-p)^{n-1}$$

גזירה פשוטה תראה כי $p = \frac{1}{n}$ ממקסם את הביטוי. ועם עוד קצת אלגברה, ההסתברות להצלחה תהיה לפחות $\frac{1}{e}$, ולכן תוחלת מס' הנסיונות עד להצלחה תהיה e (בערך).

הגדרה 186. המודל המבוזר המקומי הוא גרף לא מכוון $G = (V, E)$ בו כל קודקוד הוא מחשב שמריץ קלט מקומי. הכוח של כל קודקוד - מחשב, הוא אינסופי ובלתי מוגבל, ובפרט יכול לפתור בעיות NP בשניה. זמן יחולק לסבבים, ובכל סבב כל קודקוד יכול לשלוח הודעה גדולה כרצונו לכל

שכניו. במודל כל קודקוד מכיר את שכניו. אנחנו נניח כי לכל קודקוד אין id . סבב: חישובי מקומי פולינומי, בו יש שליחה וקבלה של הודעות. זמן הריצה תחת מודל זה יספר כמס' הסבבים. במודל זה: כל קודקוד מריץ את אותו האלגוריתם.

הגדרה 187. (בעיית הצביעה). יהי $G = (V, E)$ גרף לא מכוון. פונקציית צביעה $c : V \rightarrow \{1, \dots, \chi(G)\}$ נקראת פונקציית צביעה חוקית אם לכל $(u, v) \in E$ מתקיים כי $c(u) \neq c(v)$. כמובן שנרצה לצבוע את הגרף בכמה שפחות צבעים.

בעיית הצביעה ניתנת לפתרון בגרף דו צדדי ע"י 2 צבעים, בגרף n צדדי ע"י n צבעים. מדובר בבעיית NP , לא קיים אלגוריתם פולינומי לגרף כלשהו, ואף מאמינים שלא ניתן להגיע לכזה שכן בעיית הצביעה מקומית.

נניח שיש לנו קודקוד v ויש לו שכנים $v_1, \dots, v_k$. הבעיה של הקודקוד v הוא שהוא לא יודע כיצד שכניו פועלים. למשל: אם קודקוד v היה יודע ששכניו אינם שכנים אחד של השני - אזי קודקוד v היה רוצה לומר להם: תצבעו כולכם באותו הצבע, ואני אצבע את עצמי בצבע שונה משלכם. כרעיון בסיסי מאוד - אנו יכולים להחליט שהקודקוד v ישלח הודעה $(v_1, v_2, \dots, v_k)$ לכל שכניו. כמו כן: כל שכן ישלח הודעה לכל השכנים שלו עצמו. ואז - אני אקבל את ההודעות של כל שכניי, ואוכל להסתכל האם באחת ההודעות אני מזהה קודקוד שכבר יש לי. כלומר: האם השכנים שלי הם גם שכנים אחד של השני. נשים לב שקצת רימינו - שכן טענו כי לקודקודים אין id , אז איך נוכל לשלוח הודעה שכזו? נדבר על תהליך בחירת ID שקורה במקביל עבור כל הקודקודים.

בחירת ID :

1. כל קודקוד בוחר id בין המספרים $[1, 2, \dots, n^{c+2}]$ עבור c קבוע.
2. שלח ID לכל השכנים. (שכל אחד ידע את ID 's של השכנים שלו)

משפט 188. הסיכוי שישנם שני קודקודים עם אותו ID לאחר הרצת אלגוריתם "בחירת ID " קטן פולינומית.

הוכחה: סיכוי זה הינו $Pr[sameID] = 1 \times \frac{1}{n^{c+2}} = \frac{1}{n^{c+2}}$ עבור זוג קודקודים כלשהם, שהרי קודקוד אחד בוחר ID , והשני צריך להיות עם אחד זהה לשלו. יהי A_{uv} המאורע כי u ו- v בחרו את אותו ID . נבחין כי ע"פ חסם האיחוד מתקיים: $Pr[\bigcup_{u,v \in V} A_{uv}] \leq \sum_{u,v \in V} Pr[A_{uv}] \leq n^2 \times \frac{1}{n^{c+2}} = \frac{1}{n^c}$ וכן אם נרצה להסתכל על ההסתברות כי אף זוג קודקודים לא בחר אותו ID אזי $Pr[\bigcup_{u,v \in V} A_{uv}] = 1 - Pr[\bigcup_{u,v \in V} A_{uv}] = 1 - \frac{1}{n^c}$. כנדרש. סה"כ קיבלנו כי באמצעות טכניקה רנדומית פשוטה, יצרנו לכל קודקוד ID . מכאן: יש משמעות לשליחת הודעה לכל השכנים של רשימת השכנים.

הגדרה 189. אלגוריתם לצביעת הגרף באמצעות 2Δ צבעים

1. נבחר צבע באקראי מבין $[1, \dots, 2\Delta]$. נשים לב - ישנה כאן הנחה סמויה: כל קודקוד $v \in V$ מכיר את Δ .
 2. נשווה עם השכנים. ישנם שני מקרים -
 - א. אף שכן לא בחר את הצבע שאנחנו בחרנו: במקרה זה, אנחנו הצלחנו. נחליט שזה הצבע שלנו.
 - ב. אחרת, קיים לפחות שכן אחד שבחר את הצבע שאנחנו בחרנו, במקרה זה אנחנו ננסה לבחור שוב את הצבע. (נחזור ל-1).
- נראה את האלגוריתם עבור קודקוד יחיד $u \in V$ -

Color (Δ):

while(true):

-pick random color from $[1, \dots, 2\Delta] \rightarrow c$

- send c to neighbors

- receive colors of neighbors

-if there is no neighbor with color c so return and update $C(u) = c$.

נשים לב שבבדיקה אנחנו בודקים את כל השכנים של הקודקוד, ולא רק את אלו ש"אקטיביים" כרגע. כלומר - בודקים גם את אלו שסיימו לצבוע את הקודקוד שלהם.

משפט 190. מס' האיטרציות שעושה האלגוריתם בתוחלת הוא $O(\log n)$.

הוכחה: נרצה לבחון את הסיכוי כי הבדיקה בשורה 5 תצליח. כלומר, אין שכן שגם הוא בחר את c . לכל קודקוד דרגה $\Delta \geq d$, ולכן לא משנה איזה צבעים השכנים של קודקוד בחרו אזי יש לו לפחות $2\Delta - d$ צבעים פנויים. נסמן ב- x את מס' הצבעים הפנויים ברגע זה. אם כן:

$$Pr[\text{Success Line 5}] = \frac{x}{2\Delta} \geq \frac{2\Delta - d}{2\Delta} = 1 - \frac{d}{2\Delta} \geq_{\Delta \geq d} 1 - \frac{\Delta}{2\Delta} = \frac{1}{2}$$

נרצה לחסום את מס' הסיבובים שהאלגוריתם עושה עד לצביעה חוקית של כל הגרף. תהי V_i קבוצת הקודקודים שעדיין לא נצבעה לאחר i איטרציות של האלגוריתם. מתקיים $V_0 = V$ ונטען כי לכל $u \in V$ מתקיים כי $Pr[u \in V_i] \leq \frac{1}{2^i}$ כיוון שהסיכוי שהקודקוד עדיין לא נצבע, גורר שבמשך i איטרציות הוא נכשל, הסיכוי להכשל לכל היותר $\frac{1}{2}$ והאיטרציות ב"ת זו בזו, בפרט, $E[|V_i|] = \sum_{u \in V} Pr[u \in V_i] \leq \frac{n}{2^i}$. נבחין כי מתקיים $E[|V_i|] \leq \frac{1}{2} |V_{i-1}|$ (שכן לכל היותר מס' הקודקודים שישארו לעוד איטרציה, יהיו $\frac{1}{2}$ מאלו שהיו בקודמות, מכאן לפי אי שוויון מרקוב אם נחשב את הסיכוי שגודל קבוצה יהיה $\frac{3}{4} |V_{i-1}|$ אזי: $\frac{3}{4} |V_{i-1}| \leq \frac{E[|V_i|]}{\frac{1}{4} |V_{i-1}|} \leq \frac{\frac{1}{2} |V_{i-1}|}{\frac{1}{4} |V_{i-1}|} = \frac{2}{3}$. $Pr[|V_i| \geq \frac{3}{4} |V_{i-1}|] \leq \frac{2}{3}$ כלומר, המשלים שלו:

$$Pr[|V_i| < \frac{3}{4} |V_{i-1}|] = 1 - Pr[|V_i| \geq \frac{3}{4} |V_{i-1}|] \geq 1 - \frac{2}{3} = \frac{1}{3}$$

ומכאן שהסיכוי שבאיטרציה i הצלחנו לצבוע לפחות רבע מהקודקודים שלא היו צבועים קודם, הוא לפחות שליש. נסמן $p = \frac{1}{3}$. נגדיר איטרציה טובה כאיטרציה שהצליחה לצבוע לפחות רבע מהקודקודים שלא היו צבועים קודם. רעה, היא כמובן לא טובה. בהינתן k איטרציות טובות נותרו עם לכל היותר $(\frac{3}{4})^k n$ קודקודים. אנו רוצים להשאר עם פחות מקודקוד אחד ולכן: $(\frac{3}{4})^k \times n < 1 \implies n < (\frac{4}{3})^k \implies \log_{\frac{4}{3}}(n) < k$. מכאן: אם נקח $k = \log_{\frac{4}{3}}(n) + 1$ אזי שהצלחנו לצבוע את כל הגרף. אך לא סיימנו. נבחין כי כיוון שאנחנו יודעים שלא תמיד נקבל איטרציה טובה, אך ניתן לתאר את המשתנה המקרי למס' האיטרציות הטובות כגאומטרי עם $p = \frac{1}{3}$, אזי שבתוחלת כל $\frac{1}{p} = 3$ איטרציות נקבל איטרציה טובה. אנו זקוקים ל- k איטרציות על מנת לצבוע את כל הגרף, אך בין איטרציה לאיטרציה יהיו כאלו לא טובות. לכן סה"כ מס' האיטרציות שהאלגוריתם יבצע הוא $3k = 3 \log_{\frac{4}{3}}(n) + 3$. $O(\log(n))$ תוחלת הוא יעשה האלגוריתם יעשה בתוחלת הוא $O(\log(n))$.

הגדרה 191. (בעיית אוסף הקופונים). יהיו $x_1, \dots, x_n$ סוגי קלפים שונים. כל אחד מסוגי הכרטיסים מגיע מתוך אוסף של n אוסף כרטיסים מאותו סוג, בלתי מוגבל. מטרתנו היא למלא אלבום שמכיל מקום אחד לכל אחד מסוגי המדבקות. כל כרטיס הוא מאחד הסוגים בהתפלגות אחידה. כמה קלפים יש למשוך מהערימה עד שנמלא את האלבום?

משפט 192. תוחלת מס' הקלפים שיש להוציא בבחינת אוסף הקופונים היא $O(n \log n)$.

הוכחה: נסמן ב- X את מספר הקלפים שיש להוציא מהחבילה עד שנחזיק קלפים מכל הסוגים. לכל $1 \leq i \leq n$ את מס' X_i את מס' הקלפים שמוציאים מהרגע שיש בידינו $i - 1$ סוגי קלפים, עד הרגע שיש בידינו i סוגי קלפים. נבחין כי בהכרח $E[X_1] = 1$, כאשר יש בידינו $i - 1$ סוגי קלפים, בכל משיכה הסיכוי שנקבל קלף מסוג חדש הוא $\frac{n-i+1}{n}$. (רוצים מתוך n הקלפים שיצאו קלפים שאינם ה- $i - 1$ הראשונים). מדובר במשתנה גאומטרי, ולכן תוחלת מס' הקלפים שיש להוציא עד להוצאת קלף חדש היא $E[X_i] = \frac{n}{n-i+1}$, אם כן נקבל:

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{E}[X_i] = \sum_{i=1}^n \frac{n}{n-i+1} = n \sum_{i=1}^n \frac{1}{n-i+1}$$

אם כן, נסמן $j = n - i + 1$ ונקבל

$$= n \times \sum_{j=1}^n \frac{1}{j} = nH_n \approx n \ln n$$

לכן, תוחלת מס' הקלפים שיש להוציא עד להוצאת לפחות קלף אחד מכל סוג הינה $O(n \ln n)$.

משפט 193. זמן הריצה בסיכוי גבוה הוא גבוה פולינומית.

הוכחה: יהי X המשתנה המקרי המייצג את מספר הקלפים שנוציא עד שנראה קלף אחד לפחות מכל סוג. נסתכל על קלף i הסיכוי שנוציא את הקלף בכל אחד מהסיבובים הינו $\frac{1}{n}$. לכן הסיכוי שלא נוציא אותו הינו $1 - \frac{1}{n}$. הסיכוי שבמשך $k \ln n$ סיבובים לא נוציא את הקלף הינו: $(1 - \frac{1}{n})^{k \ln n} \leq \frac{1}{n^k}$

נסמן ב- X_i את המאורע שאחרי $k \ln n$ סיבובים טרם הוצאנו את קלף מספר i . המאורע שלא הוצאנו איזשהו קלף כלשהו ב- $k \ln n$ סיבובים הוא איחוד של כל ה- X_i עבור $1 \leq i \leq n$ ולכן לפי חסם האיחוד: $Pr[\bar{X}] \leq \sum_{i=1}^n Pr[X_i] = \frac{1}{n^k} \times n = \frac{1}{n^{k-1}}$ לכן נגדיר $k = c + 1$, הסיכוי שהוצאנו את כל הקלפים תוך $(c+1)n \ln n$ סבבים הוא לפחות $1 - \frac{1}{n^c}$, כלומר גבוה פולינומית.

הגדרה 194. (קבוצה פוגעת) נתון עולם של U איברים. ומספר $R < n$. בנוסף נתונות $k < n^{c_1}$ (כאשר c_1 קבוע כלשהו) קבוצות: $S_1, \dots, S_k \subseteq U$ כך שלכל $i \in [k]$ מתקיים $|S_i| \geq R$. קבוצה $A \subseteq U$ תקרא קבוצה פוגעת, אם לכל $i \in [k]$ מתקיים $S_i \cap A \neq \emptyset$. (כלומר, היא לא זרה לאף אחת מ- k הקבוצות).

בעיית מציאת קבוצת פוגעת בגודל מינימלי היא בעיית NP קשה - לכן לא סביר שנצליח לפתור אותה בזמן פולינומי. במקום זה, נראה כיצד למצוא קבוצה פוגעת בגודל קטן יחסית, בקלות רבה ובמהירות. נוכיח כעת שניתן למצוא בהסתברות גבוהה קבוצה פוגעת בגודל $O(\frac{n}{R} \log n)$.

משפט 195. בהינתן k קבוצות $S_1, \dots, S_k \subseteq U$ כך שלכל $i \in [k]$ מתקיים $|S_i| \geq R$, קיים אלגוריתם המוצא קבוצה פוגעת בהסתברות טובה של $1 - \frac{1}{n^c}$ בגודל $O(\frac{n}{R} \log n)$.

הגדרה 196. (בעיית בחירת פרמוטציה אקראית)

קלט: המספרים $\{1, \dots, n\}$

פלט: תמורה אקראית P של המספרים, כאשר הסיכוי לקבל כל אחת מהתמורות הוא בדיוק $\frac{1}{n!}$.

אלגוריתם ראשון: נגדיל באקראי לכל איבר את המיקום החדש שלו. בשביל לנסות למנוע התנגשויות, טווח זה יהיה גדול. נגדיל לכל מספר מס' אקראי ולבסוף נמייך את האיברים של המערך באמצעות סדר מיון שמושרה ע"י המפתחות של ההגרלה.

אלגוריתם 1 ערבוב-1(A)

1. $A.length \rightarrow n$

2. יהי $P[1..n]$ מערך חדש

3. לכל i מ-1 עד n בצע:

(א) $\text{Random}(1, n^3) \rightarrow P[i]$

4. מיינ את איברי A תוך שימוש באיברי P המתאימים כמפתחות

משפט 197. בהסתברות גבוהה של לפחות $1 - \frac{1}{n}$ לא יהיו שני איברים $i, j \in P$ כך ש $P[i] = P[j]$.
הוכחה. נסמן ב $A_{i,j}$ את המאורע ששני איברים מסויימים $i, j \in P$ מקבלים את אותו ערך. בהכרח כיוון שכל איבר מתקבל ע"י בחירות בתחום $[1, n^3]$ מתקיים $Pr(A_{i,j}) = \frac{1}{n^3}$. אם כן, נסמן $A = \bigcup_{i,j \in [n]} A_{i,j}$ את המאורע שקיימים שני אינדקסים כלשהם שמקיימים את אותו ערך ב P . מכאן לפי חסם האיחוד:

$$Pr(A) \leq \sum_{i,j \in [n]} Pr(A_{i,j}) = \binom{n}{2} \times \frac{1}{n^3} = \frac{n!}{(n-2)! \times 2 \times n^3} = \frac{n(n-1)}{2n^3} \leq \frac{n^2}{2n^3} = \frac{1}{2n} \leq \frac{1}{n}$$

הגדרה 198. בהנחה שכל הערכים שהאלגוריתם הגריל שונים זה מזה, מוחזרת תמורה אקראית בהתפלגות אחידה.

הוכחה: תהי $\pi = \langle \pi(1), \dots, \pi(n) \rangle$ תמורה. נרצה להוכיח כי הסיכוי שהאלגוריתם יוציא את π הוא בדיוק $\frac{1}{n!}$.

ההכללה לכל תמורה אחרת תעשה באופן דומה. אנחנו נדבר על תמורת הזהות. נסמן ב A_1 את המאורע שהתא $P[1]$ יקבל את הערך הכי קטן במערך, ובאופן כללי לכל $1 \leq i \leq n$ נסמן ב A_i את הסיכוי ש $P[i]$ יגריל את הערך i בגודלו מבין כל ערכי המערך. אם כן, המאורע בו קיבלנו את $\pi = \langle 1, \dots, n \rangle$ הוא בדיוק $A_1 \cap \dots \cap A_n$. אם כן,

$$Pr(A_1 \cap \dots \cap A_n) = Pr(A_1) \times Pr(A_2|A_1) \times Pr(A_3|A_1 \cap A_2) \times \dots \times Pr(A_n | \bigcap_{i=1}^{n-1} A_i) =$$

$$\frac{1}{n} \times \frac{1}{n-1} \times \frac{1}{n-2} \times \dots \times \frac{1}{1} = \frac{1}{n!}$$

משפט 199. זמן ריצת אלגוריתם ערבוב אחיד הוא $O(n)$.
הוכחה: כל הגרלה מתבצעת ב $O(1)$ זמן, וישנן n הגרלות ולכן סה"כ $O(n)$. כמו כן האיברים כמפתחות לאחר המיון הם בטווח $[1, n]$ ולכן מיון בסיס עליהם יעלה $O(n)$, סה"כ $O(n)$.

הגדרה 200. $(skip - list)$ רשימת דילוגים היא מבנה נתונים שנעזר ברנדומיות ואקראיות ותומך בשלוש הפעולות הבאות:
א. הכנסה של איבר חדש.

ב. חיפוש של איבר ע"פ המפתח שלו.

ג. הוצאת איבר קיים.

כלל הפעולות הללו נעשות בתוחלת זמן $O(\log n)$ או $O(\log n)$ זמן בהסתברות גבוהה. מיקום מבנה הנתונים הוא בתוחלת $O(n)$. מבנה הנתונים בנוי מרמות אותן נסמן $L_0, L_1, L_2, \dots$. כל רמה מכילה איברים, אשר שמורים כרשימה מקושרת ממיינת מהמספר הקטן ביותר עד המספר הגדול ביותר. לצורך נוחות, בכל אחת מהרשימות יופיעו המספרים $-\infty, \infty$ בשביל שתמיד נוכל לבצע השוואות להיות בטוחים שהאיבר שאותו נחפש לא נמצא בקצוות. בכל רגע נתון הרמה התחתונה L_0 מכילה את כל האיברים שבמבנה, וכל רמה מכילה תת קבוצה של איברים שנמצאים ברמה שמתחתיה. כלומר $L_0 \supseteq L_1 \supseteq L_2 \supseteq L_3 \dots$. כל הרמות מחזיקות את אבריהן בצורה ממיינת, וכל איבר מכיל מצביעים לאיבר הבא והקודם ברמתו ואל העותק שלו ברמה מתחת (פרט לרמה L_0 שהיא אין מתחתיה). נתאר את הפעולות על המבנה:

א. הכנסה: נחפש את מקומו של האיבר ברמה התחתונה (נתאר כיצד מחפשים מיד) ונכניס אותו במקום שהוא צריך להיות בו. לאחר מכן נטיל מטבע: אם יצא עץ - נוסיף עותק נוסף של המטבע ברמה מעליו. אם יצא פלי - נסיים. אם יצא עץ ברמה מעליו, שוב נחזור על התהליך ונטיל מטבע: מתי נפסיק? ברמה מסויימת שבה יצא פלי.

ב. חיפוש של איבר עם מפתח x : נתחיל מהרמה העליונה בצד שמאל, בכל שלב נלך ימינה ברמה, עד שנגיע לאיבר הגדול ביותר ברמה שקטן או שווה ל- x , אם הגענו ל- x עצמו נרד לעותק שלו מטה ב- L_0 באמצעות המצביעים וסיימנו. אחרת: נרד רמה אחת מטה, ונמשיך בסריקה ברמה מתחת מהאיבר אשר אליו הגענו וכך באותו אופן. כאשר נגיע לרמה התחתונה L_0 - אם x מאוחסן במבנה הנתונים אזי הוא בהכרח יופיע שם, אם לא נמצא אותו שם: סימן שאינו מאוחסן במבנה.

ג. מחיקה: נחפש איבר, כשנמצא נמחק אותו ואת כל עותקיו במבנה.

עבור ערך x שהוכנס למבנה, נאמר כי גובהו של x הוא המספר הגדול ביותר i כך $x \in L_i$. כלומר, $h(x) = \max\{i | x \in L_i\}$. הגובה של המבנה בכל רגע נתון הינו: $h_{SL} = \max\{h(x) | x \in L_0\}$

משפט 201. לכל איבר x מתקיים $\mathbb{E}[h(x)] = 2$
הוכחה: הגובה של כל איבר הוא פשתנה גאופטרי, עם הסתברות $\frac{1}{2}$ לשרוד" לשלב הבא. ולכן $\mathbb{E}[h(x)] = \frac{1}{2} = 2$

משפט 202. בהסתברות גבוהה מתקיים כי $h_{SL} = O(\log n)$
הוכחה: האיבר מגיע לגובה h ומעלה אפי"פ יצאו לו h הטלות רצופות של עץ. הסיכוי לכך הינו $\frac{1}{2^h}$ (ההטלות ב"ת). לכן, לפי חסם האיחוד:

$$Pr[h_{SL} \geq h] = Pr[\bigcup_{i \in [n]} h(x_i) \geq h] \leq \sum_{i \in [n]} Pr[h(x_i) \geq h] = \frac{n}{2^h}$$

לכן, עבור $h = c \log n$ נקבל כי $\frac{n}{2^{c \log n}} = \frac{1}{n^{c-1}}$, כלומר אכן $h_{SL} = O(\log n)$ בהסתברות גבוהה.

משפט 203. תוחלת גודל מבנה הנתונים (כפות האיברים בכל הרפות יחד) היא $O(n)$
הוכחה: פס' האיברים ברמה התחתונה הוא n . תוחלת פס' האיברים בכל רמה הוא חצי מהרמה שמתחתיה ולכן תוחלת פס' האיברים ברמה i הוא $O(\frac{n}{2^i})$. סה"כ נקבל:

$$\mathbb{E}[|sizeSL|] = \mathbb{E}[\sum_{i=1}^{\infty} |L_i|] = \sum_{i=1}^{\infty} \frac{n}{2^i} = 2n = \Theta(n)$$

פעולת חיפוש: נתבונן על סדרת הפעולות בחיפוש בכיוון ההפוך. סדרת החיפוש יכולה להיות מתוארת כמורכבת מתנועות ימינה בתוך רמה ותנועות למטה מרמה אחת לאחרת. זמן ריצת פעולת החיפוש הוא $\Theta(\log n)$ מס' הפעולות. מס' הרמות הוא בהסתברות גבוהה $O(\log n)$, לכן עלינו לחסום את סדרת התנועות. נסתכל על סדרת הפעולות מהסוף להתחלה מהאיבר האחרון - האיבר אותו חיפשנו. בכל צעד אנחנו נמצאים בקודקוד אחר במבנה, הצעד שלנו יהיה צעד כלפי מעלה (נסמנו $\uparrow$) אם לקודקוד זה יש עותק ברמה מעל, וצעד שמאלה (נסמנו $\leftarrow$) אם לקודקוד זה אין עותק ברמה מעל. כזכור, לכל קודקוד יש עותק ברמה מעליו אמנם בהטלת המטבע שלו התקבל עץ - שזהו מאורע בהסתברות חצי. נשים לב שהאלגוריתם מבצע צעד שמאלה רק אם אינו יכול לבצע צעד למעלה (אין עותק ברמה מעל), כלומר אם בהטלת המטבע של אותו הקודקוד התקבל פלי. כמות הצעדים למעלה שאלגוריתם החיפוש מבצע חסומה כמובן בגובה המבנה. כדי לחסום את כמות הצעדים שמאלה שאלגוריתם החיפוש מבצע, מספיק לחסום את כמות הפעמים שביקרנו בקודקוד שלא היה ניתן לעשות ממנו צעד למעלה במהלך אלגוריתם החיפוש, ולכן נרצה להוכיח את הלמה הבאה, שהרי אם כל צעד ברוורס הוא הטלת מטבע - הלמה הבאה תטען כי לעולם לא נתקע בצעדים שמאלה לנצח, כלומר המזל" יתאזן מהר מאוד ונצבור מספיק צעדים מעלה בשביל לכסות את כל גובה המבנה בזמן קצר.

משפט 204. בסדרת הטלות בלתי תלויות של מטבע הוגן, תוך $O(\log n)$ הטלות יתקבלו לפחות $c \log n$ הטלות של עץ, בהסתברות גבוהה.

הוכחה: נסתכל על סדרה של $10c \log n$ הטלות. נחשב את הסיכוי שיש בסדרה זו פחות מ- $c \log n$ הטלות עץ. מאורע זה זהה למאורע שישן לפחות $9c \log n$ הטלות פלי. יש $\Theta(\log n)$ בחירות של מקומות להטלת פלי. בהינתן בחירה של $9c \log n$ מקומות, הסיכוי שבכל המקומות הללו הוטל פלי הינו $2^{-9c \log n}$. מכאן לפי חסם האיחוד:

$$Pr[\text{there are less than } c \log n \text{ tails}] = Pr[\text{there are at least } 9c \log n \text{ heads}] \leq \binom{10c \log n}{9c \log n} \times \frac{1}{2^{9c \log n}} = \binom{10c \log n}{c \log n} \times \frac{1}{2^{9c \log n}}$$

לפי הנוסחה הבאה: $\left(\frac{n}{k}\right)^k \leq \binom{n}{k} \leq \left(e \times \frac{n}{k}\right)^k$ מתקיים:

$$= \binom{10c \log n}{c \log n} \times \frac{1}{2^{9c \log n}} \leq \left(e \times \frac{10c \log n}{c \log n}\right)^{c \log n} \times \frac{1}{2^{9c \log n}} = (10e)^{c \log n} \times \frac{1}{2^{9c \log n}}$$

$$= \left(\frac{10e}{2^9}\right)^{c \log n} < \left(\frac{1}{2}\right)^{c \log n} = \frac{1}{n^c}$$

ולכן, הסיכוי שיש לפחות $c \log n$ הטלות עץ הינו $1 - \frac{1}{n^c} \leq 1$, אכן סיכוי גבוה.

כעת, נעזר בלמה 204 לחסימת זמן החיפוש בתוחלת.

נסמן A את המאורע כי גובה המבנה הוא לכל היותר $c \log n$. נסמן B את המאורע שיתבצעו לפחות $c \log n$ צעדים $\uparrow$ בתוך סדרה של עד $10c \log n$ צעדים. נראה כי:

$$Pr[\overline{A \wedge B}] = Pr[\overline{A} \vee \overline{B}] \leq Pr[\overline{A}] + Pr[\overline{B}] \leq \frac{1}{n^{c-1}} + \frac{1}{n^c} \leq \frac{2}{n^{c-1}} \leq \frac{1}{n^{c-2}}$$

כלומר, ההסתברות למאורע זה היא די קטנה. מה מאורע זה אומר? ההסתברות שגובה המבנה הוא יותר מ- $c \log n$ (נעלה הרבה מעלה) או לחלופין שיתבצעו פחות מ- $c \log n$ צעדים (יטילו פחות מדי עץ מה שאומר שיהיה הרבה פלי תזוזה שמאלה) - היא חסומה ב- $\frac{1}{n^{c-2}}$. ולכן ההסתברות שפעולת החיפוש תצליח בזמן לוגריתמי די גבוהה $\leq \frac{1}{n^{c-2}} - 1$.

פעולת הכנסה תמיד מתחילה בפעולת חיפוש, כדי למצוא את מיקום האיבר המוכנס ברמה התחתונה. לאחר החיפוש נדרש ליצור עותקים נוספים ע"פ הטלות המטבע. כיוון שתוחלת מס' הרמות שיתווספו היא 2, ובהסתברות גבוהה האיבר לא יכנס ליותר מ- $O(\log n)$ רמות וכל הוספה לרמה לוקחת $O(1)$ זמן אזי $\Theta(\log n)$ ההכנסה לא לוקחת יותר מהחיפוש ולכן בהסתברות גבוהה עלותה $O(\log n)$ גם כן. עבור המחיקה - בדומה. (מתחילים בחיפוש ומוחקים מהרמות השונות).

8 חסמי Chernoff

משפט 205. אי שוויון מרקוב: אם X מ"מ אי שלילי, ויהי $k > 0$ אזי $Pr[X > k] < \frac{\mathbb{E}[X]}{k}$.

משפט 206. אי שוויון צ'בישב: אם X מ"מ (לא בהכרח אי שלילי), ונסמן $\mu = \mathbb{E}[X]$ וכן $\sigma^2 = \text{Var}[X] = \mathbb{E}[(X - \mu)^2]$, וכן $t > 0$ אז,

$$Pr[|X - \mu| \geq t] \leq \frac{\sigma^2}{t^2}$$

משפט 207. 2ℓ -moment: יהי X מ"מ כך ש $\mu = \mathbb{E}[X]$. אזי לכל $t > 0$ ולכל $\ell \geq 1$ שלם מתקיים:

$$Pr[|X - \mu| > t] < \frac{\mathbb{E}[(X - \mu)^{2\ell}]}{t^{2\ell}}$$

משפט 208. (אי שוויון צ'רנוף). יהי X סכום של משתנים מקריים אינדיקטורים $\{x_i\}_{i=1}^n$ באשר כל אחד מהם יכול להתפלג שונה, וכן כולם ב"ת. נסמן $X = \sum_{i=1}^n x_i$ ויהי $0 < \delta < 1$ אזי מתקיים:

$$Pr[X > (1 + \delta)\mu] \leq \left(\frac{e^\delta}{(1 + \delta)^{1 + \delta}}\right)^\mu$$

וכן:

$$1. Pr[X > (1 + \delta)\mu] < e^{-\frac{\mu\delta^2}{3}}$$

$$2. Pr[X < (1 - \delta)\mu] < e^{-\frac{\mu\delta^2}{2}}$$

דוגמה 209. נניח כי הטלנו מטבע $8\log(n)$ פעמים. מה הסיכוי שיצאו לפחות $\log(n)$ פעמים עץ?

נסמן ב X את מס' הפעמים שיצא עץ. נעזר במאורע המשלים (יהי $8\log(n) - X$ הטלות של פלי, נראה כי הסיכוי לפי אי שוויון מרקוב: נעזר במאורע המשלים) שווה לסיכוי שיצאו לפחות $7\log(n)$ פעמים (פלי) לכל שיצאו לכל היותר $\log(n)$ הטלות של עץ (המשלים) שווה לסיכוי שיצאו לפחות $7\log(n)$ פעמים

$$Pr[8\log(n) - X > 7\log(n)] < \frac{\mathbb{E}[8\log(n) - X]}{7\log(n)} = \frac{8\log(n) - \mathbb{E}[X]}{7\log(n)} =$$

$$\text{נבחין } \mathbb{E}[X] = \frac{1}{2} \times 8\log(n) = 4\log(n)$$

$$= \frac{8\log(n) - 4\log(n)}{7\log(n)} = \frac{4}{7}$$

לפי אי שוויון צ'בישב:

$$Pr[8\log(n) - X > 7\log(n)] = Pr[(8\log(n) - X) - 4\log(n) > 3\log(n)]$$

$$\leq Pr[|4\log(n) - X| > 3\log(n)] \leq \frac{2\log(n)}{(3\log(n))^2} = \frac{2}{9\log(n)}$$

שכן $*$ נכון כי נבחין שהשונויות הינה $2\log(n) = \frac{1}{2} \times \frac{1}{2} \times 8\log n$. נבחין כי השתפרנו כעת, עוד לא פולינומי אך השתפרנו.

לפי צ'רנוף:

$Y = 8\log n - X$. כמובן $\mathbb{E}[Y] = 4\log(n)$ כפי שראינו. נרצה לחשב חסם ל:

$$Pr[Y > 7\log(n)] = Pr[Y > (1 + \frac{3}{4})\mathbb{E}[Y]] < e^{\frac{-\mu\delta^2}{3}} = e^{\frac{-4\log(n)(\frac{3}{4})^2}{3}} = e^{-\frac{3}{4}\log(n)} = \frac{1}{n^\alpha}$$

עבור $\alpha > 1$ קבוע. מכאן שהסיכוי ל $\log(n)$ עץ גבוה פולינומית.
קבועים: נתבונן כי לבסוף עברנו אל $\frac{1}{n^\alpha}$ אך עלינו לוודא זאת (!) שאכן $\alpha > 1$ [אם זה לא היה עובד עם $8\log(n)$ ננסה $10\log(n)$ ואם לא מספיק אז ננסה $20\log(n)$ זה עדיין O של]. במבחן: עלינו בצד לוודא זאת שאכן $\alpha > 1$.

משפט 210. בסיכוי גבוה זמן הריצה של *Quick Sort* הוא $O(n\log n)$.

הוכחה: יהי עץ הרקורסיה של המיון. בידנו n איברים שנכנסו אל עץ הרקורסיה, וכל קודקוד בוחר מסלול משלו. יהי עלה בעץ הרקורסיה, נסמנו $a \in A$. נרצה לחשב את הסיכוי שהעומק של d_a של a הוא גדול - גדול ממש $10\log(n)$. לשם כך נגדיר:

איטרציה טובה היא איטרציה שבה כל קריאה רקורסיבית היא מגודל לכל היותר $\frac{3}{4}$ מגודל הקלט המקומי. הסיכוי לאיטרציה טובה הוא בדיוק $\frac{1}{2}$.

כמה איטרציות טובות ישנו? נבחין כי אם מס' האיטרציות הטובות $\log_{\frac{4}{3}}(n) < \log_{\frac{4}{3}}(n)$ אזי בהכרח נשארו 0 איברים.

מה ההסתברות שב $10\log(n)$ איטרציות שבהן a משתתף, לא היו $\log_{\frac{4}{3}}(n)$ איטרציות טובות? [שכן אם היו לפחות $\log_{\frac{4}{3}}(n)$ אזי סיימנו. לכן אנחנו מחשבים את המאורע המשלים]. תוחלת מס' האיטרציות הטובות הינו $5\log(n) = \frac{1}{2} \times 10\log(n)$. נחשב את הסיכוי שאנחנו רחוקים מהתוחלת. נסמן X כמשתנה מקרי של מס' האיטרציות הטובות (שהוא סכום של משתני אינדיקטור ב"ת שכן כל איטרציה ב"ת באיטרציה אחרת) עבור עלה כלשהו, נרצה לחשב את: $Pr[X < \log_{\frac{4}{3}}(n)]$. נרצה לחלץ את δ . כיצד? נזכר בחוק

$$\log_b(a) = \frac{\ln(a)}{\ln(b)}$$

$$\log_{\frac{4}{3}}(n) = (1 - \delta) \times 5\log(n) \implies 1 - \delta = \frac{\log_{\frac{4}{3}}(n)}{5\log(n)} = \frac{\frac{\ln(n)}{\ln(\frac{4}{3})}}{5\ln(n)} = \frac{1}{5\ln(\frac{4}{3})} \implies \delta = 1 - \frac{1}{5\ln(\frac{4}{3})} \approx 0.3$$

נבחין כי אכן $0 < \delta < 1$ וזה מתאים לתנאי צ'רנוף. לכן:

$$Pr[X < \log_{\frac{4}{3}}(n)] = Pr[X < (1 - \delta)\mu] \leq e^{\frac{-\mu\delta^2}{2}}$$

$$= e^{-\frac{5\ln(n)\delta^2}{2}} = e^{\ln(n \cdot \frac{-5\delta^2}{2})} = n^{\frac{-5\delta^2}{2}} = \frac{1}{n^{\frac{5\delta^2}{2}}} = \frac{1}{n^\alpha}$$

עבור $\alpha > 1$. מכאן, שבסיכוי גבוה של לפחות $1 - \frac{1}{n^\alpha}$ העומק של האיבר הנ"ל בעץ הרקורסיה הוא לוגריתמי (יהיו $\log_{\frac{4}{3}}(n)$ איטרציות טובות ונסיים איתו).

כעת נשאל מהו הסיכוי שקיים עלה עמוק כלשהו, מה הוכחנו עבור איבר ספציפי? הראינו כי עבור $a \in A$ כלשהו מתקיים:

$$Pr[\text{depth}(a) > 10\ln(n)] \leq \frac{1}{n^\alpha}$$

כלומר: הסתברותו להתקע בעומק גדול מדי די קטנה. מכאן עבור כל האיברים - יש לכל היותר n איברים (עלים פוטנציאליים). נרצה לדעת מה ההסתברות שקיים לפחות איבר אחד שהענף שלו ארוך מדי: זה יגרור עבורנו כי זמן הריצה איננו $O(n\log n)$. כלומר:

$$Pr[\exists a : \text{depth}(a) > 10\ln(n)] \leq \sum_{i=1}^n Pr[\text{depth}(a_i) > 10\ln(n)] \leq \frac{n}{n^\alpha} = \frac{1}{n^{\alpha-1}}$$

ולכן,

$$Pr[\forall a : \text{depth}(a) < 10\ln(n)] = \overline{Pr[\exists a : \text{depth}(a) > 10\ln(n)]} \geq 1 - \frac{1}{n^{\alpha-1}}$$

ישנה תלות בין האיטרציות. לכן בשלב זה השתמשנו בחסם האיחוד ולא בצ'רנוף. כלומר: ההסתברות שכל העץ יהיה בעומק לוגריתמי היא לפחות $1 - \frac{1}{n^{\alpha-1}}$ עבור $\alpha > 1$ קבוע כלשהו. כאשר עומק העץ הוא $O(\log n)$, בכל רמה מתבצעת לכל היותר $O(n)$ עבודה - השוואות (בעת *partiton*) אזי זמן הריצה הכולל של האלגוריתם (בהסתברות גבוהה) יהיה $O(n) \times O(\log n) = O(n\log n)$.

9 MST מתקדם

הערה. לא תימצתתי את זה כי אני יותר ממושכנע שזה יופיע במבחן והייתי ממליץ לדעת כל מה שכתוב כאן.

הנחה: קיים יחס סדר מלא על הקשתות, נקרא לפונקציה זו $ID(e)$ לכל קשת $e \in E$. בהינתן שיש יחס סדר מלא על הקשתות ניתן לומר $e < e' \iff (w(e), ID(e)) <_{Lex} (w(e'), ID(e'))$ (מסתכלים ראשית על המשקל, אם הקשתות הן באותו משקל אזי ישנה דרך לשבור סימטריה לפי ID). מסקנה: כל הקשתות במשקלים (לפי יחס הסדר) שונים).

מסקנה: הרצה של קרוסקל לפי יחס סדר זה תניב תמיד עץ פורש מינימום יחיד (שכן תמיד ישנו אותו סדר למיון בשל שבירת הסימטריה).

נזכר כי ראינו שתי תכונות עיקריות:

1. קשת קלה בחתך $\Leftarrow$ קיים MST שמכיל את e .
2. קשת כבדה במעגל $\Leftarrow$ קיים MST שלא מכיל את e .

בהקשר של יחס סדר המלא: קשת e שקלה בחתך (עם id הכי קטן) תהיה בכל MST . קשת e שביחס סדר מגיעה הכי מאוחר (עם id הכי גדול) לא תהיה באף MST .

9.1 Boruvka

מדובר באלגוריתם הראשון שהומצא ל- MST .

נסביר את הרעיון ללא פסאודו קוד:

מתחילים מאוסף של קודקודים של הגרף, ישנן קשתות בגרף אך לא מציירים אותן עדיין - כל קודקוד בוחר את הקשת הקטנה ביותר שלו לפי יחס הסדר. נבחין כי ישנם שני מקרים:

אם עבור קודקוד v הקשת הכי קטנה היא (v, u) והקשת הכי קטנה עבור u היא גם (v, u) אזי מעולה. אך זה לא בהכרח - יתכן של u יש קשת טובה יותר טובה.
נבחין: בהכרח לא ייווצר מעגל! בשלילה שיש מעגל אזי יש קודקוד ש 2 קשתות חלות בו, ונקבל סתירה ליחס הסדר במעגל (יהיה קודקוד שבחר בקשת הלא טובה ביותר שלו).

טענה: בעקבות יחס הסדר, הקשתות שנבחרו בסיום התהליך לא יצרו מעגל.

נוסיף את כל הקשתות ל MST , נסמנו E_T .
 בשלב הבא, נכוץ את הקשתות שנבחרו (ננסה לדמיין את התהליך ככה שבתחילה כל קודקוד הוא רכיב קשירות בפני עצמו. אם קודקוד 1 וקודקוד 2 חוברים בניהם כעת הם נחשבים לקודקוד אחד בלבד - 12 - קודקוד גדול!).
 נבחין כי אם קודם היו $|V|$ קודקודים, לאחר הכיווץ יהיה לכל היותר $\frac{|V|}{2}$ קודקודים (המקרה הכי טוב שכל שניים בחרו אותה קשת)
 נמשיך ברקורסיה עד למקרה הבסיס (קודקוד אחד) - יהיה לנו סה"כ $O(\log|V|)$ איטרציות.

מעט יותר פורמלית:

בכל איטרציה:

- כל הקודקודים בגרף הם מבודדים (או מגה הקודקודים שיצרנו)
- לכל קודקוד $v \in V$ מצא את הקשת e_v המינימלית המחוברת אליו.
- הוסף את כל הקשתות e_v שמצאת אל MST
- כיווץ: אגד את כל הקודקודים שחוברו ע"י הקשתות הללו ל"קודקודים גדולים" ומחק לולאות עצמיות (קשתות שחיברו בין קודקודים שכעת באותו קודקוד) - הקשתות שישארו לך בגרף הם רק קשתות שחיברו בין גושים של קודקודים גדולים.

נבחין כי עלינו למצוא דרך להפטר מהלולאות העצמיות - בכל כיווץ נפעיל DFS למצוא את הרכיבים הקשירים ואם קשת ששני קצותיה היו באותו רכיב מדובר כעת בלולאה העצמית שנפטר ממנה. זה לא יפגע בזמן הריצה אסימפטוטית. (הערה: המימוש הטכני תלוי לפרשנות וישנם דרכים רבות. למשל: בעת שהוא מאחד את הקודקודים הוא נותן להם ID חדש לכל אחד מהם, ובסיבוב הבא שיריץ DFS ויראה קשת ששני הקודקודים בה מרכיב עם אותו ID הוא יתעלם מהקשת והוא יכריז עליה כלולאה עצמית).

זמן הריצה:

נפרק את הניתוח לשניים:

1. כמה עולה סבב?

* בכל סבב כל קודקוד עובר על כל קשתותיו ומוצא את הקשת המינימלית. לכן $O(\sum_{v \in V} deg(v)) = O(|E|)$

*בשביל לדעת איזה קודקוד התאחד עם מי, מריצים $DFS \setminus BFS$ על הקשתות שבחרנו + עוברים על הקשתות ומוחקים את אלו שהפכו ללולאות עצמיות. סה"כ $O(|V| + |E|) = O(|E|)$
 לכן סבב עלותו $O(|E|)$

2. כמה סבבים ישנם?

נסמן ב V_i את מס' הקודקודים בשלב i . נבחין כי בהכרח $|V_i| \leq \frac{1}{2}|V_{i-1}|$ (במקרה הגרוע ביותר: כל זוג קודקודים מתאחדים לקודקוד אחד) ולכן מס' האיטרציות הינו $O(\log|V|)$
 לכן סה"כ זמן הריצה הוא $O(|E|\log|V|)$.

9.2 שיפור Yao

יאו ישב ואמר: בורובקה עשה רעיון יפה - אך למה שלכל קודקוד נסרוק את כל הקשתות שיוצאות ממנו? אם אנחנו יודעים כי קשת היא מאוד יקרה - אולי לא כדאי להסתכל עליה בסיבובים הראשונים.

יאו הסתכל על קודקוד $u \in V$ ועל כל הקשתות שיצאו ממנו (ישנן $deg(u)$ קשתות כאלו) וחילק את קשתות u ל k קבוצות ($Buckets$): נסמן $E_u^{(1)}, \dots, E_u^{(k)}$ ועבור כל קבוצה שכזו מתקיים כי גודלה הוא $\left\lceil \frac{deg(u)}{k} \right\rceil$. יחס הסדר צריך לקיים כי כל מי שבקבוצה הראשונה קטן ממי שבקבוצה השנייה, ומי שבשנייה קטן מהשלישית וכו'.

כיצד יוצרים את החלוקה הזו? נניח כי k חזקה של 2, ונשתמש באלגוריתם $Select$ למציאת החציון. מכאן נקבל שתי קבוצות (גדולים וקטנים מהחציון), ונמשיך ברקורסיה על כל צד (של אלו שגדולים מהחציון ואלו שקטנים ממנו). כך נמשיך ברקורסיה - עד שנקבל k קבוצות בדיוק. למעשה, כיוון שבכל שלב אנחנו מחלקים את הקבוצות ל2, מס' הפעמים שנצטרך לבצע את הרקורסיה הוא $\log(k)$. כמה עולה כל הרצה? אלגוריתם סלקט עולה זמן לינארי. כאן הלינאריות הינה $O(deg(u))$ - רק הקשתות של אותו קודקוד. לכן סה"כ זמן הריצה $O(deg(u) \times \log(k))$

כעת, קודקוד $u \in V$ צריך לחפש את הקשת הקלה ביותר שיוצאת ממנו. הוא לא צריך (בזכות עבודת העזר המקדימה) לעבור על כל השכנים שלו. אלא: רק על $E_u^{(1)}$. לכל קשת הוא יבדוק: האם הקשת מחברת אותי לקודקוד שנמצא איתי ברכיב? אם כן - מדובר בלולאה עצמית. האלגוריתם יזרוק את הקשת הזו וימשיך לקשת הבאה באותה חבילה. נבחין: הוא יעיף אותה לתמיד. אם לא - מצאנו את הקשת הרלוונטית. זו הקשת הכי זולה שיוצאת מקודקוד u . הוא יבחור אותה לסיבוב הנוכחי של בורבוקה ולא יפתח את שאר החבילות. הוא יעבור לחבילה הבאה - רק כאשר יסיים עם הקשתות בחבילה הנוכחית. הנכונות מגיעה מכך שבהכרח הקשתות בחבילות הבאות - גדולות מהקשתות בחבילה שלו. קודקוד יגדיר עבור חבילה שהיא כבר לא רלוונטית - אם ורק אם כל הקשתות שנמצאות בה גולו כלולאות עצמיות (קשתות שמחוברות לקודקוד באותו רכיב).

ניתוח זמן הריצה:

העלות מורכבת מ3 חלקים:

1. **עיבוד מקדים:** לכל קודקוד $u \in V$ חישוב ה $Buckets$ שלו: $O(\sum_{v \in V} deg(v) \times \log(k)) = O(|E| \log k)$
2. **סריקה של קשתות זבל:** נבחין כי קשת זבל (שהתגלתה כקשת עצמית) נסרקה כזבל בדיוק פעם אחת - כשמגלים שהיא זבל מסירים אותה. ולכן $O(1)$ פר קשת כלומר $O(|E|)$
3. **חיפוש קשת בכל סבב:** בכל אחד מהסבבים של בורבוקה כל קודקוד סורק חבילה אחת (או חלק ממנה). נבחין כי הקודקוד יודע באיזו קבוצה עליו לחפש כי הוא העיף כבר (חישבנו את החישוב סה"כ עבור האלגוריתם) את הקשתות שהפכו לזבל. גודל כל חבילה הינו $\left\lceil \frac{deg(v)}{k} \right\rceil$ ולכן עבודה עבור כל הקודקודים בסבב היא בעלות:

$$O\left(\sum_{v \in V} \left\lceil \frac{deg(v)}{k} \right\rceil\right) \leq \frac{|E|}{k} + |V| = O\left(\frac{|E|}{k} + |V|\right)$$

סה"כ ישנם $\log(|V|)$ סבבים אצל בורבוקה ולכן נקבל שזמן הריצה הינו:

$$\log(|V|) \times \left(\frac{|E|}{k} + |V|\right) + |E| + |E| \log k$$

נגדיר: $k = \lceil \log(|V|) \rceil$ ונקבל זמן ריצה:

$$O(|V|\log|V| + |E|\log(\log(|V|)))$$

המטרה: להפטר מ- $|V|\log|V|$.

נחלק את האלגוריתם ל-2:

1. נריץ את האלגוריתם הרגיל של בורבוקה (זה שעלותו $O(|E|)$ למציאת הקשתות סה"כ בסבב) למשך $\log(\log|V|)$ סבבים בלבד. עלות חלק זה: $O(|E|\log\log(|V|))$
- הבחנה:** כיוון שבכל סיבוב מספר הקודקודים קטן פי 2 (לפחות) נקבל כי אם נסמן ב- V' את גודל קבוצת הקודקודים לאחר ההרצות של בורבוקה אזי היא:

$$|V'| = \frac{|V|}{2^{\log(\log(|V|))}} = \frac{|V|}{\log(|V|)}$$

2. כעת, נריץ את האלגוריתם של Yao על הגרף שנותר עם V' . נבחין כי $|V'| \leq |V|$ והאלגוריתם של Yao יעלה לנו:

$$|V'|\log|V'| + |E|\log(\log(|V'|))$$

$$\leq \frac{|V|}{\log|V|} \times \log(|V|) + |E|\log(\log(|V|)) = O(|V|) + O(|E|\log(\log(|V|))) = O(|E|\log(\log(|V|)))$$

הסבר ל*: הפעלנו את $|V'| = \frac{|V|}{\log|V|}$ רק על החלק $|V'|$ הראשון והשתמשנו בכך שבהכרח $|V| \geq |V'|$ בשאר המיקומים של $|V'|$, ובחלק האחרון השתמשנו בכך ש- $O(|V|) = O(|E|)$ כי הגרף קשיר.

9.3 האלגוריתם של Karger – Klein – Tarjan (KKT)

כעת נראה אלגוריתם שרץ בתוחלת $O(|E|)$ זמן למציאת MST.

נסמן $F(\text{Forest})$ יער על V (חלק מהגרף $G = (V, E)$: מדובר ביער כלשהו של הגרף).
הגדרה: יהיו u, v באותו רכיב קשירות של F . נסמן ב- $P_F(u, v)$ את המסלול הפשוט היחיד בין u ו- v . (קיים מסלול יחיד בעץ)
הגדרה: $\max_F(u, v) = \max\{e \mid e \in P_F(u, v)\}$ (משקל הקשת הכבדה ביותר על המסלול הפשוט מ- u ל- v . כבדה ביותר ביחס ליחס סדר!)

הערה חשובה להמשך ומוטיבציה (אינטואיציה, זה אני הוספתי לא היה בהרצאה): אלגוריתם KKT מנסה להיות עצלן - במקום לעבוד קשה ולמצוא את ה-MST האמיתי של E הקשתות הוא דוגם חלק קטן מאוד מהקשתות. הוא ימצא את ה-MST שלהן (היער F) והוא ישתמש בו בכדי לסנן את שאר הקשתות בגרף המקורי. זה יהיה יער שנוצר מדגימה אקראית. למען האינטואיציה עוד: נניח כי יש לנו גרף ענק, נזרוק לכל קשת מטבע: אם יצא עץ - היא תכנס לקבוצה בשם E_s , אחרת לא תכנס. כעת

נמצא את ה-MST של הקבוצה הקטנה הזו, התוצאה היא יער שלו נקרא F .

הגדרה: קשת $e = (x, y)$ נקראת F כבדה אם:

1. x, y באותו רכיב לפי F (למה? אם הם לא מחוברים ב- F , אין לנו מסלול להשוות אליו ואז לא ניתן לומר בהכרח שהיא מיותרת).
 2. $e > \max_F(x, y)$.
- אחרת: נאמר כי e היא F קלה.

למה: e היא F כבדה $\iff e \in MST$. (זה חלק גדול מלב האלגוריתם. האלגוריתם אומר שלמרות שהשווינו אותה רק ליער חלקי F העובדה שהיא סגרה מעגל שבו היא כבדה מספיק בכדי לומר: את לא ב-MST בהכרח כי הוא יחיד)

קופסה שחורה: בהינתן F, V, E קיים אלגוריתם שמוצא את כל הקשתות ה- F כבדות בזמן לינארי $O(|E|)$.

טענה (משפט דגימה): בהינתן $G = (V, E)$, נייצר גרף אקראי $H = (V, E_S)$ כך שכל קשת $e \in E$ נבחרת בהסתברות $0 < p \leq 1$. יהי F ה-MST (יער פורש מינימום של H), אזי $\mathbb{E}[X] \leq \frac{|V|-1}{p}$ (באשר X מ"מ שסופר את מס' הקשתות E_S שהן F קלות).

הסבר רעיוני. כלומר, אם דגמנו יער כאשר קשת נכנסה אליו בהסתברות p , ונבנה מהן MSF , מס' הקשתות ה- F קלות (אלו שלא הצלחנו לזרוק) הוא לכל היותר $O(\frac{|V|}{p})$.

הוכחה:

נניח לצורך ההוכחה (זו לא באמת הדרך שבה מצאנו את F !) כי השתמשנו בקרוסקל על H על מנת למצוא את F . (זה לא אומר שהאלגוריתם שהרצנו הוא בהכרח של קרוסקל! אך כאן זה מותר להניח זאת שכן ישנו MSF יחיד ולכן כל האלגוריתמים יחזירו את אותו מבנה בהכרח). כלומר, קרוסקל מורץ על הגרף הדגום H .

נתבונן על הרגע שבו קרוסקל הוסיף קשת $e = (x, y)$ ל- F .

בזמן זה, נסמן A את הרכיב שמכיל את x ו- B את הרכיב שמכיל את y .

כעת אנחנו יכולים להניח כי הקשת e היא הקלה ביותר בחתך H שמחברת את A ו- B כלומר היא הקשת הקלה ביותר בחתך (A, B) . נתבונן על כל הקשתות בגרף המקורי! שחוצות את החתך בין A ל- B . חלק מהן נדגמו ל- H וחלק לא.

הקשת שקרוסקל בחר, היא הקשת הראשונה e מתוך הרשימה הנ"ל שאכן נדגמה ל- H . כמה קשתות היינו צריכים לדגום עד שנגיע לקשת הראשונה הזו? מדובר ב- $Geo(P)$ - בודקים עד להצלחה. תוחלת מספר הקשתות שנבדקו עד ההצלחה הראשונה היא $\frac{1}{p}$.

כל הקשתות שהיו לפני e ברשימה (אלו שלא נדגמו) הן בהכרח F קלות. מדוע? כאשר יער P יושלם, לא יהיה בינו לבין e מסלול שכולו קשתות יותר (הרי הן בהכרח לא נכנסו! היא הראשונה שנכנסה!)

הבחנה: כל הקשתות שמגיעות ביחס הסדר לאחר e הן בהכרח F כבדות.

הסבר: ברגע שקרוסקל הכניס את e ל- F , הוא חיבר את A ו- B . כל קשת אחרת e' בחתך (A, B) שגדולה מ- e : $e' > e$ סוגרת בהכרח מעגל עם הקשתות שכבר בתוך A ו- B (הרי קשת שחוצה את החתך (a, b) היא קשת כך ש- $a \in A, b \in B$ וכן אם נסמן את הקשת הקלה בחתך $e = (x, y)$ אזי A ישנו מסלול P_{ax} ו- B ישנו מסלול P_{yb} ולכן $P_{ax} \circ (x, y) \circ P_{yb} \circ (b, a)$ הוא מעגל: שבו הקשת e' היא בהכרח הקשת הכבדה ביותר - ולכן בהכרח $e' = (a, b)$ כך ש- $e' > e$ היא קשת שכבדה ביותר במעגל ולא נמצאת באף MSF . מדובר בקשת F כבדה!). ניתוח זה נכון לכל $e' > e$.

כעת, כל קשת שנכנסה ל- F (ישנן לכל היותר $|V| - 1$ כאלו) ייצגה בתוחלת $\frac{1}{p}$ קשתות לכל היותר שהיו רלוונטיות להיות פוטנציאליות קלות בחתך שלה. קשתות שלא היו בחתך של אף קשת הן בהכרח כבדות כפי שאמרנו. לכן e מגדירה $\frac{1}{p}$ קשתות קלות לכל היותר. לכן: מס' הקשתות הקלות בתוחלת לכל היותר חסום ב:

$$\mathbb{E}[X] \leq \frac{1}{p} \times (|V| - 1) = \frac{|V| - 1}{p}$$

9.4 פסאודו של KKJ

$:ALG(V, E)$

- א. אם $E = \emptyset$ החזר $\emptyset$.
 - ב. הרץ 2 סבבים של $Boruvka$ (נשארו $\frac{|V|}{4}$ קודקודים לכל היותר) לקבלת יער קשתות B .
 - ג. כוץ קשתות B ב- G . נסמן את הגרף המכוץ $G' = (V', E')$.
 - ד. כל קשת מ- E' מצטרפת ל- E_S בסיכוי $p = \frac{1}{2}$ (דגימת קבוצת הקשתות שנשארו לאחר הכיווצים).
 - ה. $F = ALG(V', E_S)$ (מדובר ביער פורש מינימום על הקשתות שדגמנו).
 - ו. מצא את כל הקשתות ה- F כבדות ב- G' (באמצעות קופסה שחורה) ומחק אותן מ- E' .
 - ז. נסמן ב- L את הקשתות שנשארו ב- G' (נותרו בתוחלת $|V'| \approx 2 \cdot \frac{|V| - 1}{2}$).
- הערה.** למעשה בדוגמה הקודמת כל מי שנכנסות אל L הן אלו שמתחת ל- e ואלו שלא הן בהכרח F ה- F כבדות שיועפו ולא יכנסו. נבחין כי בהכרח קשתות L מכילות MSF (ועוד קשתות!)
- ח. $F' = ALG(V', L)$.
 - ט. החזר $B \cup F'$.

האלגוריתם מחזיר לבסוף MST כיוון שהגרף במקור הוא קשיר מהנחתנו והיער הפורש המינימלי שלו הוא עץ.

הסבר:

- א. בבירור: אם אין קשתות, אין מה לחבר בגרף. תנאי עצירה.
- ב+ג. עושים מעין דיאטה" לקודקודים. אנחנו מריצים שתי הרצות של בורבוקה. מדוע דווקא 2? זה לא קריטי דווקא 2 - יכולנו גם 3 וכל אד חשוב לא הרצה אחת. זה יהיה חשוב לניתוח זמן הריצה בתוחלת בהמשך. נסמן ב- B את הקשתות שבורבוקה בחר: לפי תכונת החתך - הן בהכרח ב- MST (ונוכחותן של בורבוקה, אלו בהכרח קשתות שנמצאות ב- MST). נבחין כי לאחר שתי הרצות נקבל שבגרף שקבוצת קודקודיו (המכווצים) היא לכל היותר בגודל $\frac{|V|}{4}$. כעת: עוברים לעבוד על גרף הרבה יותר קטן מבחינת קודקודים.
- ד. כעת מבצעים דגימה: S . כל קשת מ- E' (אלו שנותרו בגרף המכוץ) תצטרף לגרף הדגימה E_S בהסתברות $\frac{1}{2}$.
- ה. מבצעים רקורסיה: מבקשים מהאלגוריתם למצוא את MSF של הקבוצה הקטנה הזו. שוב ושוב - עד שנגיע לכך שאין קשתות.
- בסיים שלב ה', בידינו MSF (יער פורש מינימלי) של הקשתות שדגמנו. F הוא לא MST של הגרף המקורי! הוא MST של חלק מהקשתות (בתוחלת, חצי מהקשתות) אותם בחרנו באקראי.
- ו. כעת, יש בידינו MSF שחזר, אנחנו יודעים לפי מה שטענו קודם כי הקשתות הכבדות לא יהיו בו. באמצעות הקופסה השחורה - אנחנו נמצא את כל הקשתות ופשוט נעיף אותן מ- E' . למעשה: אנחנו יודעים לזהות על הרבה קשתות כרגע שאינן ב- MST הסופי.
- בשלב זה - הקשתות מ- G' שלא נבחרו (בתוחלת חצי מהן לא נבחרו) חוזרות לתמונה. קודם לכן הרצנו יער בלעדיו, כאן הן חוזרות לתמונה.
- ז. כל הקשתות שנשארו לאחר המחיקה, יסומנו ב- L . אלו שלא הצלחנו לזרוק - ישנן בתוחלת $O(|V'|)$ כאלו, מדובר במספר דליל מאוד של קשתות! לינארי במספר הקודקודים.
- ח. בסוף, כשיש לנו גרף רזה מאוד, מריצים עליו את האלגוריתם עם קבוצת הקשתות שלא הצלחנו למחוק L ו- V' .
- ט. התוצאה מהרקורסיה תהיה MSF האמיתי של הגרף המכוץ.
- סה"כ: B אלו הקשתות שאנחנו יודעים שבהכרח ב- MST מנכונות בבורסקה, ו- F' זו תוצאת MSF על הגרף המכוץ (לאחר הרצת פעמיים של בבורסקה) ולכן סה"כ העץ שלנו מורכב מ- $B \cup F'$.

9.5 זמן הריצה

על מנת לנתח את זמן הריצה נסמן $|V| = n, |E| = m$:

$$T(n, m) = O(m + n) + T(n_1, m_1) + T(n_2, m_2)$$

נבחין כי כל הפעולות הן בזמן לינארי פרט לרקורסיות: פרט לזוג הרקורסיות כל העבודה שמתבצעת לינארית ותסומן ע"י $O(n, m)$.
 $T(n_1, m_1)$ הוא הקריאה הראשונה לרקורסיה ו- $T(n_2, m_2)$ הוא הקריאה השנייה לרקורסיה.
 נרצה לחשב את $\mathbb{E}[T(n, m)]$

$$\mathbb{E}[T(n, m)] = O(m + n) + \mathbb{E}[T(n_1, m_1)] + \mathbb{E}[T(n_2, m_2)]$$

נוכיח באינדוקציה כי $\mathbb{E}[T(n, m)] \leq 2 \times a(\overline{m} + 2\overline{n})$ באשר $\overline{x} = \mathbb{E}[X]$ וכן $a > 1$ קבוע מספיק גדול כדי לחסום את $m + n$ כלומר $(O(m + n) \leq a(m + n))$.

משפט 2.11. מתקיים $\mathbb{E}[T(n, m)] \leq 2a(\overline{m} + 2\overline{n})$
הוכחה: נוכיח את הטענה באינדוקציה על $n + m$.
בסיס: טריוויאלי.
צעד: נכוות לכל $n' + m' < n + m$ נוכיח עבור $n + m$.

$$\mathbb{E}[T(n, m)] = \mathbb{E}[a(m + n)] + \mathbb{E}[T(n_1, m_1)] + \mathbb{E}[T(n_2, m_2)]$$

$$\leq_* a\overline{m} + a\overline{n} + 2a(\overline{m_1} + 2\overline{n_1}) + 2a(\overline{m_2} + 2\overline{n_2}) =$$

* הוא הנחת האינדוקציה.
 נבחין כי $\overline{n_1}, \overline{n_2} \leq \frac{\overline{n}}{4}$ (באמצעות שני סבבי בורבוקה)
 וכן $\overline{m_1} = \mathbb{E}[|E_S|] = \frac{|E'|}{2} \leq \frac{|E|}{2} = \frac{\overline{m}}{2}$ (קשתות נכנסות בהסתברות חצי).
 וכן $\overline{m_2} \leq 2\overline{n_2} \leq 2 \times \frac{\overline{n}}{4} = \frac{\overline{n}}{2}$ (לפי משפט הדגימה).
 כעת נציב:

$$\leq a\overline{m} + a\overline{n} + 2a\left(\frac{\overline{m}}{2} + \frac{\overline{n}}{2}\right) + 2a\left(\frac{\overline{n}}{2} + \frac{\overline{n}}{2}\right) =$$

$$= 2a(\overline{m} + 2\overline{n})$$

כנדרש.

מסקנה: זמן הריצה של האלגוריתם $O(n + m) = O(m) = O(|E|)$ בתוחלת.